



400-650-7088

***Tong*Web** V7.0

应用开发手册

版本：7049_M9A01

2025 年 09 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

手册版本	适用于产品版本	更新内容
7049_M9A01	7.0.4.9_M9	无
7049_M8A01	7.0.4.9_M8	无
7049_M7A01	7.0.4.9_M7	无
7049_M6A01	7.0.4.9_M6	无
7049_M5A01	7.0.4.9_M5	无
7049_M4A01	7.0.4.9_M4	无
7049_M3A01	7.0.4.9_M3	无
7049_M2A01	7.0.4.9_M2	无
7049_M1A01	7.0.4.9_M1	无
7049A01	7.0.4.9	无
7048A01	7.0.4.8	无
7047A01	7.0.4.7	无
7046A01	7.0.4.6	无
7045A01	7.0.4.5	● 无
7044A01	7.0.4.4	● 将\${TW_HOME}改为\${TongWeb_HOME} ● 新增“9.2TransactionManager 的使用”章节
7043A01	7.0.4.3	无
7042A01	7.0.4.2	无
7041A01	7.0.4.1	● 统一使用TongWeb用户使用手册模板。 ● 整合原《TongWeb应用开发手册》

前言

本文档是TongWeb产品的用户使用手册之一，详细提供在TongWeb上开发应用的各类规范说明。

适合的对象

本手册主要适用于生产环境中的系统管理员，部分内容同样适用于应用开发人员和应用部署人员。

本手册假定您已经具备如下技能：

1. 基本系统管理任务；
2. 安装软件；
3. 使用Web浏览器；
4. 启动数据库服务器；
5. 在终端窗口中发布命令。

用户使用手册集

TongWeb为您提供的用户使用手册集包含的文档有：

- 000_TongWeb_V7.0用户手册导读：各版本手册集的目录及手册版本说明。
- 001_TongWeb_V7.0快速使用手册：提供各版本客户快速安装，应用部署，及常用配置的说明。
- 002_TongWeb_V7.0产品简介及安装指南：提供各版本各平台上的安装启动的步骤。
- 003_TongWeb_V7.0服务配置指南：提供Web容器、EJB容器、启动参数配置及各项基础服务的配置说明。
- 004_TongWeb_V7.0应用管理指南：提供管理控制台对应用进行部署和管理操作的说明。
- 005_TongWeb_V7.0工具使用指南：提供Eclipse、IDEA、Patch、TongAPM、appclient、pinpoint、web应用迁移等工具的使用说明。
- 006_TongWeb_V7.0Commandstool使用指南：提供命令行Commandstool的使用说明，基本参数及所有命令示例。
- 007_TongWeb_V7.0与Tivoli集成手册：对JSON整体结构及各监视项的说明。
- 008_TongWeb_V7.0二次开发接口：TongWeb的JMX、REST、SNMP接口说明。
- 009_TongWeb_V7.0应用开发手册：提供在TongWeb上开发应用的各类规范说明。
- 010_TongWeb_V7.0等级保护指南：提供等级保护功能开启后的使用说明。
- 100_TongWeb_V7.0集群管理指南：详细介绍集群的配置和管理。
- 200_TongWeb_V7.0用户使用手册-轻量级版：提供轻量级版本客户详细的服务配置及应用管理的说明。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

电话：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 操作系统及其版本
5. 产品版本号
6. 日志等错误的详细信息

目 录

第1章 Web应用	1
1.1 Servlet.....	1
1.1.1 Servlet3.0规范的新特性	1
1.1.2 Servlet3.1规范的新特性	2
1.1.3 调用Servlet的URL	6
1.1.4 ServletMapping.....	6
1.1.5 init-param.....	7
1.1.6 Filter.....	7
1.1.7 安全角色映射	10
1.1.8 编写HTTP Servlet	11
1.2 JSP	12
1.2.1 JSP指令	13
1.2.2 JSP动作	14
1.2.3 JSP用例	16
1.3 JSF	16
1.3.1 JSF中主要概念	17
1.3.2 JSF的生命周期	18
1.3.3 JSF用例	20
1.4 配置资源引用	23
1.4.1 支持环境条目(env-entry)	24
1.4.2 支持资源引用(resource-ref).....	25
1.4.3 支持资源环境引用(resource-env-ref)	25
1.4.4 支持EJB引用(ejb-ref)	26
1.5 Session管理.....	27
1.5.1 URL ReWriting	27
1.6 其他	27
1.6.1 Listener组件的开发	27
1.6.2 class loader delegate	28
1.6.3 使用WEB-INF下资源.....	28
1.6.4 编写自定义Valve	28
第2章 EJB应用	30
2.1 EJB2.0.....	30
2.1.1 EJB应用结构	30
2.1.2 SessionBean	30

2.1.3 访问EJB应用	32
2.1.4 在SessionBean中配置数据源.....	34
2.1.5 对weblogic-ejb-jar.xml的支持.....	35
2.1.6 Entity Bean	37
2.1.7 数据源配置	54
2.2 EJB3.0.....	54
2.2.1 SessionBean	55
2.2.2 定时服务	58
2.2.3 拦截器	60
2.2.4 依赖注入	62
2.2.5 Pool和Cache	64
2.2.6 配置资源引用	65
2.3 EJB3.1.....	65
2.3.1 Session Bean	67
2.3.2 Singleton Bean	67
2.3.3 定时服务	68
2.3.4 拦截器	69
2.4 EJB3.2.....	69
第3章 JPA.....	72
3.1 JPA2.0新增功能	72
3.2 JPA2.1新增功能	73
3.3 JPA的架构	76
3.4 容器托管EntityManager对象	78
3.4.1 通过@PersistenceContext注入.....	78
3.4.2 通过JNDI方式获取	78
3.5 应用托管EntityManager对象	79
3.5.1 EJB容器中获得	79
3.5.2 Web容器中获得	80
3.5.3 J2SE中获得	81
3.6 使用EntityManager	81
3.7 在JavaEE环境中使用JPA	82
3.7.1 使用步骤	82
3.7.2 使用示例	83
3.8 在J2SE环境中使用JPA.....	89
3.8.1 使用过程	89

3.8.2 使用示例	89
3.9 EntityManager的事务控制	93
3.10 JPA常用Annotation	93
3.11 persistence.xml及JPA功能属性	95
3.11.1 JPA常用功能属性	96
3.11.2 Openjpa JDBC属性	102
第4章 Dependency Inject	105
4.1 @inject.....	105
4.2 @Qualifier	105
4.3 Provider	106
4.4 @Named.....	107
4.5 @Scope.....	107
第5章 CDI	108
5.1 CDI新特性	108
5.2 CDI的使用	110
5.2.1 @Qualifier的补充	110
5.2.2 @Scope	111
5.2.3 @Produces与@Dispose	113
5.2.4 @Alternative	113
5.2.5 @Specialize	114
5.2.6 @Decorator	115
5.2.7 event	116
第6章 CommonAnnotation	117
6.1 @ManagedBean.....	117
6.2 @DataSourceDefinition(s)	117
第7章 Interceptor	118
7.1 @interceptor	119
7.2 @AroundTimeout.....	120
7.3 @InterceptorBinding	120
第8章 Validation	122
8.1 Bean Validation1.0.....	122
8.1.1 内置约束	122
8.1.2 自定义约束	123
8.2 Bean Validation 1.1.....	123
第9章 资源应用	127

9.1 UserTransaction	127
9.2 TransactionManager	128
9.3 TransactionSynchronizationRegistry	129
第10章 JMX	130
第11章 REST调用	130
第12章 CGI使用	130
12.1 启用CGI	130
12.2 Init参数配置	131
12.3 配置PHP示例	132
第13章 WebDAV使用	134
13.1 启用WebDAV	134
第14章 NativeJdbcExtractor工具类	136
附录A 术语及缩略语	137

第1章 Web应用

1.1 Servlet

Servlet是基于Java技术的Web组件，由容器管理并产生动态的内容。Servlet引擎作为Web服务器的扩展提供支持Servlet的功能。Servlet与客户端通过Servlet容器实现的请求/响应模型进行交互。具体开发过程中使用的Java Servlet API可参考Oracle公司提供的相关文档。TongWeb目前应用服务器支持Servlet3.1规范。

本章节中所有提到“参考TongWeb样例”的地方，都是指位于

`${TongWeb_HOME}/samples/servletjsp-samples/servlet3Jsp22-new-feature`目录下的应用

`Servlet3Jsp22Sample.ear`，可将其部署到TongWeb之后，访问其各个子模块，了解相应的特性及用法。

1.1.1 Servlet3.0规范的新特性

1. 新增的注解支持

`@WebServlet`

`@WebFilter`

`@WebListener`

`@WebInitParam`

`@MultipartConfig`

具体的注解使用可以参考TongWeb的样例`Servlet3Annotation.war`。

2. 可插性(pluggability)设计

使用该特性，可以在不修改已有Web应用的前提下，只需将按照一定格式打包的JAR文件放到WEB-INF/lib目录下，即可实现新功能的扩充，不需要额外的配置。

Servlet 3.0引入了称之为“Web模块部署描述符片段”的web-fragment.xml部署描述文件，该文件必须存放在JAR文件的META-INF目录下，该部署描述文件可以包含一切可以在web.xml中定义的内容。JAR包通常放在WEB-INF/lib目录下，除此之外，所有该模块使用的资源，包括class文件、配置文件等，只需要能够被容器的类加载器加载上即可。

具体参考TongWeb样例`Servlet3WebFragment.war`。

3. 异步处理的支持

Servlet接收到请求之后，可能会对数据进行一些预处理，之后Servlet线程将请求转交给一个异步线程来执行业务处理，线程本身返回，此时Servlet还没有生成响应数据。异步线程处理完业务以后，可以直接生成响应数据（异步线程拥有ServletRequest和ServletResponse对象的引用），或者将请求继续转发给其它Servlet。这样Servlet线程不再是一直处于阻塞状态以等待业务逻辑的处理，而是启动异步线程之后可以立即返回。

异步处理特性可以应用于Servlet和Filter两种组件。异步处理特性默认并没有开启，如果希望使用该特性，则必须按照如下的方式启用：

在web.xml中配置时，Servlet 3.0为<servlet>和<filter>标签增加了<async-supported>子标签，该标签的默认取值为false，要启用异步处理支持，则将其设为true即可。以Servlet为例，其配置方式如下所示：

```
<servlet>
<servlet-name>DemoServlet</servlet-name>
<servlet-class>com.tongweb.servlet.Demo Servlet</servlet-class>
<async-supported>true</async-supported>
</servlet>
```

使用Servlet 3.0提供的@WebServlet和@WebFilter注解进行Servlet或Filter配置的情况，这两个注解都提供了asyncSupported属性，默认该属性的取值为false，要启用异步处理支持，只需将该属性设置为true即可。以 @WebFilter为例，其配置方式如下所示：

```
@WebFilter(urlPatterns = "/demo",asyncSupported = true)
public class DemoFilter implements Filter{...}
```

此处，Servlet3.0还为异步处理提供了一个监听器，使用时只需要实现AsyncListener接口，用以捕获异步处理的成功、失败、超时等事件。具体可以参考TongWeb的样例Servlet3Async.war。

4. ServletContext 的性能增强

ServletContext对象支持在运行时动态部署Servlet、Filter和监听器，以及为Servlet和Filter增加URL映射等。参见TongWeb的样例Servlet3Programmatic.war。

5. HttpServletRequest 对文件上传的支持

HttpServletRequest提供了两个方法用于从请求中获取上传的文件：

```
Part getPart(String name)
Collection<Part> getParts()
```

第一个方法用于获取请求中给定名称的文件，第二个用于获取所有文件，每一个文件用一个javax.servlet.http.Part对象来表示。另外，可以结合前面提到的@MultipartConfig注解，来对上传操作进行自定义配置。比如限制上传文件的大小，设置保存路径等，请自行查阅相关文档。

1. 1. 2 Servlet3.1规范的新特性

1. 非阻塞 IO

Servlet 3.0中引入了Async Servlet，虽然servlet的执行过程可以是异步的了，但是从request读取和向response写入的过程，依然是同步的IO。而Servlet 3.1在Async Servlet的基础上，引入了Non-blocking IO Servlet，Web容器中的非阻塞请求处理有助于改善Web容器的可扩展性，增加Web容器可同时处理请求的连接数量。 servlet容器的非阻塞IO允许开发人员在数据可用时读取数据或在数据可写时写数据。IO的过程是通过WriteListener和ReadListener来实现的，类似于event driven的过程。

非阻塞IO仅对在Servlet和Filter中的异步请求处理和升级处理有效。否则，当调用ServletInputStream.setReadListener或ServletOutputStream.setWriteListener 方法时将抛出

IllegalStateException。

ReadListener为非阻塞 IO提供了下面的回调方法：

- **onDataAvailable()**
当可以从传入的请求流中读取数据时ReadListener的onDataAvailable方法被调用。当数据可读时容器初次调用该方法。当且仅当下面描述的ServletInputStream的isReady方法返回false，容器随后将调用 onDataAvailable方法。
- **onAllDataRead()**
当读取完注册了此监听器的ServletRequest的所有数据时调用onAllDataRead方法。
- **onError(Throwable t)**
处理请求时如果有任何错误或异常发生时调用onError方法。

容器必须线程安全的访问ReadListener中的方法。

除了上述ReadListener定义的方法外，下列方法已被添加到ServletInputStream类中。

- **boolean isFinished()**
与 ServletReader/ServletInputStream相关的请求的所有数据已经读取完时isFinished方法返回true。否则返回false。
- **boolean isReady()**
如果可以无阻塞地读取数据isReady方法返回true。如果没有数据可以无阻塞地读取该方法返回false。如果isReady方法返回false，调用read方法是非法的，且必须抛出IllegalStateException。
- **void setReadListener(ReadListener listener)**
设置上述定义的ReadListener，调用它以非阻塞的方式读取数据。一旦把监听器与给定的ServletInputStream关联起来，当数据可以读取，所有的数据都读取完或如果处理请求时发生错误，容器调用ReadListener的方法。注册一个ReadListener将启动非阻塞IO。在那时切换到传统的阻塞IO是非法的，且必须抛出IllegalStateException。在当前请求范围内，随后调用setReadListener是非法的且必须抛出 IllegalStateException。

WriteListener 提供了如下适用于容器调用的回调方法：

- **void onWritePossible()**
当一个WriteListener注册到ServletOutputStream时，当且仅当下边描述的ServletOutputStream的canWrite方法被调用并返回false，该容器将回调该方法。
- **onError(Throwable t)**
当处理响应过程中出现错误时回调。

除了WriteListener外，还有如下方法被添加到ServletOutputStream类，并允许开发人员运行时检查是否可以写数据发送到客户端。

- ServletOutputStream

- boolean canWrite()

如果往ServletOutputStream写会成功，则该方法返回true，其他情况会返回false。如果该方法返回true，可以在ServletOutputStream上执行写操作。如果没有后续的数据能写到ServletOutputStream，那么直到底层的数据被刷出之前该方法将一直返回false。且在此时容器将调用WriteListener的onWritePossible方法。随后调用该方法将返回true。

- void setWriteListener(WriteListener listener)

关联WriteListener和当前的ServletOutputStream，当ServletOutputStream可以写入数据时容器会调用WriteListener的回调方法。注册了WriteListener将开始非阻塞IO。此时再切换到传统的阻塞IO是非法的。

2. 协议升级

HTTP 1.1 (RFC 2616) 14.42节定义了HTTP升级机制，允许从HTTP 1.1过渡到其它兼容的协议。应用层的通信完全取决于升级后选择的协议。客户端和服务端完成协议升级后，后续请求都将使用新协议。典型的例子是如何将HTTP协议升级到WebSocket。

Servlet 容器提供了协议升级机制，但容器本身没有包含关于新协议的辅助类，新协议处理都封装到 HttpUpgradeHandler接口中进行。Servlet容器和HttpUpgradeHandler之间的数据读写是通过字节流。

下面的例子中，在 Servlet.service方法中决定是否升级协议，并使用到了一个新的方法：

HttpServletRequest.upgrade还有两个新的接口javax.servlet.http.HttpUpgradeHandler 和 javax.servlet.http.WebConnection：

```
if (request.getHeader("Upgrade").equals("echo")) {
    response.setStatus(HttpServletResponse.SC_SWITCHING_PROTOCOLS);
    response.setHeader("Connection", "Upgrade");
    response.setHeader("Upgrade", "echo");
    request.upgrade(MyProtocolHandler.class);
    System.out.println("Request upgraded to MyProtocolHandler");
}
```

请求中判断了Upgrade头信息，用于判断是否需要进行协议升级。如果Upgrade的值是echo，那么将会设置正确的响应status和头信息。request.upgrade方法将会传入HttpUpgradeHandler接口实现Class类。

当请求离开service方法之后，servlet容器处理完了所有的filter，然后标记当前请求需要由HttpUpgradeHandler的实例处理。

```
public class MyProtocolHandler implements HttpUpgradeHandler {
    @Override
    public void init(WebConnection wc) {
```

```

        //...
    }

    @Override
    public void destroy() {
        //...
    }
}

```

上面的代码是 `HttpUpgradeHandler` 接口的一个实现样例。Servlet 将调用 `HttpUpgradeHandler` 的 `init` 方法，并传入 `WebConnection` 对象，使 `HttpUpgradeHandler` 能访问到数据，当升级过程完成后，`HttpUpgradeHandler.destroy` 方法将会被调用。

3. SessionID 更改

与每个会话相关联是一个包含唯一标识符的字符串，也被称为会话 ID。会话 ID 的值能通过调用 `javax.servlet.http.HttpSession.getId()` 获取，且能在创建后通过调用 `javax.servlet.http.HttpServletRequest.changeSessionId()` 改变。

示例：新增了一个 `HttpSessionIdListener` 方法，用于监听 `sessionId` 被改变。

```

@WebListener
public class SessionIDChangeListener implements HttpSessionIdListener {
    @Override
    public void sessionIdChanged(HttpSessionEvent event, String oldSessionId) {
        System.out.println("[Servlet session-id-change example] Session ID " + oldSessionId + " has been changed");
    }
}

```

`SessionIDChangeTest` 类中，调用 `javax.servlet.http.HttpServletRequest.changeSessionId()` 来改变 `sessionId`。

```

@WebServlet(name = "SessionIDChangeServlet", urlPatterns = {"/SessionIDChangeServlet"})
public class SessionIDChangeServlet extends HttpServlet {
    protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        HttpSession session = request.getSession(true); try {
            StringBuilder sb = new StringBuilder();
            sb.append("<h3>Servlet SessionIDChangeTest at " + request.getContextPath() + "</h3><br/>");
        }
    }
}

```

```
sb.append("<p>The current session id is: &nbsp;&nbsp;&nbsp;"+ session.getId() + "</p>");
/* Call changeSessionID() method. */
request.changeSessionId();
sb.append("<p>The current session id has been changed, now it is: &nbsp;&nbsp;&nbsp;"+ session.getId()
+ "</p>");
request.setAttribute("message", sb.toString());
request.getRequestDispatcher("response.jsp").forward(request, response);
} finally {
out.close();
}
}
....
}
```

1. 1. 3 调用Servlet的URL

调用Servlet的URL的格式为http://server:port/context-root/servlet-mapping?name=value

表 1.1-1: URL元素说明

URL元素	说明
server:port	server表示IP地址或者主机名，port为可用的端口
context-root (应用访问前缀)	在企业应用中context-root定义在tongweb-application中，独立的Web应用中context-root由部署过程中的"应用前缀"决定。不论企业应用还是独立的Web应用，默认使用的context-root为war包的包名。如独立Web应用test.war默认的context-root为test，企业应用a.ear中Web应用b.war默认的context-root为b。
servlet-mapping	web.xml中定义的servlet-mapping或@WebServlet中指定的urlPatterns
name=value	可选的请求参数，name为参数名，value为参数值

调用Servlet的URL如：http://localhost:8080/test/testpage?a=8.0&b=360&c=180000

其中localhost为主机名，test为context-root，testpage是servlet-mapping，a、b、c为请求定义的参数。

1. 1. 4 ServletMapping

servlet-mapping用于控制如何访问Servlet，它往往在web.xml中体现。其中url-pattern体现访问Servlet的URLs。Servlet Mapping的例子如下：

```
<servlet>
<servlet-name>ServletServlet</servlet-name>
```

```
<servlet-class>twins.servlet.ServletServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>ServletServlet</servlet-name>
<url-pattern>/myservlet/*</url-pattern>
</servlet-mapping>
```

1. 1. 5 init-param

通过设定web.xml中Servlet类的<load-on-startup>标记，可使服务器在启动时初始化Servlet应用。

同时，通过设定包含此Servlet的init-param将初始化参数传递到此Servlet。

在web.xml中设定了一个名为"Test_name"，值为"test"的初始化参数，具体配置如下：

```
<servlet>
<servlet-name>HelloWorld</servlet-name>
<servlet-class>examples.servlets.HelloWorld</servlet-class>
<init-param>
<param-name>Test_name</paramname>
<param-value>test</param-value>
</init-param>
</servlet>
```

Servlet应用可通过调用getInitParameter(String name)方法可取得此参数。

1. 1. 6 Filter

Filter对象用来改变请求或者响应的报头和正文。与其他Servlet对象的区别在于，它自己不创建响应，然而它可以依附在任何类型的Web资源上。Filter的主要任务如下：

- 拦截并查询请求，然后让其正常工作。
- 阻塞请求和响应，不让它们继续被传递。
- 修改请求的报头和数据，替换为自己定制的请求。
- 修改响应的报头和数据，替换为自己定制的反应。
- 与外部资源交互。

利用Filter可以实现多种功能，常见的包括身份认证和授权，日志，图像转换，数据压缩等。

应用Filter的步骤包括两步：编写Filter类，Filter的声明。

1.1.6.1 编写Filter类

编写过滤器的API是javax.servlet包中Filter、FilterChain和FilterConfig接口中定义的一些方法。定义一个过滤器需要实现Filter接口。Filter接口中最主要的方法是doFilter()方法，它接收三个参数：Request对象、Response对象、FilterChain对象。

```
void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
```

doFilter方法能够执行的动作包括：

- 检查请求的头信息。
- 定制请求对象，改变请求的头信息或数据。
- 定制响应对象，改变响应的头信息或数据。
- 调用在过滤器链中的下一个实体。如果当前过滤器是链中的最后一个过滤器，那么下一个实体就是客户请求的资源，否则，链中的下一个过滤器会被调用。
- 检查响应的头信息。
- 抛出异常显示处理过程中的错误。

Filter为了改变响应，必须在它被返回客户端之前截获它。但是需要注意的是，在Filter链中的任何Filter都有可能关闭响应，结果导致响应立刻返回。Filter是串栈式的结构，所设置对响应的改变是在doFilter()函数调用之后，也就是栈底。可是响应对象还没有到栈底就被返回了怎么办？为此，必须传递给doFilter函数一个替身。怎么才能找到这个替身？需要创建一个响应的包装类，并且重载它的getWriter和getOutputStream函数，以返回这个替代的流。

标准的Servlet实现提供了现成的包装实现类：ServletRequestWrapper和HttpServletRequestWrapper，ServletResponseWrapper和HttpServletResponseWrapper。用户只需实现这四个中的一个，并覆盖响应的函数即可。下面的例子就是对响应的封装，它使得以后的Filter不能直接提交响应。

```
public class CharResponseWrapper extends
    HttpServletResponseWrapper {
    private CharArrayWriter output;
    public String toString() {
        return output.toString();
    }
    public CharResponseWrapper(HttpServletResponse response){
        super(response);
        output = new CharArrayWriter();
    }
    public PrintWriter getWriter(){
        return new PrintWriter(output);
    }
}
```

```

Filter类
public class CharFilter implements Filter{
    .....
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)throws
IOException,ServletException{
        PrintWriter out = response.getWriter();
        CharResponseWrapper wrapper = new
CharResponseWrapper((HttpServletResponse)response);
        chain.doFilter(request, wrapper);
    }
}
    
```

容器初始化Filter时，会传递给它一个FilterConfig对象，用来传递一些参数。这些初始化参数可以通过web.xml的init-param元素或@WebServlet的属性来定义。web.xml具体配置如下：

```

<filter>
<filter-name>Compression Filter</filter-name>
<filter-class>CompressionFilter</filter-class>
<init-param>
<param-name>compressionThreshold</param-name>
<param-value>10</param-value>
</init-param>
</filter>
    
```

1. 1. 6. 2 Filter的声明

在部署描述文件web.xml中加入<filter-mapping>元素，映射过滤器到Servlet应用，具体配置如下：

```

<filter-mapping>
<filter-name>Char Filter</filter-name>
<servlet-name>HelloWorldServlet</servlet-name>
</filter-mapping>
<servlet>
<servlet-name>HelloWorldServlet</servlet-name>
<servlet-class>HelloWorldServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>HelloWorldServlet</servlet-name>
<url-pattern>/HelloWorldServlet</url-pattern>
</servlet-mapping>
    
```

映射过滤器到URL：

```
<filter>
<filter-name>Char Filter</filter-name>
<filter-class>com.tongtech.examples.CharFilter</filter-class>
</filter>
<filter-mapping>
<filter-name>Logging Filter</filter-name>
<url-pattern>/servlet/*</url-pattern>
</filter-mapping>
```

说明：

servlet过滤器的执行顺序就是servlet过滤器在web.xml里面映射过滤器到URL(filter-mapping)的顺序。

1.1.7 安全角色映射

编写Servlet应用时通常会面临很多安全问题。当然在程序中加入大量的安全代码可以解决这个问题，但加入这样的安全硬编码将使程序变得不灵活，而且不易维护。下面介绍一种无需修改代码即可达到安全目的方法。

1. 在部署描述文件 web.xml 文件中定义安全约束。安全约束将保护响应的 URL，以使特定的用户才能访问它。web.xml 中安全约束的描述如下：

```
<security-constraint>
<web-resource-collection>
<web-resource-name>protected</web-resource-name>
<!-- 指定需要保护的资源的url(相对于应用前缀) -->
<url-pattern>/proteced/*</url-pattern>
<!-- 指定需要保护的方法 -->
<http-method>DELETE</http-method>
<http-method>GET</http-method>
<http-method>POST</http-method>
<http-method>PUT</http-method>
</web-resource-collection>
<auth-constraint>
<!-- 指定能访问受保护资源的角色 -->
<role-name>twns</role-name>
</auth-constraint>
</security-constraint>

-----

<security-role>
<!--定义安全角色-->
```

```
<role-name>twins</role-name>
</security-role>
```

2. 自定义部署描述文件 tongweb-web.xml 中提供配置将 web.xml 中定义的 role 映射到服务器中定义的用户 (principle) 或组 (group)，tongweb-web.xml 中配置如下：

```
<security-role-mapping>
<!-- web.xml中定义的Role-->
<role-name>twins</role-name>
<!-- 服务器中定义的用户 -->
<principal-name>twins</principal-name>
<!-- 服务器中定义的组 -->
<group-name>tongweb</group-name>
</security-role-mapping>
```

1.1.8 编写HTTP Servlet

1. 引入相应的包或类。

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

2. 继承 javax.servlet.http.HttpServlet。

```
public class MyServlet extends HttpServlet{
```

3. 实现 service 方法。Servlet 的主要功能是接收从浏览器发送过来的 HTTP 请求，并返回 HTTP 响应。这个工作是在 service 方法中完成。service 方法包括从 request 对象获得客户端数据和向 response 对象创建输出。如果一个 servlet 从 javax.servlet.http.HttpServlet 继承，实现了 doPost 或 doGet 方法，那么这个 servlet 只能对 POST 或 GET 作出响应。如果开发人员想处理所有类型的请求，只要简单的实现 service 方法即可。但假如选择实现 service 方法，则不必实现 doPost 或 doGet 方法，除非在 service 方法的开始调 super.service()。所有的 service 方法使用一样的参数。HttpServletRequest 提供关于请求的信息，servlet 可以使用 HttpServletResponse 对 HTTP 客户端作出响应。service 方法定义如下：

```
public void service(HttpServletRequest req,HttpServletResponse res)
    throws IOException{
```

4. 设置响应内容的类型。

```
res.setContentType("text/html");
```

5. 获得 java.io.PrintWriter 对象的引用，用来输出。

```
PrintWriter out = res.getWriter();
```

6. 使用 PrintWriter 对象的 println() 方法创建一些 HTML 代码，例如：

```
out.println("<html><head><title>Hello World!</title></head>");
out.println("<body><h1>Hello World!</h1></body></html>");
}
}
```

7. 编译 Servlet。从存放此 Servlet 源代码文件的目录编译此 Servlet 到包含此 Servlet 的应用中的 WEB-INF/classes 目录。例如：

```
javac -d /your_application_dir/WEB-INF/classes your_servlet.java
```

将此Servlet作为Web应用的一部分，打包部署Web应用。

8. 从浏览器访问 Servlet。
9. 一般说来，调用 Servlet 的 URL 取决于包含 Servlet 的 Web 应用的名字和 web.xml 中的 Servlet 映射的名字或者通过@WebServlet 声明的名称。一般 Servlet 的 URL 格式如下：
http://server:port/context-root/servlet-mapping?name=value。具体说明参见“[调用 Servlet 的 URL](#)”章节。

1.2 JSP

JavaServer Pages (JSP)技术允许开发人员轻松的创建静态或动态的Web内容。JSP不仅具有像Servlet一样创建动态Web页面的能力，而且更接近创建静态内容的方式。JSP使得页面的动态内容与它的表示进一步分离。JSP的设计迎合了两个不同的技术层面：页面的图形设计和创建动态内容的软件开发。JSP的主要特性包括：

- JSP 是基于文本的文档，用来处理客户端请求和构造响应。
- 构造 server 端访问的对象。
- 定义扩展 JSP 语言的机制。

TongWeb的JSP建立在Sun Microsystem提出的JSP2.3规范之上，完全符合JSP2.3规范，同时兼容JSP2.2、JSP2.0与JSP1.2规范。

JSP将静态内容和动态内容以不同形式表达，迎合了不同开发角色的需求。本质上JSP就是Servlet，每个JSP页面都会被应用服务器编译成对应的Servlet。这些对应的Servlet在服务器端负责响应客户端的请求，并把服务器处理结果发送给客户端。

JSP是以文本为基础的文档，它包含两种类型的文本：静态模板数据和JSP元素。

- 静态模板数据：表达成文本格式内容，如 HTML、XML 等。
- JSP 元素：用来构造动态内容。

每个JSP页面在响应请求前都会被转换成Servlet，并由此Servlet实际响应请求。所以JSP的生命周期与Servlet密切相关。JSP的生命周期分为以下几个阶段。

1. 初始化

JSP容器会在第一个用户请求某个JSP文件时，将此JSP文件转换成Java文件(扩展了Servlet类)，然后再编译成Class文件。装载这个JSP页面的Servlet类，构造这个Servlet类的实例，并在实际响应客户端的请求之前，Web容器将初始化该Servlet--调用JspInit()函数。

2. 处理请求

在JspInit()方法成功执行后，容器调用该Servlet的service()方法：

```
void _jspService(HttpServletRequest request, HttpServletResponse response);
```

对于每一个客户的请求，JSP容器新建一个线程来处理。

3. 终止 Servlet

如果需要消除一个JSP页面的Servlet(例如：当容器需要释放内存或者被关闭时)，容器会先调用它的JspDestroy()方法。

1.2.1 JSP指令

在JSP规范中，指令被定义为“JSP页面发送给容器的消息”。即用来设定某个JSP页面的全局变量，如类声明、response内容的类型等。其标准语法是：

```
<% directive {attr="value"} * %>
```

例如，下面的伪指令引入了java.util包和关联JSP页面到当前的Http Session。

```
<%@ page import="java.util.*" session="true"%>
```

JSP规范定义了三种伪指令：page、include和taglib。

表 1.2-1：JSP定义的三种伪指令

指令	说明	语法
include	包含其它文件的内容	<%@ include file="relativeURL"%>
page	定义JSP页面的属性	<pre><%@ page page_directive_attr_list %> page_directive_attr_list ::= {language="scriptingLanguage"} {extends="className"}</pre>

		<pre>{import="importList"} {session="true false"} {buffer="none sizekb"} {autoFlush="true false"} {isThreadSafe="true false"} {info="info_text"} {errorPage="error_url"} {isErrorPage="true false"} {contentType="ctinfo"} {pageEncoding="peinfo" }</pre>
taglib	为自定义标记设置标记库和前缀	<code><%@ taglib uri="URI" prefix=" tagPrefix"%></code>

1. 2. 2 JSP动作

JSP动作通常会与特定的请求相关，如为请求处理提供信息，实例化对象等。JSP动作的使用减少了JSP页的Java代码，使JSP页面更易于维护。JSP提供标准化的动作，也支持用户定制的动作。

典型的JSP标准动作语法如下：

```
<jsp:useBean="bean" class="beanclass"/>
```

说明：

与标准JSP标记“<%”和“%>”不同，所有标准动作标记都用jsp作为前缀。且JSP动作是大小写敏感的。动作元素为请求处理阶段提供信息。动作元素遵循XML语法，包括开始标记、结束标记和属性等内容。例如，jsp:include动作可以插入JSP页面到当前的页面。

```
<jsp:include page="header.jsp" flush="true"/>
```

例如：jsp:useBean动作可以创建JavaBean对象。

```
<jsp:useBean id="cBean" scope="application" class="Expns.CBean">
<%
cBean.getConnection();
cBean.getEngine();
%>
</jsp:useBean>
```

下表列出了JSP规范定义的标准动作：

表 1. 2-2：JSP规范的标准动作

动作元素	说明	语法
------	----	----

<jsp:forward>	传递请求到其他Web资源	<jsp:forward page="relativeURLspec"/>和 <jsp:forward page="urlSpec"> {<jsp:param />}* </jsp:forward>
<jsp:getProperty>	插入JavaBeans的属性值到结果中	<jsp: getProperty name="beanInstanceName"property="propertyName"/>
<jsp:include>	包含静态文件或Web组件的执行结果	<jsp:include page="urlSpec"flush="true false"/>和 <jsp:include page="urlSpec"flush="true false"> {<jsp:param/>}* </jsp:include>
<jsp:plugin>	引起applet或bean的执行	<jsp:plugin type="bean applet" code="objectCode" codebase="objectCodebase" {align="alignment"} {archive="archiveList"} {height="height"} {hspace="hspace"} {jreversion="jreversion"} {name="componentName"} {vspace="vspace"} {width="width"} {nspluginurl="url"} {iepluginurl="url"}> {<jsp:params> {<jsp:param name="paramName" value="paramValue"/>}+ </jsp:params> {<jsp:fallback> arbitrary_text </jsp:fallback>} </jsp:plugin>
<jsp:setProperty>	设置JavaBeans的属性	<jsp: setProperty name=" beanInstanceName" { property="*" property="propertyName" [param="parameterName "]] property=" propertyName" value="{string <%=expression%>}"}

		/>
<jsp:useBean>	定位或实例化JavaBeans, 指定名字和作用范围	<pre> <jsp: useBean id="beanInstanceName" scope="page request session application" {class="packa ge.class" [type="package.class"] beanName="{package.class} <%=expression %>}" type="package.class" type="package.class " } {/> > other elements </jsp: useBean> </pre>
<jsp:param>	提供键/值信息	<jsp:param name="name" value="value" />

1.2.3 JSP用例

本节展示了JSP2.2的一个特性：**omit属性**。

特性描述：

在<jsp:element>中，如果把一个<jsp:attribute>的omit属性设为true，则该<jsp:attribute>在输出时就会被忽略。omit属性的取值范围是：true/false，默认为false。

示例：

以下是两个使用<jsp:element>来产生一个标签的例子，它们使用<jsp:attribute>产生其color属性。

- 例1代码：

```

<jsp:element name="font">
<jsp:attribute name="color" omit="false">blue</jsp:attribute>
<jsp:body>omit is false</jsp:body>
</jsp:element>

```

- 例2代码：

```

<jsp:element name="font">
<jsp:attribute name="color" omit="true">blue</jsp:attribute>
<jsp:body>omit is true</jsp:body>
</jsp:element>

```

这两个例子的唯一区别就是其产生color属性的<jsp:attribute>中的omit属性值不同。第一个的omit是false，则color属性起作用了。第二个的omit是true，则color被忽略了。

1.3 JSF

Java Server Faces (JSF) 是一种用于构建Web应用程序的新标准Java框架，它提供了一种以组件为中心，以事件驱动来开发Java Web用户界面的方法，从而简化了开发。JSF系统结构图如下图所示：

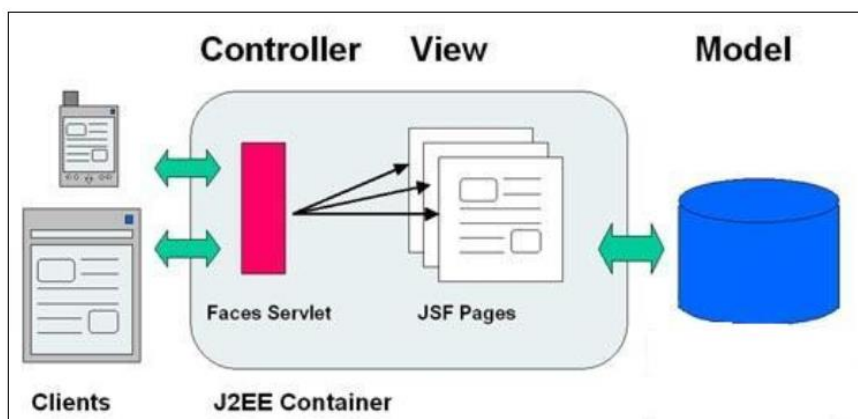


图 1.3-1: JSF系统结构

Faces Controller Servlet充当用户和JSF应用程序之间的纽带。Faces Controller Servlet在明确限定的JSF生命周期的范围内工作。例如，一旦系统接收到访问应用的用户请求，Faces Controller Servlet便通过事先准备的JSF上下文来处理请求。然后控制器把用户指引到所请求的页面。该页面通常使用简单的表达式语言来处理来自JSF上下文的应用程序数据。由此可知FacesServlet是JSF的核心组件，它处理所有请求，所以如果想要使用JSF，必须在web.xml中有类似下面的定义：

```
<servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.jsf</url-pattern>
</servlet-mapping>
```

1.3.1 JSF中主要概念

1. UI 组件(UI Component)

UI组件是具有属性，方法和事件的JavaBean，如封装了HTML/CSS等的日历控件。通常每一个页面的组件功能组成一棵树，组件之间维护父子关系。

2. 验证器(Validator)

验证器负责检查用户输入的值以确保该值符合期望的格式或内容。验证器提供了强大的框架来简化这个任务。JSF附带了一套标准的验证器来进行诸如输入长度和范围检查之类的验证，但也可以自己编写或者使用第三方的验证器。

3. 托管 Bean (Managed Bean)

托管Bean又称Backing Bean，是标准的JavaBean，其基本功能是提供一组属性与页面中UI组件对应以及提供一组方法与页面组件功能对应，在MVC的结构框架中是Controller的角色。

4. 事件(Event)和监听器(Listener)

通常情况下，开发Web应用需要开发人员考虑请求和响应——即下层无状态HTTP协议的通信机制。这些问题对于业务应用自身来说没有必要，JSF使用了JavaBean事件/监听器模型，实现了基于Web的事件机制，已提供4种事件使得开发人员不必考虑通信细节。

- 值改变事件: 值改变事件是在用户修改组件的值后，由输入控件触发。
- 动作事件: 用户激活命令组件(比如按钮)时产生，如点击 submit 按钮。
- 数据模型事件: 数据感知组件选择了一行要处理的数据时触发。
- 生命周期事件: 在 JSF 处理 HTTP 请求时的不同阶段触发。

1.3.2 JSF的生命周期

JSF页面的生命周期与JSP页面的生命周期类似：客户向页面发出HTTP请求，服务器返回翻译成HTML的响应，但JSF提供了更多处理页面的服务。生命周期与页面编写者无关，这个概念主要面对开发人员。下图为JSF的生命周期。

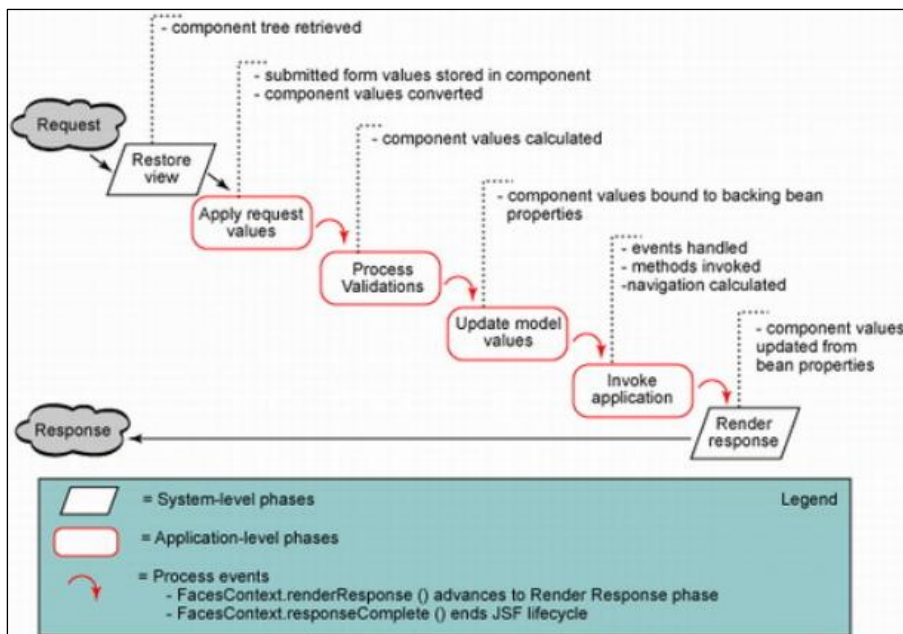


图 1.3-2: JSF的生命周期

JSF的生命周期分6个阶段，分别是恢复视图(Restore View)、应用请求值(Apply request values)、处理验证(Process Validations)、更新模型值(Update model values)、调用应用程序(Invoke application)和显示响应(Render response)。

1.3.2.1 恢复视图

当请求JSF页面时，如点击按钮或链接，JSF开始恢复视图阶段。

在这个阶段JSF建立页面的视图，给视图中的组件设置事件处理器、校验器，在FacesContext中保存视图。FacesContext含有所有处理请求的信息，所以页面元素包括组件标签、事件处理器、转换器、校验器都要接触FacesContext。如果该请求是第一次的请求，JSF在这个阶段产生一个空的视图，生命周期进入显示应答阶段，这个空的视图会在页面返回的时候用到。如果请求不是第一次，对应于这个页面的视图已经存在，JSF用存在客户端或服务器端的信息恢复视图。

1.3.2.2 应用请求值

在这个阶段，使用请求中的当前值来更新每个组件的当地值。这些值可能来自请求参数，请求报头及Cookies等。在这个阶段的处理中，组件可以往事件队列里加入某些事件，这些事件可在随后的处理阶段处理。

可编辑值的输入控件都有个immediate属性。如果为true，将立刻进行验证，而不是要等到处理验证阶段。按钮或者超链接也有immediate属性，如果该属性被设置为true，它将在本阶段触发动作事件。

阶段执行后，事件会广播给激活的事件监听器。任何响应此生命周期事件的代码都可以截断生命周期，直接跳到显示响应阶段，或者终止处理。

1.3.2.3 处理验证

当每个组件的本地值被更新后，在此阶段中，每个组件都有一些值需要根据应用程序的验证规则进行有效性验证。这些验证规则可以是预先进行定义的（JSF中提供的），也可以由开发者进行定义。

用户所输入的值会与这些验证规则进行比较。如果验证成功，将到生命周期的下一个阶段。否则控制将跳到显示响应阶段，显示验证和转换错误信息。

1.3.2.4 更新模型值

只有当组件树中所有组件的本地值都通过验证后，才有可能到达该阶段，开始处理相关的后台Bean或模型对象。完成此阶段时，JSF将向激活的监听器广播事件。同样监听器可以跳转到呈现相应阶段。

到这一步为止，基于用户的输入JSF更新了组件的值，验证了这些值，更新了相关的bean，但是还没有编写任何代码。JSF将自动完成大量的UI处理任务。

1.3.2.5 调用应用程序

在这个阶段，JSF实现处理所有应用层次的事件，比如提交表单或链接到其它页面等。这里才真正涉及

到具体的业务。在该阶段，JSF同样向所有激活的监听器广播此阶段的所有事件。

1.3.2.6 显示响应

这个阶段的任务是发送响应给用户，以及保存视图的状态，以使用户在此请求时可以在恢复视图阶段恢复该视图。

1.3.3 JSF用例

以下用例为使用JSF实现加法计数器功能：用于计算两个数字之和，并同时记录了session范围内计算过的加法。用例的受管Bean中使用了@ManagedBean、@RequestScoped、@SessionScoped、@ManagedProperty等注解。

加法计数器的JSF页面主要代码如下，需要提交的数字放置在<h:form>标签中，并添加了验证器，<h:commandButton>用于表单的提交并调用受管Bean相应的方法。

```
<h:form>
<h:inputText value="#{calculator.num1}" id="number1" required="true" requiredMessage="数字1不能
为空"> /*第一个数字*/
    <f:validateLongRange maximum="1000" minimum="0"/>
</h:inputText>
<h:messages for="number1"></h:messages>
<br/>
<h:inputText value="#{calculator.num2}" id="number2" required="true" requiredMessage="数字2不能
为空"> /*第二个数字*/
    <f:validateLongRange maximum="1000" minimum="0"/>
</h:inputText>
<h:messages for="number2"></h:messages>
<br/>
<h:outputText id="plus" value="两者之和: #{calculator.result}" rendered="{calculator.result != null}"/>
<br/>
<h:outputText id="total" value="Session范围内计算过的加法: #{calculator.total.total}"/>
<br/>
<h:commandButton value="计算" action="#{calculator.add}"/></h:commandButton>
</h:form>
```

JSF的前台页面完成后需要设计对应的受管Bean用于接受前台的数字并提供计算加和的方法。同时，为了保存session范围内计算过的加法，还需定义额外的属性，并在每次计算后更新该属性。用于加法计算的CalculatorBean和用于存储session范围内信息的CalculatorRecorderBean主要代码如下：

```
@ManagedBean(name="calculator")
```

```

@RequestScoped
public class CalculatorBean {
    private long num1;
    private long num2;
    private long result;
    @ManagedProperty(value="#{calculatorRecoder}")
    private CalculatorRecoderBean total;
    public long getNum1() {
        return num1;
    }
    public void setNum1(long num1) {
        this.num1 = num1;
    }
    public long getNum2() {
        return num2;
    }
    public void setNum2(long num2) {
        this.num2 = num2;
    }
    public long getResult() {
        return result;
    }
    public void setResult(long result) {
        this.result = result;
    }
    public CalculatorRecoderBean getTotal() {
        return total;
    }
    public void setTotal(CalculatorRecoderBean total) {
        this.total = total;
    }

    public void add(){
        result = num1+num2;
        total.setTotal(total.getTotal()+String.valueOf(num1)+"+"+String.valueOf(num2)+"="+result+";");
    }
}
@ManagedBean(name="calculatorRecoder")

```

```

@SessionScoped
public class CalculatorRecorderBean {
    private String total="";
    public String getTotal() {
        return total;
    }
    public void setTotal(String total) {
        this.total = total;
    }
}

```

JSF 2.0中引入了注解功能，上例中注解ManagedBean定义该JavaBean为受管Bean，RequestScoped、SessionScoped等定义了Bean的作用范围，ManagedProperty用于属性的注入等。JSF注解的引入直接简化了faces-config.xml，在本加法计数器的用例中，不需要配置faces-config.xml文件即可部署运行。

另外在JAVAE7中，JSF2.2提供了一些新的特性，其中一个重要的更新是JSF对html5的支持。

虽然只是小版本的升级，但对于希望在JSF应用中使用HTML5技术的开发人员而言，JSF 2.2带来的更新很重要，尤其是pass through能力，它允许在JSF组件不知情的情况下传递HTML属性。

HTML5中增加了很多新特性，其中有些是在已有的元素上增加了对新属性的支持。例如，input元素的type属性支持text、search、email、url、tel、range、number和date等属性值。另外，它还有一系列的自定义数据属性，用来在HTML元素上关联少量数据。这些数据不会显示，但可以用JavaScript读取。

对于像JSF这样基于组件的库，上述情况带来一个问题：为了识别新属性，所有已有的组件必须更新。对于需要显式支持这些属性的组件而言，的确如此。但在很多情况下，组件只需要在最终生成的标记中包含这些属性即可。JSF 2.2的pass-through属性就是这样实现的。

在Facelet页中，pass-through属性可以通过以下三种方法设置：

1. 通过组件标签的命名空间属性

```

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:p="http://java.sun.com/jsf/passthrough">
<h:form>
<h:inputText value="#{bean.value}" p:placeholder="Enter text"/>
</h:form>
</html>

```

2. 通过TagHandler f:passThroughAttribute设置单个属性

```

<html xmlns="http://www.w3.org/1999/xhtml"

```

```

xmlns:h="http://java.sun.com/jsf/html"
xmlns:f="http://java.sun.com/jsf/core">
<h:form>
<h:inputText value="#{bean.value}">
<f:passThroughAttribute name="placeholder" value="Enter text" />
</h:outputText>
</h:form>
</html>

```

3. 通过TagHandler f:passThroughAttributes设置多个属性

```

<h:outputText value="Something">
<f:passThroughAttributes value="#{bean.multipleAttributes}" />
</h:outputText>

```

`#{bean.multipleAttributes}` 代表一个 `Map<String, Object>` 对象。它的值可以是常量，也可以是表达式。

通过使用 Express Language 3，多个属性也可以通过 EL 表达式直接定义。

```

<h:outputText value="Something">
<f:passThroughAttributes value="{ 'one':1, 'two':2, 'three':3 }" />
</h:outputText>

```

上述改变结果是，开发人员现在可以使用纯 HTML 来编写 JSF 视图。

在服务器端，可以使用类 `UIComponent` 的新方法 `getPassThroughAttributes()` 和 `getPassThroughAttributes(boolean create)` 来设置属性。

```

UIComponent component = new SomeComponent();
Map passThrough = component.getPassThroughAttributes();
passThrough.put("placeholder", "Enter text");

```

1.4 配置资源引用

用户可以在 Web 应用中访问外部资源（数据源、EJB、JMS 等），即将全局 JNDI 资源映射到组件资源（`java:comp/env`）。这样应用在使用外部资源时不用修改代码来调整全局 JNDI 的名称，只需要通过修改配置实现调用组件资源中的 JNDI，无需修改应用代码即可实现对外部资源的灵活使用。

本节的各项资源引用的配置中除 `message-destination` 外，每个配置项都增加了如下的配置内容：

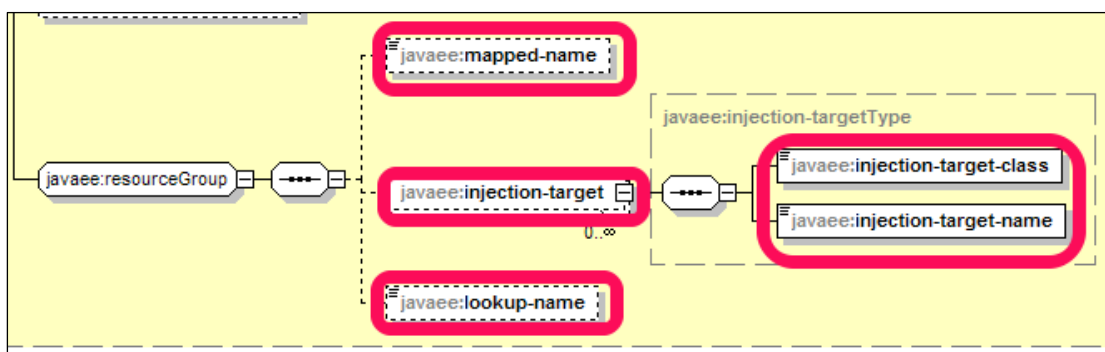


图 1.4-1：配置资源引用

具体配置项的含义：

- **mapped-name**：该资源需要映射到的名称。
- **injection-target**：将资源注入到域 (fields) 或者JavaBean的属性中，其可以指定一个类的全路径（即注入的目标）。
- **lookup-name**：待查找转化为资源引用的JNDI名称。

如果开发过程中需要上述的配置，可以参照下文env-entry的web.xml例子进行配置。

1.4.1 支持环境条目 (env-entry)

env-entry元素声明Web应用的环境项。它由一个可选的description元素、一个env-entry-name元素（一个相对于 java: comp/env 环境的 JNDI 名）、一个 env-entry-value 元素值、一个 env-entry-type 元素（java.lang 程序包中一个类型的完全限定类名，java.lang.Boolean、java.lang.String 等）、以及可选的 mapped-name、injection-target、lookup-name 组成。

下面是web.xml配置的例子：

```
<env-entry>
<env-entry-name>TestParameter</env-entry-name>
<env-entry-value>test</env-entry-value>
<env-entry-type>java.lang.String</env-entry-type>
<mapped-name>test</mapped-name>
<injection-target>
<injection-target-class>com.abc.test</injection-target-class>
<injection-target-name>aaa</injection-target-name>
</injection-target>
<lookup-name>ttt</lookup-name>
</env-entry>
```

用户可以在应用程序中lookup该环境项（如 java:comp/env/TestParameter），以便应用所需的环境项可通过配置灵活使用，而不是在代码中写定。

1.4.2 支持资源引用(resource-ref)

web.xml中的resource-ref元素用于指定对外部资源引用的声明。

- **<res-ref-name/>**资源引用名。该名称是一个与java:comp/env上下文相对应的JNDI名称，并且在整个Web应用中必须是惟一的。
- **<res-type/>**引用资源的类名，如javax.sql.DataSource。资源管理器连接工厂的名称。
- **<res-auth/>**servlet代码通过编程注册到资源管理器，该元素的值必须为Application或Container。
- **<res-sharing-scope/>**是否可以共享通过给定资源管理器连接工厂引用获得的连接。该元素的值必须为Shareable(默认值)或Unshareable。

web.xml代码片段：

```
<resource-ref>
<res-ref-name>TestDS</res-ref-name>
<res-type>javax.sql.DataSource</res-type>
<res-auth>Container</res-auth>
<mapped-name>test</mapped-name>
<injection-target>
<injection-target-class>com.abc.test</injection-target-class>
<injection-target-name>aaa</injection-target-name>
</injection-target>
<lookup-name>ttt</lookup-name>
</resource-ref>
```

Servlet代码片断：

```
InitialContext initialcontext = new InitialContext();
javax.sql.DataSource ds =(DataSource)initialcontext.lookup("java:comp/env/TestDS");
```

1.4.3 支持资源环境引用(resource-env-ref)

web.xml中的resource-env-ref元素用于指定对外部资源环境引用的声明。tongweb-web.xml中的resource-env-ref将资源环境引用名和全局资源名关联起来。

- **<resource-env-ref-name/>**资源环境引用名，该名称是一个与java:comp/env上下文相对应的JNDI名称，并且在整个Web应用中必须是惟一的。
- **<resource-env-ref-type/>**资源环境的类型，资源相关的管理对象类型。

web.xml代码片断：

```
<resource-env-ref>
<resource-env-ref-name>jms/testQueue
</resource-env-ref-name>
<resource-env-ref-type>javax.jms.Queue
</resource-env-ref-type>
</resource-env-ref>
```

Servlet代码片断:

```
InitialContext initialcontext = new InitialContext();
javax.jms.Queue queue =(Queue)initialcontext.lookup("java:comp/env/jms/testQueue");
```

1.4.4 支持EJB引用(ejb-ref)

web.xml中的ejb-ref元素用于指定EJB引用的声明。

- <description/>配置描述, 此属性可选
- <ejb-ref-name/>ejb引用名
- <ejb-ref-type/>引用的ejb的类型, 可选值为Entity和Session
- <home/>引用的ejb的home接口类名,
- <remote/>引用的ejb的remote接口类名
- <ejb-link/>可选项, 如果Web应用与EJB应用在同一个企业应用中(EAR), 则可以使用ejb-link指向EJB JAR或同一个企业应用中的一个EJB, 且ejb-link的名字必须和被应用的ejb-name匹配(ejb-jar.xml中定义)。如果使用了ejb-link则不用在tongweb-web.xml中使用ejb-ref。或者
<ejb-link>../Customer.jar#CustomerEJB</ejb-link>

说明:

<ejb-link>中引入jar_name#EJB_name是用于区分不同EJB JAR中的同名Bean(路径相对于Web应用)。

1.4.4.1 ejb-ref中不使用ejb-link

web.xml中定义不带link的ejb-ref:

```
<ejb-ref>
<description>This is a reference to the store products entity </description>
<ejb-ref-name>ejb/ProductHome</ejb-ref-name>
<ejb-ref-type>Entity</ejb-ref-type>
<home>org.jboss.store.ejb.ProductHome</home>
<remote> org.jboss.store.ejb.Product</remote>
</ejb-ref>
```

Servlet代码片断:

```
InitialContext initialcontext = new InitialContext();
Object ejbhome=initialcontext.lookup("java:comp/env/ejb/ProductHome");
```

1.4.4.2 ejb-ref中使用ejb-link

web.xml中定义带link的ejb-ref:

```
<ejb-ref>
<ejb-ref-name>ejb/ShoppingCartHome</ejb-ref-name>
<ejb-ref-type>Session</ejb-ref-type>
<home>org.jboss.store.ejb.ShoppingCartHome</home>
<remote> org.jboss.store.ejb.ShoppingCart</remote>
<ejb-link>ShoppingCartBean</ejb-link>
</ejb-ref>
```

Servlet代码片断:

```
InitialContext initialcontext = new InitialContext();
Object ejbhome=initialcontext.lookup("java:comp/env/ejb/ShoppingCartHome");
```

1.5 Session管理

1.5.1 URL ReWriting

在浏览器不支持Cookies或者“是否使用Cookies存放Session”属性设置为false时，无法使用Cookies保存Session，为保证使用同一网页多次访问某应用时SessionID在服务器端的唯一性，用户可以通过在应用实现URL ReWriting来保证。

在应用中使用Response的encodeURL方法完成URL ReWriting。示例如下：

```
<a href="<%= response.encodeURL("/test1/testjsp.jsp") %>">Test</a>
```

当访问test1/testjsp.jsp时，点击链接a，如果无法使用Cookies保存Session则SessionID将会写在URL上，如果可以使用Cookies保存Session则SessionID不会写在URL上。

1.6 其他

1.6.1 Listener组件的开发

用户可以在tongweb-web.xml中增加listener组件的配置，用于监听相应的应用事件的发生。

开发方式如下：

1. 创建一个实现了 com.tongweb.web.thor.LifecycleListener 接口的类，假设名称为 A。
2. 在类 A 的 lifecycleEvent(LifecycleEvent event) 方法内部添加具体的用户代码逻辑，可以监测到的

应用事件包括 start、after_start、before_stop、stop、after_stop、before_destroy 等。

3. 将类 A 打包成 jar 文件放到 TongWeb 的 lib 目录下。
4. 配置需要监听的 Web 应用的 tongweb-web.xml 文件,其 class-name 属性值为前面创建的类 A 的全路径,例如:

```
<listeners><listener name="test" class-name="com.tongweb.test.A"/></listeners>
```

5. 部署应用到 TongWeb 应用服务器。
6. 用户代码逻辑在接收到相应的事件后执行。

1.6.2 class loader delegate

TongWeb用户手册类加载部分的类加载机制中,TongWeb应用服务器提供两种类加载模式,分别是父优先和子优先。该模式决定了类加载器在查找一个类的时候,是先查找类加载器自身指定的类路径还是先查找父类加载器上的类路径。

tongweb-web.xml中的delegate属性用于决定应用使用子优先模式还是父优先模式。delegate默认使用父优先模式。

1.6.3 使用WEB-INF下资源

用户可通过当前线程的成员ContextClassLoader中提供的getResource、getResourceAsStream方法使用WEB-INF/classes下的资源文件。

1.6.4 编写自定义Valve

TongWeb在web容器中的请求、响应处理流程中提供了阀门(Valve)机制。TongWeb内部提供了一些默认的阀门,如AccessLogValve来处理访问日志业务,同时TongWeb也为阀门提供了更好的扩展性,既用户可以自定义阀门,通过这些阀门来实现拦截请求、处理响应并将其加入到应用所在的虚拟主机中进行应用。

开发方式如下:

1. 创建一个自定义阀门类,假设名称为ClassPrintIPValve。
2. 需要继承 com.tongweb.catalina.valves.ValveBase, ValveBase 是 TongWeb 抽象的一个基础类,它帮我们实现了生命接口及 MBean 接口,使开发人员只需专注阀门的逻辑处理即可。
3. 需要重写 invoke 方法,invoke 方法的两个参数:

```
com.tongweb.catalina.connector.Request
com.tongweb.catalina.connector.Response
```

这两个类是TongWeb对javax.servlet.http.HttpServletRequest及javax.servlet.http.HttpServletResponse的标准实现,在invoke方法中可以直接调用这两个对象的相关方法对请求和响应进行处理。

自定义阀片断:

```
package com.tongweb.valve;
import java.io.IOException;
```

```
import javax.servlet.ServletException;
import com.tongweb.catalina.connector.Request;
import com.tongweb.catalina.connector.Response;
import com.tongweb.catalina.valves.ValveBase;

public class ClassPrintIPValve extends ValveBase {
    @Override
    public void invoke(Request request, Response response) throws IOException, ServletException {
        System.out.println(request.getRemoteAddr());
        getNext().invoke(request,response);
    }
}
```

注意：

一定要执行调用下一个阀门操作，即执行 `getNext().invoke(request, response)`，否则运行时将出现错误，请求到这个阀门就停止往下处理。

4. 将实现好的自定义阀的 class 加入到 TongWeb 的 classpath 中。
5. 登录管理控制台，点击“虚拟主机”，选择应用所在的虚拟主机进行编辑，指定自定义 Valve。如：`com.tongweb.valve.ClassPrintIPValve`。
6. 保存后应用于虚拟主机下的所有应用中。

第2章 EJB应用

2.1 EJB2.0

2.1.1 EJB应用结构

Enterprise JavaBeans (EJB) 是 J2EE 规范中定义的一种基于 Java 的服务器组件模型，EJB 应用对分布式企业应用中的业务逻辑进行封装，运行于 EJB 容器中。EJB 的应用结构如下所示：

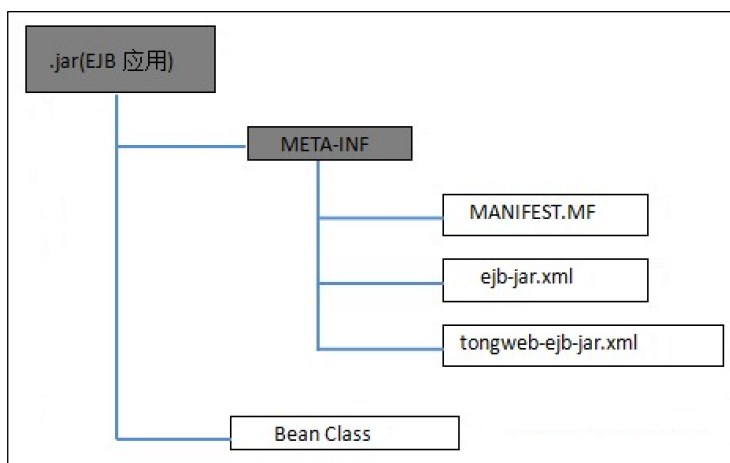


图 2.1-1: JB的应用结构

- `ejb-jar.xml`: J2EE 标准部署描述文件。
- `tongweb-ejb-jar.xml`: 描述 TongWeb 中特定 EJB 属性的部署描述文件。

2.1.2 SessionBean

会话Bean可以分为有状态会话Bean和无状态会话Bean。有状态会话Bean可以在客户访问之间保持数据，而无状态会话Bean不会与客户访问之间保持数据。下面简单介绍一下有状态SessionBean的开发步骤及示例：

1. 定义 HelloBean 的远程接口。

```
package examples;
import javax.rmi.RemoteException;
import javax.ejb.CreateException;
import javax.ejb.EJBObject
public abstract interface Hello extends EJBObject{
    public abstract void sayHello() throws RemoteException,CreateException;
}
```

2. 定义 HelloBean 的 Home 接口。

```
package examples;

import java.rmi.RemoteException;
import javax.ejb.CreateException;
import javax.ejb.EJBHome;

public abstract interface HelloHome extends EJBHome{
    public abstract Hello create() throws RemoteException, CreateException;
}
```

3. 定义 HelloBean 的业务接口。

```
package examples;
import javax.rmi.RemoteException;
import javax.ejb.*;

public class HelloBean implements SessionBean{
    private SessionContext cts;

    public void ejbActivate()throws EJBException, RemoteException{}
    public void ejbPassivate()throws EJBException, RemoteException{}
    public void ejbRemove()throws EJBException, RemoteException{}
    public void setSessionContext(SessionContext arg0)throws EJBException,
        RemoteException{
this.cts = this.cts
    }
    Public void ejbCreate(){}
    Public void sayHello()throws Exception{
        System.out.println("Hello EJB");
    }
}
```

4. 配置文件 ejb-jar.xml，配置片段如下。

```
<ejb-jar>
<enterprise-beans>
<session>
<ejb-name>Hello</ejb-name>
<home>examples.HelloHome</home>
<remote>examples.Hello</remote>
<ejb-class>examples.HelloBean</ejb-class>
<session-type>Stateful</session-type>*定义有状态会话bean
<transaction-type>Container</transaction-type>
</session>
</enterprise- beans >
```



```
<ejb-jar>
```

5. 配置 tongweb-ejb-jar.xml，配置片段如下：

tongweb-ejb-jar配置片段：

```
<tongweb-ejb-jar>
<ejb-deployment ejb-name="Hello">
<jndi name="myHello" interface="examples.HelloHome"></jndi>
  </ejb-deployment>
</tongweb-ejb-jar>
```

2.1.3 访问EJB应用

2.1.3.1 同一个JVM中访问

在同一个JVM中，客户程序访问EJB应用的示例如下。

1. 查找Remote接口代码片段：

```
Context jndiContext = new InitialContext();
HelloHome home = (HelloHome)jndiContext.lookup("myHello");
```

2. ejb-jar.xml配置片段：

```
<enterprise-beans>
<session>
<ejb-name>Hello</ejb-name>
<home>examples.HelloHome</home>
<remote>examples.Hello</remote>
<ejb-class>examples.HelloBean</ejb-class>
.....
</session>
</enterprise-beans>
```

3. tongweb-ejb-jar.xml配置片段：

```
<tongweb-ejb-jar>
<ejb-deployment ejb-name="Hello">
  <jndi name="myHello" interface=" examples.HelloHome"></jndi>
</ejb-deployment>
</tongweb-ejb-jar>
```

2.1.3.2 不同JVM中访问

客户端程序访问远程JVM中EJB应用时，访问方式示例如下：

1. 查找Remote接口代码片段:

```
Properties props = new Properties();
props.setProperty("java.naming.factory.initial",
"com.tongweb.tongejb.client.RemoteInitialContextFactory");
props.setProperty("java.naming.provider.url",
"http://168.1.100.113:5100/ejbserver/ejb");
Context jndiContext = new InitialContext(props);
HelloHome home = (HelloHome)jndiContext.lookup("myHello");
```

2. ejb-jar.xml配置片段:

```
<enterprise-beans>
<session>
<ejb-name>HelloEJB</ejb-name>
<home>examples.HelloHome</home>
<remote>examples.Hello</remote>
<ejb-class>examples.HelloBean</ejb-class>
.....
</session>
</enterprise-beans>
```

3. tongweb-ejb-jar.xml配置片段:

```
<tongweb-ejb-jar>
<ejb-deployment ejb-name="Hello">
<jndi name="myHello" interface="examples.HelloHome"></jndi>
</ejb-deployment>
</tongweb-ejb-jar>
```

2.1.3.3 通过Local接口访问

Web应用和EJB应用都可以采用<ejb-link>方式, 通过Local接口访问EJB应用。

通过HelloEJB.jar对Customer.jar中CustomerEJB的调用配置示例如下:

1. 在调用端 (HelloEJB) 的配置:

```
Context jndiContext = new InitialContext();
CustomerHome home = (CustomerHome)jndiContext.
lookup("java:comp/env/CustomerHomeLocal");
```

2. ejb-jar.xml配置片段:

```
<session>
<ejb-name>HelloEJB</ejb-name>
```

```
<home>examples.HelloHome</home>
<remote>examples.Hello</remote>
<ejb-class>examples.HelloBean</ejb-class>
<session-type>Stateful</session-type>
<transaction-type>Container</transaction-type>
<ejb-local-ref>
<ejb-ref-name>CustomerHomeLocal</ejb-ref-name>
<ejb-ref-type>Session</ejb-ref-type>
<local-home>com.customer.CustomerHome</local-home>
<local>com.customer.Customer</local>
<ejb-link>Customer.jar#CustomerEJB</ejb-link>
或者
<ejb-link> CustomerEJB</ejb-link>
</ejb-local-ref>
</session>
```

注意：

<ejb-link>中引入jar_name#是用于区分不同EJBJAR中的同名Bean，通常可以省略。

3. 在被调用端（CustomerEJB）ejb-jar.xml的配置片段：

```
<session>
    <ejb-name>CustomerEJB</ejb-name>
    <local-home>com.customer.CustomerHome</local-home>
    <local>com.customer.Customer</local>
    <ejb-class>com.customer.CustomerBean</ejb-class>
    <session-type>Stateful</session-type>
    <transaction-type>Container</transaction-type>
</session>
```

2.1.4 在SessionBean中配置数据源

SessionBean通过资源应用的方式使用数据源。数据源信息在ejb-jar.xml和tongweb-ejb-jar.xml文件中进行配置。

1. ejb-jar.xml配置片段：

```
<enterprise-beans>
    <session>
        <ejb-name>Hello</ejb-name>
        <home>examples.HelloHome</home>
        <remote>examples.Hello</remote>
```

```
<ejb-class>examples.HelloBean</ejb-class>
<session-type>Stateful</session-type>
<transaction-type>Container</transaction-type>
<resource-ref>
    <description>DB Connection</description>
    <res-ref-name>myds</res-ref-name>
    <res-type>javax.sql.DataSource</res-type>
    <res-auth>Container</res-auth>
</resource-ref>
</session>
</enterprise-beans>
```

2. tongweb-ejb-jar.xml配置片段:

```
<tongweb-ejb-jar>
    <ejb-deployment ejb-name="Hello">
        <jndi name="myHello" interface="examples.HelloHome"></jndi>
        <resource-link res-id="testDS" res-ref-name="myds"></resource-link>
    </ejb-deployment>
</tongweb-ejb-jar>
```

3. 在EJB中查找数据库的代码片段:

```
Context initContext = new InitialContext();
DataSource ds = (DataSource)ic.lookup("java:comp/env/myds");
```

2.1.5 对weblogic-ejb-jar.xml的支持

TongWeb支持weblogic-ejb-jar.xml中的<ejb-name>，<jndi-name>，<local-jndi-name>元素。

<ejb-name>对应的是tongweb-ejb-jar.xml中ejb-deployment标签中的ejb-name属性。另外两个元素对应的是tongweb-ejb-jar.xml中的jndi标签中的name属性。如果通过Remote接口访问EJB，需要配置<jndi-name>；如果通过Local接口访问EJB，需要配置<local-jndi-name>。示例如下：

2.1.5.1 通过Remote接口访问EJB

1. 调用端的代码片段:

```
Context jndiContext = new InitialContext();
HelloHome home = (HelloHome)jndiContext.lookup("myHello");
```

2. ejb-jar.xml配置片段:

```
<enterprise-beans>
```

```
<session>
<ejb-name>Hello</ejb-name>
<home>examples.HelloHome</home>
<remote>examples.Hello</remote>
<ejb-class>examples.HelloBean</ejb-class>
.....
</session>
</enterprise-beans>
```

3. weblogic-ejb-jar.xml配置片段:

```
<weblogic-enterprise-bean>
<ejb-name> Hello </ejb-name>
    <jndi-name> myHello </jndi-name>
</weblogic-enterprise-bean>
```

2.1.5.2 通过Local接口访问EJB

1. 调用端的代码片段:

```
InitialContext context = new InitialContext();
ComputerHome home = (ComputerHome)context.lookup("myComputer ");
```

2. ejb-jar.xml配置片段:

```
<enterprise-beans>
<session>
    <display-name>computer</display-name>
    <ejb-name>computer</ejb-name>
    <local-home>com.tongweb.test.ComputerHome</local-home>
    <local>com.tongweb.test.Computer</local>
    <ejb-class>com.tongweb.test.stateless.ComputerBean</ejb-class>
    <session-type>Stateless</session-type>
    <transaction-type>Bean</transaction-type>
</session>
</enterprise-beans>
```

3. weblogic-ejb-jar.xml配置片段:

```
<weblogic-enterprise-bean>
<ejb-name> computer </ejb-name>
    <local-jndi-name>myComputer </local-jndi-name>
</weblogic-enterprise-bean>
```

2.1.6 Entity Bean

2.1.6.1 配置CMR

CMR存在多种映射关系，且仅支持部署描述符文件配置。使用以下具体示例对TongWeb上的CMR配置进行说明。

CMR示例使用了两个CMP Entity Bean：

- **Account Bean：** 包括 accno、balance、customer 域。
- **Addressinfo Bean：** 包括 id1、street、city、state、zip 域。

使用三个数据库表：

- **ACCOUNT：** 包括 accno（主键）、balance、customer 字段。
- **ADDRESSINFO：** 包括 id1（主键）、accno（外键）、street、city、state、zip 字段。
- **ACCOUNTADDRESS：** 包括 accno、id1 字段。

2.1.6.2 一对一映射

- **单向一对一**

Account Bean到Addressinfo Bean的单向一对一映射关系。在配置文件中的描述如下：

1. 在ejb-jar.xml中，标签①和③定义一对一映射关系，标签②定义Account Bean到AddressinfoBean的单向CMR域。

```
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AccountRelationshipRole
</ejb-relationship-role-name>
<multiplicity>One</multiplicity>①
<relationship-role-source>
<ejb-name>Account</ejb-name>
    </relationship-role-source>
    <cmr-field>
    <cmr-field-name>addressinfo</cmr-field-name>
    </cmr-field> ②
</ejb-relationship-role>
    <ejb-relationship-role>
<ejb-relationship-role-name>
```

```

                AddressinfoRelationshipRole
            </ejb-relationship-role=name>
<multiplicity>One</multiplicity>③
<relationship-role-source>
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
</ejb-relationship-role>
</ejb-relation>
</relationships>

```

2. tongejb-cmr.xml 配置片段:

```

<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>accno</primkey-field>
</entity>

        <entity>
<ejb-name>Addressinfo</ejb-name>
<table-name>ADDRESSINFO</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>idl</primkey-field>
</entity>
</enterprise-beans>
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<many-to-many-table-name></many-to-many-table-name>

```

```

<ejb-relationship-role >
<ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
<relationship-role-source>
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>true</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>AddressinfoRelationShipRole</ejb-relationship-role-name>
<relationship-role-source >
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>false</foreign-key-column-on-source>
<role-mapping></role-mapping>
</ejb-relationship-role>
</ejb-relation>
</relationships>

```

3. tongejb-cmr.xml 配置片段:

```

<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>

```



```

<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>accno</primkey-field>
</entity>
<entity>
<ejb-name>Addressinfo</ejb-name>
<table-name>ADDRESSINFO</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>idl</primkey-field>
</entity>
</enterprise-beans>
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<many-to-many-table-name></many-to-many-table-name>
<ejb-relationship-role>
<ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
<relationship-role-source>
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>true</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>AddressinfoRelationShipRole

```

```

        </ejb-relationship-role-name>
    <relationship-role-source>
    <ejb-name>Addressinfo</ejb-name>
    </relationship-role-source>
    <cmr-field>
    <cmr-field-name>accno</cmr-field-name>
    </cmr-field>
    <foreign-key-column-on-source>false</foreign-key-column-on-source>
    <role-mapping>
    <cmr-field-mapping>
    <key-column>accno</key-column>
    <foreign-key-column>accno</foreign-key-column>
    </cmr-field-mapping>
    </ejb-relationship-role>
</ejb-relation>
</relationships>
</tongejb-cmr>

```

- **双向一对一**

Account Bean到Addressinfo Bean的双向一对一映射关系。在配置文件中的描述如下：

在ejb-jar.xml中，标签①和③定义一对一映射关系，标签②和④定义Account Bean到Addressinfo Bean的双向CMR域。

```

    <relationships>
    <ejb-relation>
    <ejb-relation-name>account-addressinfo</ejb-relation-name>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AccountRelationshipRole
    </ejb-relationship-role-name>
    <multiplicity>One</multiplicity>①
    <relationship-role-source>
    <ejb-name>Account</ejb-name>
    </relationship-role-source>
    <cmr-field>
    <cmr-field-name>addressinfo</cmr-field-name>
    </cmr-field> ②
    </ejb-relationship-role>
    <ejb-relationship-role>

```

```

    <ejb-relationship-role-name>
        AddressinfoRelationShipRole
    </ejb-relationship-role-name>

    <multiplicity>One</multiplicity >③
    <relationship-role-source>
    <ejb-name>Addressinfo</ejb-name>
    </relationship-role-source>

    <cmr-field>
    <cmr-field-name>account</cmr-field-name>④
    </cmr-field>
    </ejb-relationship-role>
    </ejb-relation>
    </relationships>

```

2.1.6.3 一对多映射

- 单向一对多

Account Bean到Addressinfo Bean的单向一对多映射关系。在配置文件中的描述如下。

在ejb-jar.xml中，标签①和③定义一对多映射关系，标签②定义Account Bean到Addressinfo Bean的单向CMR域。

```

    <relationships>
    <ejb-relation>
    <ejb-relation-name>account-addressinfo</ejb-relation-name>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AccountRelationshipRole
    </ejb-relationship-role-name>
    <multiplicity>One</multiplicity>①
    <relationship-role-source>
    <ejb-name>Account</ejb-name>
    </relationship-role-source>
    <cmr-field>
    <cmr-field-name>addressinfo</cmr-field-name>
    <cmr-field-type>java.util.Set</cmr-field-type>
    </cmr-field> ②
    </ejb-relationship-role>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AddressinfoRelationShipRole

```

```

        </ejb-relationship-role=name>
    </multiplicity>Many</multiplicity >③
    <relationship-role-source>
    <ejb-name>Addressinfo</ejb-name>
    </relationship-role-source>
    </ejb-relationship-role>
    </ejb-relation>
</relationships>

```

● 双向一对多

Account Bean到Addressinfo Bean的双向一对多映射关系。在配置文件中的描述如下：

1. 在ejb-jar.xml中，标签①和③定义一对多映射关系，标签②和④定义Account Bean到Addressinfo Bea的双向CMR域。

```

<relationships>
    <ejb-relation>
    <ejb-relation-name>account-addressinfo</ejb-relation-name>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AccountRelationshipRole
    </ejb-relationship-role-name>
    <multiplicity>One</multiplicity>①
    <relationship-role-source>
    <ejb-name>Account</ejb-name>
    </relationship-role-source>
    <cmr-field>
    <cmr-field-name>addressinfo</cmr-field-name>
    <cmr-field-type>java-util.Set</cmr-field-type>
    </cmr-field> ②
    </ejb-relationship-role>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AddressinfoRelationShipRole
    </ejb-relationship-role=name>
    <multiplicity>Many</multiplicity >③
    <relationship-role-source>
    <ejb-name>Addressinfo</ejb-name>
    <relationship-role-source>
    <cmr-field>

```

```
<cmr-field-name>account</cmr-field>④
</cmr-field>
</ejb-relationship-role>
</ejb-relation>
</relationships>
```

2. tongejb-cmr.xml配置片段:

```
<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>accno</primkey-field>
</entity>
<entity>
<ejb-name>Addressinfo</ejb-name>
<table-name>ADDRESSINFO</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>id1</primkey-field>
</entity>
</enterprise-beans>
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<many-to-many-table-name></many-to-many-table-name>
<ejb-relationship-role>
<ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
<relationship-role-source>
<ejb-name>Account</ejb-name>
```

```

</relationship-role-source>
<cmr-field >
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>true</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>AddressinfoRelationShipRole
    </ejb-relationship-role-name>
<relationship-role-source>
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>false</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</ejb-relationship-role>
</ejb-relation>
</relationships>
</tongejb-cmr>

```

2.1.6.4 多对一映射

- 单向多对一

Account Bean到Addressinfo Bean的单向多对一映射关系。在配置文件中的描述如下：

在ejb-jar.xml中，标签①和③定义多对一映射关系，标签②定义Account Bean到AddressinfoBean的单向CMR域。

```
<relationships>
```

```

<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AccountRelationshipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity>①
<relationship-role-source>
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>addressinfo</cmr-field-name>
</cmr-field> ②
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AddressinfoRelationShipRole
</ejb-relationship-role-name>
<multiplicity>One</multiplicity> ③
<relationship-role-source>
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
</ejb-relationship-role>
</ejb-relation>
</relationships>

```

● 双向多对一

Account Bean到Addressinfo Bean的双向多对一映射关系。在配置文件中的描述如下：

1. 在ejb-jar.xml中，标签①和③定义多对一映射关系，标签②和④定义Account Bean到Addressinfo Bean的双向CMR域。

```

<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AccountRelationshipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity>①

```

```

        <relationship-role-source>
        <ejb-name>Account</ejb-name>
        </relationship-role-source>
        <cmr-field>
        <cmr-field-name>addressinfo</cmr-field-name>
        </cmr-field> ②
    </ejb-relationship-role>
    <ejb-relationship-role>
    <ejb-relationship-role-name>
        AddressinfoRelationShipRole
    </ejb-relationship-role-name>
    <multiplicity>One</multiplicity> ③
    <relationship-role-source>
    <ejb-name>Addressinfo</ejb-name>
    <relationship-role-source>
    <cmr-field>
    <cmr-field-name>account</cmr-field-name>
    <cmr-field-type>java.util.Collection</cmr-field-type>
    </cmr-field> ④
    </ejb-relationship-role>
</ejb-relation>
</relationships>

```

2. tongejb-cmr.xml配置片段:

```

<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>accno</primkey-field>
</entity>
<entity>
<ejb-name>Addressinfo</ejb-name>

```



```

<table-name>ADDRESSINFO</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>id1</primkey-field>
</entity>
</enterprise-beans>
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<many-to-many-table-name></many-to-many-table-name>
<ejb-relationship-role >
<ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
<relationship-role-source >
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field >
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>true</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
<ejb-relationship-role >
<ejb-relationship-role-name>AddressinfoRelationShipRole
</ejb-relationship-role-name>
<relationship-role-source >
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>

```

```
<foreign-key-column-on-source>false</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</ejb-relationship-role>
</ejb-relation>
</relationships>
</tongejb-cmr>
```

2.1.6.5 多对多映射

- 单向多对多

Account Bean到Addressinfo Bean的单向多对多映射关系。在配置文件中的描述如下：

1. 在ejb-jar.xml中，标签①和③定义多对多映射关系，标签②定义Account Bean到Addressinfo Bean的单向CMR域。

```
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AccountRelationshipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity>①
<relationship-role-source>
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>addressinfo</cmr-field-name>
<cmr-field-type>java.util.Set</cmr-field-type>
</cmr-field> ②
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AddressinfoRelationShipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity> ③
<relationship-role-source>
```

```
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
</ejb-relationship-role>
</ejb-relation>
</relationships>
```

2. tongejb-cmr.xml配置片段:

```
<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>accno</primkey-field>
</entity>
<entity>
<ejb-name>Addressinfo</ejb-name>
<table-name>ADDRESSINFO</table-name>
<cmp-field-mapping>
<cmp-field-name>accno</cmp-field-name>
<cmp-field-class>java.lang.String</cmp-field-class>
<table-column>accno</table-column>
</cmp-field-mapping>
<primkey-field>id1</primkey-field>
</entity>
</enterprise-beans>
<relationships>
<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<many-to-many-table-name>ACCOUNTADDRESS</many-to-many-table-name>
<ejb-relationship-role>
<ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
<relationship-role-source>
<ejb-name>Account</ejb-name>
```

```

</relationship-role-source>
<cmr-field >
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>true</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
<ejb-relationship-role >
<ejb-relationship-role-name>AddressinfoRelationShipRole
    </ejb-relationship-role-name>
<relationship-role-source >
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>false</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
</ejb-relation>
</relationships>
</tongejb-cmr>

```

● 双向多对多

Account Bean到Addressinfo Bean的双向多对多映射关系。在配置文件中的描述如下：

1. 在ejb-jar.xml中，标签①和③定义多对多映射关系，标签②和④定义Account Bean到Addressinfo Bean的双向CMR域。

```
<relationships>
```

```

<ejb-relation>
<ejb-relation-name>account-addressinfo</ejb-relation-name>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AccountRelationshipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity>①
<relationship-role-source>
<ejb-name>Account</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>addressinfo</cmr-field-name>
<cmr-field-type>java.util.Set</cmr-field-type>
</cmr-field> ②
</ejb-relationship-role>
<ejb-relationship-role>
<ejb-relationship-role-name>
    AddressinfoRelationShipRole
</ejb-relationship-role-name>
<multiplicity>Many</multiplicity>③
<relationship-role-source>
<ejb-name>Addressinfo</ejb-name>
<relationship-role-source>
<cmr-field>
<cmr-field-name>account</cmr-field-name>
    <cmr-field-type>java.util.Set</cmr-field-type>
</cmr-field>④
</ejb-relationship-role>
</ejb-relation>
</relationships>

```

2. tongejb-cmr.xml配置片段:

```

<tongejb-cmr>
<enterprise-beans>
<entity>
<ejb-name>Account</ejb-name>
<table-name>ACCOUNT</table-name>
<cmp-field-mapping>

```

```

    <cmp-field-name>accno</cmp-field-name>
    <cmp-field-class>java.lang.String</cmp-field-class>
    <table-column>accno</table-column>
  </cmp-field-mapping>
  <primkey-field>accno</primkey-field>
</entity>

  <entity>
    <ejb-name>Addressinfo</ejb-name>
    <table-name>ADDRESSINFO</table-name>
    <cmp-field-mapping>
      <cmp-field-name>accno</cmp-field-name>
      <cmp-field-class>java.lang.String</cmp-field-class>
      <table-column>accno</table-column>
    </cmp-field-mapping>
    <primkey-field>id1</primkey-field>
  </entity>
</enterprise-beans>

<relationships>
  <ejb-relation>
    <ejb-relation-name>account-addressinfo</ejb-relation-name>
    <many-to-many-table-name>ACCOUNTADDRESS</many-to-many-table-name>
    <ejb-relationship-role >
      <ejb-relationship-role-name>AccountRelationshipRole</ejb-relationship-role-name>
      <relationship-role-source >
        <ejb-name>Account</ejb-name>
      </relationship-role-source>
    <cmr-field >
      <cmr-field-name>accno</cmr-field-name>
    </cmr-field>
    <foreign-key-column-on-source>true</foreign-key-column-on-source>
  </role-mapping>
  <cmr-field-mapping>
    <key-column>accno</key-column>
    <foreign-key-column>accno</foreign-key-column>
  </cmr-field-mapping>
</role-mapping>
</ejb-relationship-role>
  <ejb-relationship-role >

```

```
<ejb-relationship-role-name>AddressinfoRelationShipRole
    </ejb-relationship-role-name>
<relationship-role-source >
<ejb-name>Addressinfo</ejb-name>
</relationship-role-source>
<cmr-field>
<cmr-field-name>accno</cmr-field-name>
</cmr-field>
<foreign-key-column-on-source>false</foreign-key-column-on-source>
<role-mapping>
<cmr-field-mapping>
<key-column>accno</key-column>
<foreign-key-column>accno</foreign-key-column>
</cmr-field-mapping>
    </role-mapping>
</ejb-relationship-role>
</ejb-relation>
</relationships>
</tongejb-cmr>
```

2.1.7 数据源配置

数据源在persistence.xml中配置，配置如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
<persistence-unit name="cmp" transaction-type="JTA">
<provider>com.tongweb.openjpa.persistence.PersistenceProviderImpl</provider>
<jta-data-source>OneDB</jta-data-source>
</persistence-unit>
</persistence>
```

2.2 EJB3.0

EJB2.x与EJB3.0的变化：

1. 在所有的EJB3.0规范中定义的EJB，都不必实现Home接口。

2. 用户不需要创建部署描述文件 `ejb-jar.xml`，而使用标注(Annotation)替代。尽管这样，用户还是可以使用部署描述文件，如果部署描述文件和 Annotation 的内容发生冲突，则 Annotation 的内容将被部署描述文件中的内容覆盖。
3. 因使用标注(Annotation)，Bean 文件中不必实现 `javax.ejb.SessionBean` 或 `javax.ejb.MessageDrivenBean`。
4. 因不必实现 `javax.ejb.SessionBean` 或 `javax.ejb.MessageDrivenBean`，所以也不必实现生命周期回调方法，如 `ejbCreate`，`ejbPassivate` 等。
5. Bean 文件需要定义业务逻辑接口，并且需要使用标注 `@javax.ejb.Remote` 或 `@javax.ejb.Local`。
6. 业务逻辑接口不需要扩展 `javax.ejb.EJBObject` 或 `javax.ejb.EJBLocalObject`。
7. 业务逻辑接口中的方法不必 throw `java.rmi.RemoteException`，除非该业务逻辑接口扩展 `java.rmi.Remote`。

EJB3.0的新特性：

1. Bean 文件中可使用标注(Annotation)，替代部署描述文件。
2. Bean 文件中仅需的标注是表明 EJB 类型的标注，`@javax.ejb.Stateless`、`@javax.ejb.Stateful`、`@javax.ejb.MessageDriven`、`@javax.ejb.Entity`。
3. 支持依赖注入(Dependency Injection)。
4. 支持拦截器(Interceptors)功能。
5. 用户可以配置两类拦截器，一类用于拦截业务方法(business methods)，一类用于拦截生命周期回调(lifecycle callback)。
6. 用户可以配置多个拦截器，并以拦截器链(Chain)的方式执行。
7. 用户可以配置默认的拦截器，该拦截器对 jar 文件中的所有 EJB 生效。
8. 在应用结构上多了一个 `persistence.xml` 配置文件，是 Entity 使用 JPA 时所需要的持久化配置文件。

2.2.1 SessionBean

SessionBean的开发步骤：

1. 定义业务逻辑接口
2. 实现业务逻辑接口
3. 编译&打包
4. 部署
5. 查找和调用EJB3.0

2.2.1.1 StatelessBean

Stateless SessionBean的开发步骤及示例：

1. 定义业务逻辑接口。示例如下：

```
package examples;
import javax.ejb.Remote;
```



```
@Remote
public interface Service
{ public void sayHello(); }
```

2. 实现业务逻辑接口。示例如下：

```
package examples;
import static javax.ejb.TransactionAttributeType.*;
import javax.ejb.Stateless;
import javax.ejb.TransactionAttribute;
import javax.interceptor.ExcludeDefaultInterceptors;
@Stateless
@TransactionalAttribute(NEVER)
@ExcludeDefaultInterceptors
public class ServiceBeanimplements Service
{
    public void sayHello()
    {
        System.out.println("Hello Hello!");
    }
}
```

说明：

- @Stateless表示该EJB类型为Stateless SessionBean
- @TransactionalAttributes (NEVER) 表示EJB的事务属性
- @ExcludeDefaultInterceptors表示配置中定义的默认拦截器不会被调用。

3. 通过开发工具或者命令行编译&打包。

4. 通过自动部署或者管理控制台部署该 EJB 应用。

5. 查找和调用 EJB3.0。

通过lookup的方式获取EJB资源。Session Bean的JNDI名默认为“业务逻辑接口的包名+类名”，如“test.TestSessionBean”。

说明：

在EJB中如何修改JNDI名：使用mappedName来修改JNDI名,如@Stateless(mappedName=“修改后的JNDI名”)或@Stateful(mappedName=“修改后的JNDI名”)

2.2.1.2 StatefulBean

Stateful SessionBean的开发步骤及示例：

1. 定义业务逻辑接口，示例如下：

```
public interface Account {
    public void deposit(int amount);
    public void withdraw(int amount);
    public void sayHelloFromAccountBean();
}
```

2. 实现业务逻辑接口，示例如下：

```
package examples;
import static javax.ejb.TransactionAttributeType.*;
import javax.ejb.Stateful;
import javax.ejb.TransactionAttribute;
import javax.ejb.Remote;
import javax.ejb.EJB;
import javax.annotation.PreDestroy;
import javax.interceptor.Interceptors;
import javax.interceptor.ExcludeClassInterceptors;
import javax.interceptor.InvocationContext;

@Stateful
@TransactionAttribute(NEVER)
@Remote({examples.Account.class})
@Interceptors({examples.AuditInterceptor.class})
public class AccountBean
implements Account
{
    private int balance = 0;
    @EJB(beanName="ServiceBean")
    private Service service;
    public void deposit(int amount) {
        balance += amount;
        System.out.println("deposited: "+amount+" balance: "+balance);
    }
    public void withdraw(int amount) {
        balance -= amount;
        System.out.println("withdrew: "+amount+" balance: "+balance);
    }
    @ExcludeClassInterceptors
    public void sayHelloFromAccountBean() {
```

```

service.sayHelloFromServiceBean();
}
@PreDestroy
public void preDestroy() {
System.out.println("Invoking method: preDestroy()");
}
}

```

说明：

- @Stateful 表示该 EJB 类型为 Stateful SessionBean。
- @TransactionAttributes(NEVER) 表示 EJB 的事务属性。
- @Interceptors({examples.AuditInterceptor.class}) 表示与该 Bean 文件关联的拦截器。默认情况下所有的业务方法调用拦截器类中以@AroundInvoke 标注的方法。

3. 通过开发工具或者命令行编译&打包。

4. 通过自动部署或者管理控制台部署该 EJB 应用。

5. 查找和调用 EJB3.0。

通过lookup的方式获取EJB资源。Session Bean的JNDI名默认为“业务逻辑接口的包名+类名”。

说明：

在EJB中如何修改JNDI名：使用mappedName来修改JNDI名，如@Stateless(mappedName="修改后的JNDI名")或@Stateful(mappedName="修改后的JNDI名")

2.2.2 定时服务

定时服务用于在一段特定时间后执行某段程序，在EJB3.0之前可以用在Stateless Session Bean、Message-Driven Bean 和 EntityBean 中，在每个需要 Timer 服务的 EJB bean 类，均要实现 javax.ejb.TimedObject接口的定时器回调方法ejbTimeout()，该方法是计时器到期时要回调的方法。在EJB3.0中可以用在Stateless Session Bean、Message-Driven Bean和Entity中使用@Resource 标注注入定时服务，使用容器对象SessionContext创建定时器。使用@Timeout标注将某方法声明为定时器回调方法。

定时器的定义和使用步骤如下：

1. 定义定时器接口，代码如下：

```

package ejb;

public interface TimerTest {

    public void schedule(long m);

}

```

2. 实现定时器接口，代码如下：

```

package ejb;

```

```
import javax.ejb.Stateless;
import javax.ejb.Remote;
import javax.ejb.SessionContext;
import javax.ejb.Timeout;
import javax.ejb.Timer;
import javax.annotation.Resource;
import java.util.Date;
import ejb.TimerTest;
@Remote
public @Stateless class TimerTestBean implements TimerTest {
    private @Resource SessionContext ctx;
    private int count = 0;
    public void schedule(long m){
        ctx.getTimerService().createTimer((new Date(new Date().getTime() + m)),m,"testtest!");
    }
    @Timeout
    public void timeoutHandle(Timer timer){
        System.out.println(timer.getNextTimeout() + "" + count + "" + timer.getInfo());
        count ++;
        if(count > 5){
            System.out.println("Timer canceled!");
            timer.cancel();
            System.out.println("Timer canceled!");
        }
    }
}
```

说明:

通过依赖注入@Resource SessionContext ctx, 获得SessionContext对象, 调用 ctx.getTimerService().createTimer(Date arg0, long arg1, Serializable arg2)方法创建定时器, 使用@Timeout标注声明定时器方法。

3. 通过开发工具或者命令行编译&打包。
4. 通过自动部署或者管理控制台部署该 EJB 应用。
5. 客户端调用, 代码如下:

```
InitialContext ctx = new InitialContext();
TimerTest tt = (TimerTest) ctx.lookup("ejb.TimerTestBean");
```

```
tt.schedule(30001);
```

2.2.3 拦截器

拦截器可以监听程序的一个或所有方法。拦截器对方法调用流提供了细粒度控制，可以在无状态会话bean及有状态会话bean上使用它们。

- 拦截器可以是同一 Bean 类中的方法或者是一个外部类。
- 拦截器可以拦截 Session Bean 的方法调用或生命周期事件。
- 拦截器按照使用范围和定义可以分为类级，方法级，和 Bean 级，其中类级和方法级的拦截器的实现是外部类。
 - 类级用标注@Interceptors 定义在 Bean 的类名前，作用范围是 Bean 类的所有方法。
 - 方法级用标注@ Interceptors 定义在 Bean 类的某一方法前，作用范围是该方法。
 - Bean 级的拦截器实现是 Bean 类的某一方法（非业务方法），在该方法前使用标注@AroundInvoke 表示其为用做拦截器的方法，作用范围是除了拦截器方法自身之外 Bean 类的所有方法。
 - Bean 级别定义的拦截器方法只允许一个。类级别和方法级别的拦截器允许有多个。
 - 以上使用标注的情况均可在部署描述符中定义。
- 拦截器的定义可以使用部署描述符和标注，部署描述符优先级高于标注，部署描述符支持通配符定义拦截器作用范围。
- 多个拦截器的执行顺序：定义在外部类的拦截器的执行优先级别高于定义在 Bean 类中的优先级别，同一级别的按照定义顺序执行。
- 拦截器的实现规则：必须使用标注@AroundInvoke 表示其为用作拦截器的方法，这一方法必须遵守以下格式：`public Object xxx(javax.interceptor.InvocationContext ctx) throws Exception`，其中 xxx 可以任意取名，`javax.interceptor.InvocationContext` 封装了客户端所调用业务方法的一些信息。必须调用 `InvocationContext` 对象的 `proceed()` 方法作为返回结果，否则其他拦截器和业务方法不会执行。
- 拦截器的使用：如果不希望某些 Bean class 使用拦截器，可以在其名称上面加入 `@ExcludeDefaultInterceptors` 标注。此外，拦截器中也可以使用注入。

拦截器的定义和使用步骤如下：

1. 定义拦截器，代码如下：

```
import javax.interceptor.AroundInvoke;
import javax.interceptor.InvocationContext;
public class HelloInterceptor {
    @AroundInvoke
```

```

public Object log(InvocationContext ctx) throws Exception {
    System.out.println("*** HelloInterceptor intercepting");
    long start = System.currentTimeMillis();
    try{
        if (ctx.getMethod().getName().equals("SayHello")){
            System.out.println("*** SayHello 已经被调用! *** ");
        }
        if (ctx.getMethod().getName().equals("Myname")){
            System.out.println("*** Myname 已经被调用! *** ");
        }
        return ctx.proceed();
    }catch (Exception e) {
        throw e;
    }finally {
        long time = System.currentTimeMillis() - start;
        System.out.println("用时:" + time + "ms");
    }
}
}

```

说明:

@AroundInvoke 注释指定了要用作拦截器的方法. 用@AroundInvoke注释指定的方法必须遵守以下格式:
 public Object XXX(InvocationContext ctx) throws Exception 其中XXX代表方法名可以任意.

2. 使用外部拦截器，以下代码为在 SessionBean 中使用外部拦截器。

业务逻辑接口：

```

@Remote
public interface HelloChinaRemote {
    public String SayHello(String name);
    public String Myname();
}

```

业务接口的实现类：

```

import javax.ejb.Local;
import javax.ejb.Remote;
import javax.ejb.Stateless;
import javax.interceptor.Interceptors;

@Stateless
@Interceptors({HelloInterceptor.class})
public class HelloChinaBean implements HelloChinaRemote {

```

```

    public String SayHello(String name) {
        return name + "Hello";
    }

    public String Myname() {
        return "Myname";
    }
}

```

说明：

@Interceptors标注指定一个或多个在外部类中定义的拦截器。

2.2.4 依赖注入

在J2EE1.4以前，EJB组件查找Java EE环境中的资源(引用其它EJB，数据源等)需要通过JNDI的查找方式来完成。从JavaEE5开始，EJB组件可以通过依赖注入的方式获取。依赖注入机制的引入大大简化了EJB的开发量，提高了开发效率。

2.2.4.1 引用EJB

在EJB3.0中，引入了一个标注@EJB，该标注可以用于标注一个类、实例变量或者一个方法。下面是标注一个实例变量的例子：

```

@EJB
private TestBeanRemote testremote;
.....
public void sayHello() {
    testremote.test();
}

```

@EJB标注的name，beanName，mappedName属性不同的使用场合

1. @EJB 不带任何属性使用

当TestBeanRemote接口在ejb-jar中只被一个Bean class使用时，可以这样引用。例如：

```

@EJB
private TestBeanRemote testremote;

```

说明：默认的JNDI名为“包名.类名/testremote”

2. @EJB 带 beanName 属性使用

当业务接口被多个Bean class使用时，必须指定beanName。例如：

```

@Stateless
@Remote(HelloRemote.class)
public class HelloBean implements HelloRemote{

```

```

    public String SayHello(String name){
        return name+"HelloBean";
    }
}

@Stateless
@Remote(Hello.class)
public class HelloManBean implements HelloRemote{
    public String SayHello(String name){
        return name+"HelloManBean";
    }
}

```

如果引用的EJB不在@EJB使用类所在的Jar文件中，而在ear的其他jar中，则可以通过下面形式进行引用

```

@EJB(beanName="a.jar#HelloRemote")
private HelloRemote hello;

```

3. @EJB 带 name 属性使用

带name属性的EJB引用，其name值被注册到组件名空间中而不是全局名空间中。例如：

```

@EJB(name="ejb/HelloTest")
private HelloRemote hello;

```

4. @EJB 带 mappedName 属性使用

mappedName属性中定义全局的JNDI名。使用该引用方式时，容器根据mappedName的值直接从全局JNDI中查找EJB引用。例如：

```

@EJB(mappedName="HelloTest")
private HelloRemote hello;

```

2.2.4.2 引用Resource

在EJB3.0中，引入了标注@Resource，该标注可以用于引用外部资源，包括JDBC数据源。@Resource可用于标注一个类、实例变量或者一个方法。

下面是在**实例变量**中引用数据源的例子：

```

@Resource(name="testjdbc",mappedName="ds")
private DataSource ds;

```

说明：

例子中name属性表示被引用资源在应用服务器的全局JNDI名，mappedName属性为该资源在EJB对象的命名环境中的名字。

下面是在**方法**中引用数据源的例子：

```
private DataSource ds;
@Resource(name="testjdbc")
private void setDs(DataSource myds) {
    this.ds = myds;
}
```

下面使用@Resource标注一个**类**：

```
@Resource(name="testjdbc", type=javax.sql.DataSource.class)
public class HelloWorldSessionBean implements
HelloWorldSessionRemote, HelloWorldSessionLocal {
    @Resource
    private SessionContext sessionContext;
    public String testDataSource()
    { DataSource ds=(DataSource)sessionContext.lookup("testjdbc"); return ds.toString(); }
}
```

说明：

当用@Resource标注一个类时，相当于在其命名环境中定义了对资源的引用。在EJB的bean实例中可以通过方法context.lookup("testjdbc")来查找相应的资源。其中lookup方法的参数名字就是标注@Resource中name属性定义的值。

2.2.5 Pool和Cache

实例池(Pool)

为提高无状态会话Bean的响应性能，EJB容器提供了实例池（Pool）机制进行Bean实例管理。

为了避免EJB容器为用户的每次方法调用都进行实例的创建与销毁，因而影响系统性能，减少系统吞吐量，EJB容器提供了EJB池（实例池）来应对大量用户的访问。

当EJB应用部署到应用服务器时，EJB容器会预先创建一定数量的Bean实例放入EJB池，由于客户端应用程序不会直接访问真正的Bean实例，而是通过EJB对象所实现的接口与Bean实例进行交互，因此EJB服务器也就不需要为每个客户端应用分别维护相应的Bean实例，而是只需维护足以完成工作的少量Bean实例，并重复使用这些Bean实例为不同的请求提供服务就可以了。

如果实例池中的所有实例都在处理请求，那么EJB容器首先会判断实例池中该Bean的实例数是否已经达到了“最大实例数”，如果没有达到这个最大实例数，EJB容器会创建新的Bean实例放入实例池中，并为客户端服务。如果达到了“最大实例数”，那么EJB容器会挂起请求，等待有空闲的实例可用，直到超时为止。如果池中的Bean处于空闲状态并持续一段时间，EJB容器会销毁一定数量的Bean实例。

实例缓存(Cache)：

对于EJB池，因为EJB容器不为每个客户端分别维护相应的Bean实例，同时还重复使用这些Bean实例，所以实例池不适用于有状态的会话Bean。对于有状态的会话Bean，因为EJB容器需要为每个客户端分别维护相应的Bean实例，所以EJB池不适用，但是EJB容器为其提供了EJB缓存、钝化和激活机制来管理大量Bean实例。

对有状态的会话Bean来说，由于每个用户都有自己的Bean实例，那么有多少个用户就有多少个Bean实例为之服务，如果不采用任何机制，势必会导致服务器资源的严重浪费。

EJB容器提供了EJB缓存来维护使用过的EJB，这使得对EJB的请求能够被更快的响应，而当Bean实例在一定时间内没有被再次访问的话，EJB容器将其状态数据保存到硬盘，并从EJB对象和缓存中删除此实例，释放缓存，这一过程称为钝化。

而当客户机请求此Bean时，容器会自动在缓存中创建一个新的实例，并按照钝化期间保存的内容逐一设置数据成员，这一过程称为激活。

2.2.6 配置资源引用

同Web应用的“[配置资源引用](#)”章节。只是将web.xml改成ejb-jar.xml，将tongweb-web.xml改成tongweb-ejb-jar.xml。

2.3 EJB3.1

EJB3.1的新特性

1. 无接口的 EJB

Session Bean不再需要额外的接口，只需要给一个Pojo标注上@Stateless或@Stateful就可完成Session Bean的功能。

2. 引入 SingletonBeans

通过@Singleton标记该SessionBean为Singleton。在该Container只有一个实例。同时单例bean通过锁机制支持多客户端并发调用。

3. 异步调用

在EJB3.1中，通过@Asynchronous使该方法可以异步调用。异步调用的方法可以返回一个java.util.concurrent API的Future对象，客户端可以通过这个Future对象获取调用的状态。

@Asynchronous这个注解可以用于类或者方法，如果用于类或者接口，那么类和接口所有的方法都是异步的。

4. 统一的 JNDI 命名

Ejb3.1规范中统一了ejb的jndi命名，以实现不同容器间的兼容性，格式如下：

```
java:global[/<app-name>]/<module-name>/<bean-name>[!<fully-qualified-interface-name>]
java:app/<module-name>/<bean-name>[!<fully-qualified-interface-name>]
java:module/<bean-name>[!<fully-qualified-interface-name>]
```

假设fooapp.ear应用中的fooejb.jar中定义了一个名为FooBean的ejb，那么获取这个bean实例的jndi名称有以下几种方式：

```
-java:global/fooapp/fooejb/FooBean
-java:global/fooapp/fooejb/FooBean!com.acme.Foo
-java:app/fooejb/FooBean
-java:app/fooejb/FooBean!com.acme.Foo
-java:module/FooBean
-java:module/FooBean!com.acme.Foo
```

5. 简化的部署

EJB 3.1 中一个重要的改进是可以直接将EJB组件打包到WAR文件中，不用再独立创建jar。如下图所示：

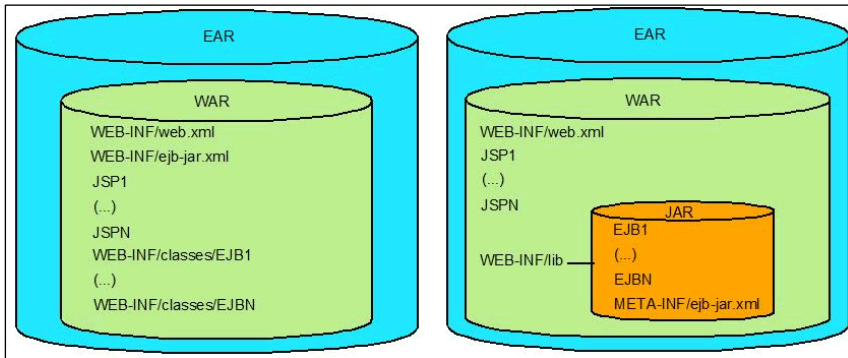


图 2.3-1：EJB3.1简化的部署

6. EJB Lite 的推出

许多企业级应用不需要EJB完整的功能，因此在EJB 3.1中引入了EJB Lite，它是EJB API的一个子集。EJB Lite包括了创建一个企业级应用的所有功能，但不包括专业的API。EJB Lite提供了厂家选项，让厂家可以在它们自己的产品中实施EJB API的子集，使用EJB Lite创建应用程序可以部署到任何支持EJB的服务器上，不管它是完整的EJB还是EJB Lite。

7. 更强大的 Timer 服务

EJB3.1版本时，有两种方式来创建timer：

- 编程式：使用现有的TimerService接口，并且为了更加灵活的创建timer，对原有接口有了很大的改进和提高。
- 声明式：使用注解@Schedule或部署描述符号来实现。采用这种方式的话，timer就以静态的形式定义在应用程序中，然后在应用程序启动时，自动创建。只有被@Schedule标注过的方法才可以任务调度。

定时器新特性主要体现在以下两个方面：

- 自动创建EJB定时器
- Calendar-Based Time Expressions
 - a. 列表：如每一分钟内每到第15、25这两个秒数时。
 - b. 范围：如每一分钟内每到0-5这个范围的秒数时。
 - c. 增量：如每一分钟内30秒之后每过10秒。

通过@Schedule可以定义自动创建的ejb定时器。以及更灵活的表达式定时，例如：

dayOfWeek = “Mon, Wed, Fri” 定义了定时执行的列表，只有在每周的这三天才执行；
 dayOfWeek = “Fri-Mon” 定义了定时执行的范围，在每周的在各分时间范围内才执行；
 second = “30/10” 定义了定时的增量，30秒的时候开始执行，每10秒执行一次。

2.3.1 Session Bean

EJB3.1带接口的session bean开发步骤及实例同EJB3.0相同，可以参见EJB3.0的“[Session Bean](#)”章节。

无接口的session bean的开发步骤如下。

1. 编写业务逻辑实现类，在该类上加注解@Stateless或@Stateful即可。
2. 编译&打包
3. 部署应用
4. 查看或调用EJB3.1

2.3.2 Singleton Bean

Singleton Bean的开发步骤如下。

1. 定义业务逻辑接口（可省略，此处不写）
2. 实现业务逻辑接口（开发业务实现类），例如：

```
package examples;
import javax.ejb.Lock;
import javax.ejb.Singleton;
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.Map;
@Singleton
@Lock(READ)
public class ComponentRegistry{
    private final Map<Class, Object> components = new HashMap<Class, Object>();
    public <T> T getComponent(Class<T> type) {
        return (T) components.get(type);
    }
    @Lock(WRITE)
    public <T> T setComponent(Class<T> type, T value) {
        return (T) components.put(type, value);
    }
}
```

说明：

- @Singleton：定义这个类为单例bean。
- @Lock：控制并发的锁，有read和write两个值。缺省情况下，lock的属性值是write。当某个方法持有write lock时，容器只允许其中一个并发线程去调用该方法。其它线程必须等待，直到该方法再次变

得可用。@Lock可以用于类、接口和方法。如其它Annotation类似，@Lock也有继承性。在类级别使用@Lock，它的所有相关方法也会被应用上，除非单独限制某一个具体的方法。

- 下面是一个bean处理并发的单例bean定义。按照这种方式定义bean，@Lock将不会生效，同步控制由bean自己来做。

```
@Singleton
@Startup
@ConcurrencyManagement(BEAN)
publicclass PropertyRegistry {
    publicvoid applicationStartup() {
    }
}
```

3. 通过开发工具或者命令行编译&打包。
4. 通过自动部署或者管理控制台部署该 EJB 应用。
5. 查看或调用 EJB3.1。通过 lookup 查找 jndi 名的方式获取 EJB 资源。

2.3.3 定时服务

定时服务用于在一段特定时间后执行某段程序，EJB3.1在3.0的基础上，提供了增强的定时服务功能。定时器的定义和使用步骤如下：

1. 定义定时器接口

```
package com.tongweb.test;

public interface Timer {
    int getChecks();
}
```

2. 实现定时器接口

```
package com.tongweb.test.ejb;

import com.tongweb.test.Timer;
import java.util.concurrent.atomic.AtomicInteger;
import javax.ejb.Lock;
import javax.ejb.LockType;
import javax.ejb.Schedule;
import javax.ejb.Singleton;

@Singleton
@Lock(LockType.READ)
```

```

public class TimerBean
{
    implements Timer{
        private final AtomicInteger checks = new AtomicInteger();
        @Schedule(second="*/1", minute="*", hour="*")
        private void checkOnTheDaughters() {
            this.checks.incrementAndGet();
        }
        public int getChecks() {
            return this.checks.get();
        }
    }
}

```

3. 通过开发工具或者命令行编译&打包。
4. 通过自动部署或者管理控制台部署该 EJB 应用。
5. 客户端通过两种方式调用：
 - @EJB注入的方式
 - 通过lookup方式调用

2.3.4 拦截器

拦截器的定义和使用步骤见EJB3.0的“[拦截器](#)”章节。

2.4 EJB3.2

EJB3.2的新特性

EJB3.2规范相对于EJB3.1及以前的规范没有革命性的变化，仅仅是对以前具有的功能的扩展和补充，以及放宽了某些实现限制，朝着使EJB轻量和易用的目标更进了一步。

1. 扩展了 EJB3.2Lite 功能集，增加了支持本地异步会话 Bean 调用和非持久性 EJB Timer Service。这个只是将 EJB 3.1Full 规范中的内容加入到 EJB3.1Lite 规范中。
2. 可以完全禁用特定的有状态会话 bean 的钝化（passivation）。

开发有状态会话Bean时，可使用属性passivationCapable来定义是否可以钝化该有状态会话Bean，例如：

```

@Stateful(passivationCapable = false)
public class CartBean implements Cart {
    List<String> items;
    public CartBean() {
        items = new ArrayList<>();
    }
}

```

```

    }

    @Override
    public void addItem(String item) {
        items.add(item);
    }

    @Override
    public void removeItem(String item) {
        items.remove(item);
    }

    @Override
    public void purchase() {
        //. . .
    }

    @Override
    public List<String> getItems() {
        return items;
    }

    @Remove
    public void remove() {
        items = null;
    }
}

```

说明：该示例定义了有状态会话Bean不可钝化。

3. 扩展了 TimerService API，可以在同一个 EJB 模块中查询所有活动计时器。例如：

```

@Startup
@Singleton
public class AutomaticBean {
    @Resource
    SessionContext ctx;

    public void cancelTimes() {
        Collection<Timer> timers = ctx.getTimerService().getAllTimers();
        for (Timer t : timers) {
            System.out.println("Timer " + t.getInfo() + " will be canceled !");
            t.cancel();
        }
    }
}

```

说明：

通过@Resource注入获取ctx，然后通过ctx获取EJB Timer Service, 然后通过其新增API, getAllTimers() 来获取所有活动定时器，并进行取消。

第3章 JPA

JPA (Java Persistence API, Java持久化API), 定义了对象-关系映射 (ORM) 以及实体对象持久化的标准接口。

JPA1.0是JSR-220 (EJB3.0) 规范的一部分, 在JSR-220中规定实体对象 (EntityBean) 由JPA进行支持。

JPA2.0已经迁移到JSR-317 (Java Persistence 2.0), 已经不局限于EJB3.0, 而是作为POJO持久化的标准规范。TongWeb6.1的JPA实现框架Openjpa 2.2.0则是实现了JSR-317规范。

JPA2.1 (JSR-338) 相对于JPA2.0来说, 也有很多功能改进, 比如加入了存储过程等功能。TongWeb7的JPA实现框架为eclipselink-2.6.3则是实现了JSR-338。

3.1 JPA2.0新增功能

JPA1.0 规范功能上已经比较完整了。而JPA2.0中又引入的一些新特性, 而这些新特性特别是对对象/关系 (O/R) 映射和域建模方面的特性都是针对“边缘”情况的, 不是常用的场景类型。

其他主要新特性, 比如Criteria APIs等, 只是对JPA的持久性功能的锦上添花。下面大致介绍一下新特性的使用。

1. 对象/关系映射增强

表 3.1-1: 对象关系映射增强

Annotation	说明
@ElementCollection	指定集合中的基本数据类型或可嵌入的对象被存储于一个叫做集合表的分离的表中。
@CollectionTable	指定关于这个集合的细节, 如它的列columns。

2. Java 持久化查询语言增强

JPA2.0加强了Java持久查询语言, 如可以在查询语句中使用case表达式。Jpql用于查询实体及它们的状态, 比如用case表达式增加员工的薪水, 如果员工的级别为1则乘以1.1; 如果员工的级别为2则乘以1.05; 否则乘以1.01。

```
UPDATE Employee e
SET e.salary =
CASE WHEN e.rating = 1 THEN e.salary * 1.1
WHEN e.rating = 2 THEN e.salary * 1.05
ELSE e.salary * 1.01
END
```

3. 一种新的基于标准的查询 API (Criteria APIs)

JPA2.0引入了一种新的查询方式criteria API。使用这种API可以采用面向对象的方法去构造查询，而不是像jpql那样使用字符串方式处理。例如一个普通的查询语句可以修改为：

```
public List<User> findTest() {
    CriteriaBuilder cb = manager.getCriteriaBuilder();
    CriteriaQuery<User> cq = cb.createQuery(User.class);
    Root<User> emp = cq.from(User.class);
    EntityType<User> emp_ = emp.getModel();
    cq.select(emp);
    cq.where(cb.equal(emp.get(emp_.getSingularAttribute("user",
String.class)), "user_asdf"));
    TypedQuery<User> query= manager.createQuery(cq);
    return query.getResultList();
}
```

4. 支持悲观锁定

3.2 JPA2.1新增功能

Java EE7带来了JPA 2.1，它也包含了许多新特性和改进。

- 以标准化的方式支持存储过程。
- 支持属性转换器，实现基本值在数据库表示和相应对象表示之间进行转换。
- 支持实体图，可以用于定义一个实体及其子元素的加载方式。
- 支持通过Criteria批量的更新和删除数据。
- 支持通过CDI实现实体类生命周期事件监听。
- 支持通过SynchronizationType控制当前事务持久化上下文的同步。
- 支持借助于@ConstructorResult注解能够使用从一个SQL查询返回的参数值构造对象。

1. 支持存储过程

在JPA 2.0中，并没有针对存储过程的实际支持，本地查询是调用存储过程的唯一方式。JPA2.1. 引入了@NamedStoredProcedureQuery和更为动态的StoredProcedureQuery，改变了这一现状。

在下面代码段中可以看到，@NamedStoredProcedureQuery的定义非常简洁，只需要指定查询的名称、数据库中的存储过程名称以及输入和输出参数。在本例中，用输入参数x和y调用存储过程calculate，期望的输出参数为sum，其它支持的参数类型还有用于输入和输出的参数INPUT和用于检索结果集的REF_CURSOR。

```
@NamedStoredProcedureQuery(
    name = "calculate",
    procedureName = "calculate",
```

```
parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, type = Double.class, name = "x"),
    @StoredProcedureParameter(mode = ParameterMode.IN, type = Double.class, name = "y"),
    @StoredProcedureParameter(mode = ParameterMode.OUT, type = Double.class, name =
"sum") })
```

@NamedStoredProcedureQuery 的使用方法与 @NamedQuery 相似，需要向实体管理器的 createNamedStoredProcedureQuery 方法提供查询名称，以便在本次查询中获取 StoredProcedureQuery 对象，然后，用 setParameter 方法设定输入参数，之后再用 execute 方法调用存储过程。

```
StoredProcedureQuery query = this.em.createNamedStoredProcedureQuery("calculate");
query.setParameter("x", 1.23d);
query.setParameter("y", 4.56d);
query.execute();
Double sum = (Double) query.getOutputParameterValue("sum");
```

2. 支持属性转换器

JPA2.1 使用 AttributeConverter 属性类型转换器。关于 AttributeConverter<X, Y> 说明如下。

- X 是实体属性的类型
- Y 是数据库字段的类型
- Y convertToDatabaseColumn(X) 作用：
将实体属性 X 转化为 Y 存储到数据库中，即插入和更新操作时执行。
- X convertToEntityAttribute(Y) 作用：
将数据库中的字段 Y 转化为实体属性 X，即查询操作时执行。

3. 支持实体图

根据经验，使用过多的 SQL 查询获取所要求的实体是导致性能问题最普遍的原因。即使是看起来最简单的查询，如果操作不当，也会触发几十次甚至上百次的 SQL 查询。为避免这一情况 JPA2.1 提供了 @NamedEntityGraph 来解决此问题。

实体图通过独立于查询的方法指定应该从数据库中获取的实体的图。这意味着，需要为实体图创建一个独立的定义，并在需要时与查询合并。

下段代码展示了如何定义根据作者名提取书名的 @NamedEntityGraph。

```
@Entity
@NamedEntityGraph(name = "graph.AuthorBooks", attributeNodes =
@NamedAttributeNode("books"))
public class Author implements Serializable {
    ...
}
```

实体管理器可以用这个图作为参考，通过一次查询获取所有作者和书名。在图的定义中可以看到，Author只提供了包含相关实体的属性名称。因此，Author将@NamedEntityGraph作为loadgraph（负载图），这样便可提取其他所有属性及其定义的获取类型，如下所示：

```
EntityGraph graph = this.em.getEntityGraph("graph.AuthorBooks");
List authors = this.em.createQuery("SELECT DISTINCT a FROM Author a",
    Author.class).setHint("javax.persistence.loadgraph", graph).getResultList();
```

该示例展示了一个非常简单的实体图。在实际的应用中，很可能会用到更复杂的图，但这也不成问题。可以定义多个@NamedAttributeNodes以定义更复杂的图，也可以用@NamedSubGraph注解来创建多层次的图。

4. 支持通过 Criteria 批量的更新和删除数据

逐个更新实体是造成JPA性能问题的另一个常见原因。Java开发者习惯处理对象，并用面向对象的方式思考问题。尽管这是实现复杂逻辑和应用的好方法，但也是处理数据库时导致性能退化的一个常见原因。

从面向对象的角度来看，对实体进行更新和删除操作是完全可以接受的。但当不得不更新一大组实体时，这种操作就会非常低效。持久性提供者(Persistence Provider)将为每个更新实体创建一个更新语句，并在下一次flush操作时发送至数据库中。

SQL提供了一个更为高效的方式。它允许创建可一次性更新多个实体的更新语句，还可以对JPA 2.1引入的CriteriaUpdate和CriteriaDelete语句进行同样的操作。

在下面的代码段中可以看到，需要从实体管理器中获取CriteriaBuilder并用它创建CriteriaUpdate对象，对CriteriaQuery进行的操作与此类似，主要区别在于用于定义更新操作的set方法。

```
CriteriaBuilder cb = this.em.getCriteriaBuilder();
// create update
CriteriaUpdate update = cb.createCriteriaUpdate(Author.class);
// set the root class
Root a = update.from(Author.class);
// set update and where clause
update.set(Author_.firstName, cb.concat(a.get(Author_.firstName), " - updated"));
update.where(cb.greaterThanOrEqualTo(a.get(Author_.id), 3L));
// perform update
Query q = this.em.createQuery(update);
q.executeUpdate();
```

在CriteriaDelete操作中，只需要在实体管理器中调用createCriteriaDelete方法以获取CriteriaDelete对象，并用它来定义与上例类似的FROM和WHERE查询部分。

5. 支持通过 CDI 实现实体类生命周期事件监听

JPA2.1支持通过CDI为相应的生命周期事件指定回调方法。该注释可以应用于实体类，映射超类或回调监听器类的方法如：@PrePersist，@PostPersist，@PreUpdate和@PreRemove。

6. 支持借助于@ConstructorResult 注解能够使用从一个 SQL 查询返回的参数值构造对象

比如下面的例子，可以大概了解ConstructorResult的用法。

```
@NamedNativeQuery(
    name="findAllEmployeeDetails",
    query="SELECT E.EMP_ID, E.F_NAME, E.L_NAME, A.CITY from EMP E, ADDRESS A WHERE E.EMP_ID
    = A.EMP_ID",
    resultSetMapping="employee-details" )
@SqlResultSetMapping(
    name="employee-details",
    classes={
        @ConstructorResult(targetClass=EmployeeDetails.class, columns={
            @ColumnResult(name="EMP_ID", type=Integer.class),
            @ColumnResult(name="F_NAME", type=String.class),
            @ColumnResult(name="L_NAME", type=String.class),
            @ColumnResult(name="CITY", type=String.class) }) } )
public class Employee { ... }
```

3.3 JPA的架构

JPA的架构如下图所示：

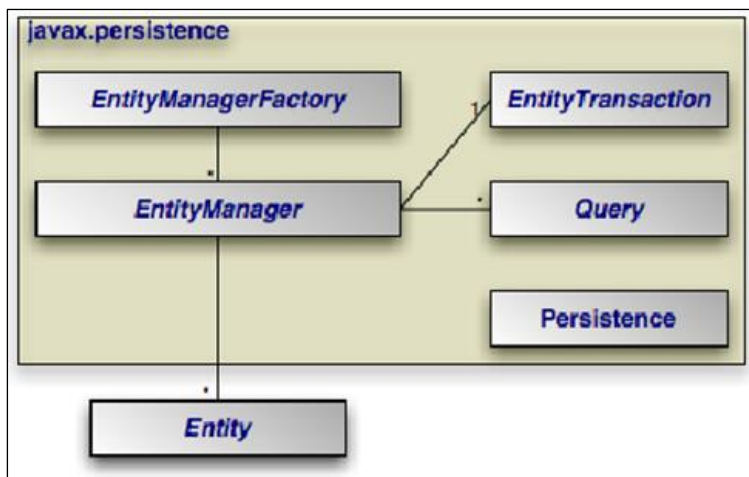


图 3.3-1：JPA架构

在开发的时候需要通过获取EntityManager，根据它提供的API来操作实体、事务、查询等。

- **Persistence**

Persistence 是一个工具类，负责根据配置文件提供的参数创建 EntityManagerFactory 对象。

- **EntityManagerFactory**

javax.persistence.EntityManagerFactory 类是创建 EntityManager 的工厂类。

- **EntityManager**

JPA 应用中使用的基本对象，通过它提供的相应方法可以管理持久化对象，也可以新建或者删除持久化对象。另外 EntityManager 还负责创建 Query 实例。

- **Entity**

Entity 用于封装持久对象。

- **EntityTransaction**

EntityTransaction 用于封装事务，javax.persistence.EntityTransaction 和 EntityManager 之间是一对一的关系。

- **Query**

javax.persistence.Query 接口用于持久对象的查询。它支持 Java Persistence Query Language (JPQL) 和 Structured Query Language (SQL)。

- **PersistenceException**

JPA 异常体系的根是 PersistenceException，它继承于 RuntimeException。OpenJPA 中抛出的异常都实现了 org.apache.openjpa.util.ExceptionInfo 接口，用于提供额外的信息。

根据EntityManager对象的管理方式，可以有以下两种类型。

1. **容器托管的 (container-managed) EntityManager 对象**

容器托管的EntityManager对象最简单，程序员不需要考虑EntityManager连接的释放，以及事务等复杂的问题，所有这些都交给容器去管理。容器托管的EntityManager对象必须在EJB容器中运行，而不能在Web容器和J2SE的环境中运行。

2. **应用托管的 (application-managed) EntityManager 对象**

应用托管的EntityManager对象，程序员需要手动控制它的释放和连接、手动控制事务等。但这种获得应用托管的EntityManager对象的方式，不仅可以在EJB容器中应用，也可以使JPA脱离EJB容器，而与任何的Java环境集成，比如说Web容器、J2SE环境等。所以从某种角度上来说，这种方式是JPA能够独立于EJB环境运行的基础。

说明：

- 理想状态下，最好是选用容器托管的 EntityManager 对象的方式，但在特殊的环境下，还是需要使用应用托管的 EntityManager 对象这种方式。
- 正是因为应用托管的 EntityManager 对象的连接释放、事务控制比较复杂，所以在使用时涉及的相关内容比较多，这些内容将在本章后面部分详细讲述，这里读者应对两种方式有一个大致的了解，两种 EntityManager 对象类型的比较如下表所示。

表 3.3-1: EntityManager对象的管理方式对比

比较内容	容器托管的 (container-managed) EntityManager对象	应用托管的 (application-managed) EntityManager对象
------	---	---

获得方式	两种方式： @PersistenceContext注入；JNDI获得	EntityManagerFactory创建
支持事务	JTA	JTA、RESOURCE_LOCAL
运行环境	EJB容器	EJB容器、Web容器、J2SE环境

3.4 容器托管EntityManager对象

容器托管的EntityManager对象只能运行在EJB容器中，在EJB容器中获得EntityManager对象主要有两种方式，即通过@PersistenceContext注入和JNDI方式获得。

3.4.1 通过@PersistenceContext注入

这种方式获得EntityManager对象最为常用。示例如下：

```
@PersistenceContext(unitName = "jpaUnit")
private EntityManager entityManager;
public List findAllCustomers() {
    Query query = entityManager.createQuery("SELECT c FROM CustomerEO c");
    .....
}
```

说明：

@PersistenceContext标注中unitName为persistence.xml文件中元素中的属性“name”的值，表示要初始化哪个持久化单元。

3.4.2 通过JNDI方式获取

如果指定了@PersistenceContext注释中的name值，则设置了持久化上下文的JNDI名称。通过SessionContext可以创建EntityManager对象。下面的代码为通过JNDI方式获取EntityManager对象。

```
@Stateless
@PersistenceContext(name="jpa")
public class CustomerService implements ICustomerService {
    @ResourceSessionContext ctx;
    public List findAllCustomers() {
        EntityManager entityManager = (EntityManager) ctx.lookup("jpa");
        Query query = entityManager.createQuery("SELECT c FROM CustomerEO c");
        .....
    }
}
```

```
}
```

3.5 应用托管EntityManager对象

应用托管的EntityManager对象，不仅可以在Java EE环境中获得，也可以应用在J2SE的环境中。但无论是在什么情况下获得的EntityManager对象，都是通过实体管理器工厂（EntityManagerFactory）对象创建的。

3.5.1 EJB容器中获得

在EJB容器中，EntityManagerFactory对象可以通过使用注入@PersistenceUnit注释获得。下面代码为在EJB容器中，获得应用托管的EntityManager对象的方法。

```
@Stateless
public class CustomerService implements ICustomerService {
    @PersistenceUnit(unitName="jpaUnit")
    private EntityManagerFactory emf;
    public List findAllCustomers() {
        /**创建EntityManager对象*/
        EntityManager em = emf.createEntityManager();
        Query query = em.createQuery("SELECT c FROM CustomerEO c");
        List result = query.getResultList();
        for (CustomerEO c : result) {
            System.out.println(c.getId()+" "+c.getName());
        }
        /**关闭EntityManager */
        em.close();
        return result;
    }
}
```

说明：

1. 应用托管的 EntityManager 对象，要在代码中手动进行创建和关闭，这是与容器托管的 EntityManager 对象最大的不同。
2. 应用托管的 EntityManager 对象，都是通 EntityManagerFactory 对象来创建的。在容器中可以通过使用注入@PersistenceUnit 标注的方法实现，其中，属性 unitName 为 persistence.xml 文件中元素中的属性“name”的值，表示要初始化哪个持久化单元，与@PersistenceContext 注释中 unitName 属性相同。

3.5.2 Web容器中获得

在Web容器中，EntityManagerFactory对象也可以通过使用注入@PersistenceUnit注释获得。下面代码为在Servlet中，获得应用托管的EntityManager对象的方法。

```
public class TestServlet extends HttpServlet {
    @PersistenceUnit(unitName = "jpaUnit")
    private EntityManagerFactory emf;
    public TestServlet() {super();}
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
        doPost(request, response);
    }
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("");
        out.println("");
        out.println("");
        out.println("");
        if (emf != null) {
            /**创建EntityManager 对象*/
            EntityManager entityManager = emf.createEntityManager();
            try {
                Query query = entityManager.createQuery("SELECT c FROM CustomerEO c");
                List result = query.getResultList();
                for (CustomerEO c : result) {
                    System.out.println(c.getId() + "," + c.getName());
                }
            } finally {
                /**关闭EntityManager*/
                entityManager.close();
            }
        }
        out.println("");
        out.println("");
        out.flush();
        out.close();
    }
}
```

```
}
}
```

3.5.3 J2SE中获得

在J2SE环境中，获得应用托管的EntityManager对象只能通过手动创建的方式，而不能使用注释的方式，与Web容器中不使用注释的方法相同，都是通过Persistence类中createEntityManagerFactory来创建的。下面代码为J2SE环境中获得应用托管EntityManager对象的方法。

```
public class CustomerClient {
    public static void main(String[] args) {
        /** 创建EntityManagerFactory对象 */
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("jpaUnit");
        /** 创建entityManager对象 */
        EntityManager entityManager = emf.createEntityManager();
        Query query = entityManager.createQuery("SELECT c FROM CustomerEO c");
        List result = query.getResultList();
        for (CustomerEO c : result) {
            System.out.println(c.getId() + "," + c.getName());
        }
        /** 关闭entityManager对象 */
        entityManager.close();
        /** 关闭EntityManagerFactory对象 */
        emf.close();
    }
}
```

说明：

在J2SE环境中使用JPA需要将实现的JPA的第三方类包和数据库的驱动包，设置到当前的运行环境下。

3.6 使用EntityManager

1. 创建 Entity

像创建普通java对象一样创建Entity对象，调用EntityManger的persist方法将Entity对象持久化到数据库中。示例代码如下：

```
UserInfo user=new UserInfo("foo@foo.com");
user.setUsername("foo.bar");
user.setUserPassword("123");
em.persist(user);
```

2. 删除 Entity

删除Entity需要调用Entity Manager的remove方法。

3. 更新 Entity

更新Entity过程分两个步骤：第一步先更新Entity属性值，第二步调用Entity Manager的flush方法使得Entity上下文与数据库同步。示例代码如下：

```
UserInfo user=em.find(UserInfo.class,userValue.getUsermail());
if(user!=null){
    user.setUsername("xxx");
    user.setUserpassword("CCCCC");
    em.flush();
}
```

4. 查找 Entity

查找Entity通过javax.persistence.Query来完成。可以通过Entity Manager的API来创建Query对象，然后通过执行Query对象的方法进行查找操作。下面是一个通过用户名查找用户的方法示例。

```
public UserInfo getUserInfo(String username){
    Query query =em.createNamedQuery("UserInfo.findByUsername");
    try{
        UserInfo user=(UserInfo)query.setParameter("username",username).getSingleResult();
        return user;
    }catch(Exception e){
        e.printStackTrace();
        return null;
    }
}
```

3.7 在JavaEE环境中使用JPA

JavaEE环境即包括EJB容器和Web容器的运行环境。在JavaEE环境中使用JPA与在J2SE环境中使用JPA的区别主要在于获取容器与应用托管的EntityManager对象和获取方式的不同。

3.7.1 使用步骤

1. 创建数据源；
2. 定义 Entity，通过标注方式实现该 Java 对象与关系数据库表中的映射；
3. 定义 persistence.xml；

4. 在 JavaEE 应用中获取 EntityManager，并进行持久化等操作。

3.7.2 使用示例

注意：此示例并未使用 TongWeb 自带的 JPA。

1. 服务器配置名称为 MyDataSource 的数据源。
2. 定义名为 User 的 Entity，具体代码如下：

```
package test.jpa;

import java.io.Serializable;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "TWUSER")

public class User implements Serializable {
    private long id;
    private String firstName;
    private String lastName;
    public User(long id, String firstName, String lastName) {
        super();
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
    }
    public User() {
    }
    @Id
    public long getId() {
        return id;
    }
    public void setId(long id) {
        this.id = id;
    }
    public String getFirstName() {
        return firstName;
    }
}
```

```

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}

```

3. 定义 persistence.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
    http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd"
    version="1.0">
<persistence-unit name="user">
<provider>org.apache.openjpa.persistence.PersistenceProviderImpl</provider>
<jta-data-source>MyDataSource</jta-data-source>
<class>test.jpa.User</class>
<properties>
<property name="openjpa.DataCache" value="true (CacheSize=5000000,
SoftReferenceSize=0)"/>
<property name="openjpa.QueryCache" value="true (CacheSize=5000000,
SoftReferenceSize=0)"/>
<property name="openjpa.RuntimeUnenhancedClasses" value="supported"/>
<property name="openjpa.jdbc.SynchronizeMappings"
value="buildSchema (ForeignKeys=true)"/>
</properties>
</persistence-unit>
</persistence>

```

4. 创建 UserService 接口及实现类，完成对 User 实体的持久化及查询等操作。

```

public interface UserService {
    public void add(User user);
    public User find(int id);
    public List<User> findAll();
}

```

```

}

package test.jpa;
import javax.ejb.Stateless;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import java.util.List;
@Stateless
public class UserServiceImpl implements UserService {
    @PersistenceContext(unitName = "user")
    private EntityManager manager;
    public void add(User user) {
        manager.persist(user);
    }
    public User find(int id) {
        return manager.find(User.class, id);
    }
    public List<User> findAll() {
        List<User> userList = null;
        try{
            userList = manager.createQuery("select u from User u").getResultList();
        }catch(Exception e){
            e.printStackTrace();
        }
        return userList;
    }
}

```

5. 创建 Servlet，负责调用 UserService。

```

package TestWeb;
package test.jpa;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;

```

```
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class JpaServlet extends HttpServlet {
    UserService service = null;
    public JpaServlet() {
        super();
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String method = request.getParameter("method");
        if (method.equals("list")) {
            listUsers(request, response);
        }
        if (method.equals("add")) {
            addUser(request, response);
        }
        if (method.equals("get")) {
            getUser(request, response);
        }
    }
    private void getUser(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html");
        int id = Integer.parseInt(request.getParameter("id"));
        PrintWriter out = response.getWriter();
        out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.01 Transitional  
//EN\">");
        out.println("<HTML>");
        out.println("<HEAD><TITLE>Get One Entity</TITLE></HEAD>");
        out.println("<BODY>");
        out.print("Get One Entity </br>");
        try {
            User user = service.find(id);
            out.println("User id:      " + user.getId() + "<br/>");
            out.println("User firstname: " + user.getFirstName() + "<br/>");
            out.println("User lastname:  " + user.getLastName() + "<br/>");
        } catch (Exception e) {
```

```

        out.print("<br><font color='red'>get failed!!</font>");
    }
    out.println("</BODY>");
    out.println("</HTML>");
    out.flush();
    out.close();
}

private void addUser(HttpServletRequest request,
    HttpServletResponse response) throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.01 Transitional
//EN\">");
    out.println("<HTML>");
    out.println("<HEAD><TITLE>Persistence One Entity</TITLE></HEAD>");
    out.println("<BODY>");
    out.print("Persistence One Entity <br>");
    try {
        User user = new User();
        user.setId((int) (Math.random() * 10000000));
        user.setFirstName(getName());
        user.setLastName(getName());
        service.add(user);
        out.print("<br><font color='green'>add success!!</font>");
    } catch (Exception e) {
        out.print("<br><font color='red'>add failed!!</font>");
    }
    out.println("</BODY>");
    out.println("</HTML>");
    out.flush();
    out.close();
}

public void listUsers(HttpServletRequest request,
    HttpServletResponse response) throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.01 Transitional
//EN\">");

```



```

        out.println("<HTML>");
        out.println("<HEAD><TITLE>User List</TITLE></HEAD>");
        out.println("<BODY>");
        out.print("User List <br>");
        try {
            List<User> userlist = service.findAll();
            for (User c : userlist) {
                out.print("ID: <a href=" + request.getContextPath()
                    + "/jpatest?method=get&id=" + c.getId() + ">"
                    + c.getId() + "</a> firstname: " + c.getFirstName()
                    + " lastname: " + c.getLastName() + "</br>");
            }
        } catch (Exception e) {
            out.print("<br><font color='red'>get failed!!</font>");
        }
        out.println("</BODY>");
        out.println("</HTML>");
        out.flush();
        out.close();
    }

    @Override
    public void init() throws ServletException {
        // TODO Auto-generated method stub
        Properties props = new Properties();
        props.put(Context.INITIAL_CONTEXT_FACTORY,
            "com.tongweb.tongejb.core.LocalInitialContextFactory");
        Context ctx;
        try {
            ctx = new InitialContext(props);
            service = (UserService) ctx.lookup("UserServiceImplLocal");
        } catch (Exception e) {
        }
    }

    private String getName() {
        List<Object> list = new ArrayList<Object>();
        for (char c = 'a'; c <= 'z'; c++) {
            list.add(c);
        }
    }

```

```
String str = "";
for (int i = 0; i < 5; i++) {
    int num = (int) (Math.random() * 26);
    str = str + list.get(num);
}
return str;
}
```

6. 将该应用打成 war 包并部署在服务器中，访问应用来查看页面打印信息。

3.8 在J2SE环境中使用JPA

J2SE环境是最普通Java运行环境，例如一个HelloWorld的Java程序就是运行在J2SE的环境中，通常使用main入口方法作为程序启动的触发。在J2SE环境中使用JPA主要是在main函数中使用应用托管EntityManager完成对象持久化等操作。

3.8.1 使用过程

1. 创建数据源；
2. 定义 Entity，通过标注方式实现该 Java 对象与关系数据库表中的映射；
3. 定义 persistence.xml；
4. 在 J2SE 应用中获取 EntityManager，并进行持久化等操作。

3.8.2 使用示例

注意：此示例并未使用 TongWeb 自带的 JPA。

1. 创建数据源，基于 Oracle 的数据源 testdatasource；
2. 定义名为 Student 的 Entity，具体代码如下：

```
package testejb;
import java.io.Serializable;
import javax.persistence.Basic;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.NamedQueries;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import javax.persistence.*;
@Entity
```

```

@Table(name = "student")
@NamedQueries({@NamedQuery(name = "Student.findAll", query = "SELECT s FROM Student s"),
@NamedQuery(name = "Student.findByStuid", query = "SELECT s FROM Student s WHERE s.stuid
= :stuid"), @NamedQuery(name = "Student.findByStuName", query = "SELECT s FROM Student
s WHERE s.stuName = :stuName"), @NamedQuery(name = "Student.findBySex", query = "SELECT
s FROM Student s WHERE s.sex = :sex"), @NamedQuery(name = "Student.findByAge", query =
"SELECT s FROM Student s WHERE s.age = :age")})
public class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @Basic(optional = false)
    @Column(name = "stuid")
    private Integer stuid;
    @Basic(optional = false)
    @Column(name = "stu_name")
    private String stuName;
    @Column(name = "sex")
    private String sex;
    @Column(name = "age")
    private Integer age;
    public Student() {
    }
    public Student(Integer stuid) {
        this.stuid = stuid;
    }
    public Student(Integer stuid, String stuName) {
        this.stuid = stuid;
        this.stuName = stuName;
    }
    public Integer getStuid() {
        return stuid;
    }
    public void setStuid(Integer stuid) {
        this.stuid = stuid;
    }
    public String getStuName() {
        return stuName;
    }
}

```

```

public void setStuName(String stuName) {
    this.stuName = stuName;
}

public String getSex() {
    return sex;
}

public void setSex(String sex) {
    this.sex = sex;
}

public Integer getAge() {
    return age;
}

public void setAge(Integer age) {
    this.age = age;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (stuid != null ? stuid.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    // TODO: Warning - this method won't work in the case the id fields are not set
    if (!(object instanceof Student)) {
        return false;
    }
    Student other = (Student) object;
    if ((this.stuid == null && other.stuid != null) || (this.stuid != null
    && !this.stuid.equals(other.stuid))) {
        return false;
    }
    return true;
}

@Override
public String toString() {
    return "testejb.Student[stuid=" + stuid + "]";
}

```

```
}
```

3. 定义 persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
<persistence-unit name="TestUnit"
transaction-type="RESOURCE_LOCAL"><provider>oracle.toplink.essentials.PersistenceProvid
er</provider>--表示持久性提供者
<jta-data-source>testdatasource</jta-data-source>--表示持久化所需数据源的JNDI
<class>testejb.Student</class>--该Unit包含的实体类
<exclude-unlisted-classes>true</exclude-unlisted-classes>--表示排除未列出的实体类
<properties>
<property name="openjpa.jdbc.SynchronizeMappings"
value="buildSchema(ForeignKeys=true)"/>
</properties>
</persistence-unit>
</persistence>
```

4. 构造 main 函数：

```
public static void main(String[] args) {
/** 创建EntityManagerFactory对象 */
EntityManagerFactory emf = Persistence.createEntityManagerFactory("TestUnit");
/** 创建entityManager对象 */
EntityManager entityManager = emf.createEntityManager();
Student test = new Student(new Integer("111"));
test.setStuName("wang");
test.setAge(new Integer("10"));
test.setSex("man");
entityManager.persist(test);
/** 关闭entityManager对象 */
entityManager.close();
/** 关闭EntityManagerFactory对象 */
emf.close();
}
```

说明：在J2SE环境中使用JPA需要将实现的JPA的第三方类包和数据库的驱动包，设置到当前的运行环

境下。

3.9 EntityManager的事务控制

JPA规范中规定了实体管理器两种事务类型，JTA和RESOURCE_LOCAL。

RESOURCE_LOCAL 事务，既能在JavaEE环境使用，又可以在Java SE环境中使用。代码中使用RESOURCE_LOCAL管理事务时，要通过调用实体管理器EntityManager的getTransaction ()方法来获得本地事务对象，进行事务的管理。

JTA事务，只能在Java EE环境使用，事务是通过JTA方式来控制的，而不是EntityManager自己控制。JTA事务类型的EntityManager将加入到当前事务中，事务的开始和结束都由外部程序来完成。

EntityManager事务类型通过Persistence.xml文件Persistence-Unit中的transaction-type属性来定义。

3.10 JPA常用Annotation

表 3.10-1: JPA常用Annotation说明

Annotation	说明
@Basic	默认情况下，JPA持续性提供程序为大多数Java基元类型、基元类型的包装程序以及枚举自动配置一个@Basic映射。
@Embeddable	默认情况下，JPA持续性提供程序假设每个实体均持久保存到它自己的数据库表。使用@Embeddable批注指定一个类，该类的实例存储为拥有实体的固有部分并共享该实体的身份。嵌入对象的每个持久属性或字段都将映射到实体的数据库表。
@Embedded	使用@Embedded批注指定一个持久字段，该字段的@Embeddable 类型可以存储为拥有实体的固有部分，并共享该实体的身份。嵌入对象的每个持久属性或字段均映射到拥有实体的数据库表。
@EntityResult	使用 @EntityResult 批注返回实体。
@FieldResult	默认情况下，JPA持续性提供程序假设在使用@EntityResult返回实体时，SELECT语句将包含与返回的实体的所有字段或属性相对应的所有列，且SELECT语句中的列名对应于字段或属性名（未使用AS语句）。 如果SELECT语句只包含某些与返回的实体的字段或属性相对应的列，或SELECT语句中的列名并不对应于字段或属性名（使用了AS语句），则在使用@EntityResult返回实体时，请使用@FieldResult批注将 SELECT语句中的列映射到字段或属性。
@PersistenceContext	在使用JPA持续性提供程序的应用程序中，可以使用实体管理器执行所有持续性操作（创建、读取、更新和删除）。Java EE应用程序使用相关性注入或在JNDI名称空间中直接查找实体管理器获取实体管理器。

Annotation	说明
@PersistenceContexts	如果要指定多个@PersistenceContext,则必须使用一个 @PersistenceContexts批注指定所有持续性上下文。
@PersistenceUnit	使用@PersistenceUnit批注指定EntityManagerFactory, 您希望 JPA 持续性提供程序使用它来: ● 获取指定的实体管理器 ● 指定 EntityManagerFactory 和持续性单元
@Entity	指明这是一个实体Bean, 每个实体Bean类映射数据库中的一个表
@Id	注释指定personid属性为表的主键
@OneToMany	声明一对多关联
@ManyToMany	声明多对多关联
@OneToOne	声明一对一关联
@ManyToOne	声明多对一关联
@JoinColumn	默认情况下, 在实体关联中JPA持续性提供程序使用一个基于现有名称(如字段或属性名称)的数据库模式, 以便它可以自动确定要使用的单个连接列(包含外键的列)。
@JoinTable	JPA持续性提供程序在映射多对多关联(或在单向的一对多关联中)的拥有方上的实体关联时使用一个连接表。连接表名称及其列名均在默认情况下指定, 且JPA持续性提供程序假设: 在关系的拥有方上的实体主表中, 每个主键列有一个连接列。
@Table	注释的name属性指定映射的数据表名称
@Column	注释定义了映射到列的所有属性, 如列名是否唯一, 是否允许为空, 是否允许更新等。
@ColumnResult	执行@NamedNativeQuery时, 它可以返回实体(包括不同类型的实体)、标量值或实体和标量值的组合。
@GeneratedValue	指明一种策略来为ID字段分配一个唯一的值。可选的策略有 IDENTITY, SEQUENCE, TABLE和 AUTO。
@Version	指明实体中的版本字段。JPA使用版本字段来检测对于数据的并发修改。 当JPA检测到有多个操作同时修改一个数据, 它将向最后提交的事务抛出一个异常。这将保护你先提交的事务能够稳定地提交数据。

Annotation	说明
@PrePersist	声明JPA中的回调函数： 在执行给定实体的相应EntityManager持久操作之前，调用该实体的@PrePersist回调方法。
@PostPersist	在实体成为持久实体后，调用该实体的@PostPersist回调方法。对该操作层叠到的所有实体调用该方法。在数据库插入操作之后调用该方法。这些数据库操作可能在调用了持久操作之后立即发生，也可能在刷新操作（可能在事务结束时发生）发生之后立即发生。
@PostLoad	将实体加载到数据库的当前持续性上下文中后或在向其应用了刷新操作后，调用实体的@PostLoad方法。在返回或访问查询结果之前或在遍历关联之前调用该方法。
@PreUpdate	在对实体数据进行数据库更新操作之前，调用实体的@PreUpdate回调方法。这些数据库操作可以在更新实体状态时发生，也可以在将状态刷新到数据库（可能位于事务结尾）时发生。
@PostUpdate	在对实体数据进行了数据库更新操作后，调用实体的@PostUpdate回调方法。这些数据库操作可以在更新实体状态时发生，也可以在将状态刷新到数据库（位于事务结尾）时发生。
@PreRemove	在针对给定实体执行相应的EntityManager删除操作之前，调用该给定实体的@PreRemove回调方法。对该操作层叠到的所有实体调用该方法。
@PostRemove	在删除实体后，调用该实体的@PostRemove回调方法。对该操作层叠到的所有实体调用该方法。在数据库删除操作之后调用该方法。这些数据库操作可能在调用了删除操作之后立即发生，也可能在刷新操作（可能在事务结束时发生）发生之后立即发生。
@EntityListeners	声明JPA监听器
@NamedQuery	在JPA中映射EJBQL/HQL查询

3.11 persistence.xml及JPA功能属性

persistence.xml定义了一个持久化单元(Persistence Unit)。用户可以通过persistence.xml指定被管理的Entity类，以及设定一些与持久化单元、EntityManager相关的属性。

persistence.xml的根元素为persistence元素，persistence元素可以包含一个或者多个persistence-unit元素。persistence-unit元素包含name和transaction-type属性以及如下元素：

description、provider、jta-data-source、non-jta-data-source、mapping-file、jar-file、class、exclude-unlisted-classed和properties。

name 属性是必须的，用于标识Persistence Unit。该属性将在标注PersistenceContext 和

PersistenceUnit中被引用。

transaction-type属性用于设置该Persistence Unit的事务类型。transaction-type属性可以设置为JTA或者RESOURCE_LOCAL两种。在JavaEE环境中，默认为JTA事务类型，在JavaSE环境中默认为RESOURCE_LOCAL事务类型。

关于persistence.xml中其他元素以及schema的详细说明参见：

http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd。

3.11.1 JPA常用功能属性

对于JPA提供的特色功能，都可以通过在persistence.xml的**properties**中进行配置，如：

```
<properties>
<property name="openjpa.jdbc.SynchronizeMappings"
value="buildSchema(ForeignKeys=true)"/>
</properties>
```

下表为JPA常用的功能属性：

表 3.11-1：JPA常用功能属性

属性	功能描述	参数及描述
openjpa.AutoClear	自动清理实例状态的时机：在插入一个datastore事物时，或者插入任何事物时	默认datastore
openjpa.Callbacks	对规范回调函数扩展	有两个布尔类型参数： PostPersistCallbackImmediate：当一个新的实例受管的时候，post-persist回调函数立即被调用。默认是false，当commit或者flush到数据库的时候调用。 <pre><property name="openjpa.Callbacks" value="PostPersistCallbackImmediate=true"/></pre> AllowsMultipleMethodsForSameCallback：同一个类的多个方法能够处理同一个回调事件。默认是false
openjpa.Compatibility	此属性设置是为了兼容openjpa以前版本的功能，用“，”分割： <pre><property name="openjpa.Compatibility" value=</pre>	CopyOnDetach=true FlushBeforeDetach=true CascadeWithDetach=true autoOff=true

属性	功能描述	参数及描述
	<code>"QuotedNumbersInQueries=true, CopyOnDetach=true"/></code>	<code>PrivatePersistentProperties=true IgnoreDetachedStateFieldForProxySerial ization=true QuotedNumbersInQueries=true</code>
open.jpa.ConnectionDriverName	数据连接的驱动类的全称，不同的数据库实现类不同。 示例如下： <code><property name= "open.jpa.ConnectionDriverName" value="com.mysql.jdbc.Driver"/></code>	-
open.jpa.ConnectionFactory	通过数据源对象指定数据源，通常用在应用实现中 <code>propMap.put("open.jpa.ConnectionFactory", _ds);</code>	-
open.jpa.ConnectionFactory2	同open.jpa.ConnectionFactory，用于非JTA数据源	-
open.jpa.ConnectionFactoryName	在property中通过JNDI位置指定数据源： <code><property name="open.jpa.ConnectionFactoryName" value="java:comp/env/jdbc/MyDataSource"/></code>	与<jta-data-source>对应
open.jpa.ConnectionFactory2Name	同ConnectionFactoryName，用于非JTA数据源。	与<non-jta-data-source>对应
open.jpa.ConnectionFactoryMode	Connection交给本地还是应用服务器管理： <code><property name="open.jpa.ConnectionFactoryMode" value="managed" /></code>	local：交给本地管理。 managed：交给应用服务器管理如配置一个jta数据源。
open.jpa.ConnectionFactoryProperties	配置数据库连接工厂时的属性。示例如下： <code><property name="open.jpa.ConnectionFactoryProperties" value="PrettyPrint=true, PrettyPrintLineLength=80"/></code>	可选值： QueryTimeout: JDBC驱动执行查询超时的时间，以秒为单位。 PrettyPrint: 是否格式化输出SQL语句。 PrettyPrintLineLength: SQL每行输出的最大长度。 PrintParameters: 为了安全open.jpa默认是不打印sql中的参数内容的，当该属性设置为true的时候则可以打印。

属性	功能描述	参数及描述
openjpa.ConnectionPassword	数据连接的密码： <code><property name="openjpa.ConnectionPassword" value="123"/></code>	-
openjpa.ConnectionProperties	数据库连接的属性，相当于JDBC连接数据库时Driver.connect (String url, Properties props)方法中的属性设置，如： <code><property name="openjpa.ConnectionProperties" value="Username=root, Password=root"/></code>	-
openjpa.ConnectionURL	数据连接的URL示例如下： <code><property name="openjpa.ConnectionURL" value="jdbc:mysql://localhost:3306/jpademo"/></code>	-
openjpa.ConnectionUserName	数据连接的用户名： <code><property name="openjpa.ConnectionUserName" value="root"/></code>	-
openjpa.DataCache	DataCache用来缓存实体。例如： <code><property name="openjpa.DataCache" value="true (CacheSize=5000,SoftReferenceSize=0)"/></code> <code><property name="openjpa.RemoteCommitProvider" value="sjvm"/></code>	CacheSize: 缓存实体的最大数目，默认值1000, 如果缓存实体超出这一数值时，OpenJPA 会随机逐出一些实体，直至实体数目小于设定的最大数目。 SoftReferenceSize: OpenJPA的缓存如果达到最大存储数目，或根据一定的算法需要从缓存中移出一下数据时，会把它放到一个 softReferenceMap中，softReferenceMap 中的数据也会删除，只是这些数据比直接删除会待的时间久一些，以防止在缓存删除以后程序马上就访问它们。这样下次访问到这些移出的数据时就会到 softReferenceMap 中去找，比直接删除和重新访问数据库查找的效率好一点。这跟其他的缓存实现有点不一样，其它的缓存是直接从缓存中删除这些数据。 如果不想使用 softReferenceMap，可设成0。

属性	功能描述	参数及描述
		EvictionSchedule: 通过设置这个属性在某个特定的时间清除cache。比如 (EvictionSchedule='15,45 15 * * 1')表示在星期天的下午三点15分到45分之间清除cache。
open.jpa.DataCacheMode	设置实体的缓存方式	ALL: 缓存所有实体。 NONE: 禁止缓存所有实体。 ENABLE_SELECTIVE: 缓存所有被缓存注解标识过的实体, 未标识的实体不被缓存。 DISABLE_SELECTIVE: 除了被标注不被缓存的实体, 都被缓存。
open.jpa.DataCacheTimeout	缓存的失效时间, 设置毫秒数代表在此时间内缓存有效。当该属性设置为-1 (默认) 时, 代表缓存不会过期。 <property name="open.jpa.DataCacheTimeout" value="10000" />	默认值-1
open.jpa.FetchBatchSize	查询结果一次转化为对象的最多个数, 相当于JDBC的结果集对象Statement.setFetchSize。默认为-1, 表示所有的查询对象立即初始化; 0表示使用JDBC驱动默认的数值。 示例如下: <property name="open.jpa.FetchBatchSize" value="20"/>	-
open.jpa.FlushBeforeQueries	当前事务执行查询之前是否进行flush操作。 此设置只适用于查询, 否则必须在内存中执行。	true始终是flush操作, 而不是在内存中执行 false查询前不进行flush操作
open.jpa.IgnoreChanges	如果设置为true, open.jpa将忽视持久化对象的更改直接执行数据库查询操作。设置为false, open.jpa会考虑当前事务中的变化是否影响查询。如果是的话则在内存中进行查询或者在运行查询之前将当前事务flush到数据库中。	-
open.jpa.Id	标识一个持久化单元id, 在其他功能下可以被引用, 比如日志的一个参数DiagnosticContext。通常和持久化单元的name一致。	-
open.jpa.Instrumentation	配合java仪表化的工具, 例如配置了该属性可以	-

属性	功能描述	参数及描述
on	将要观察的对象注册成mbean，在jconsole中进行观察mbean中的属性。例如： <pre><property name="openjpa.Instrumentation" value="jmx (Instrument=' DataCache, QueryCache')"/> <property name="openjpa.DataCache" value="true (EnableStatistics=true)"/> <property name="openjpa.QueryCache" value="true (EnableStatistics=true)"/></pre>	
openjpa.LockTimeout	在尝试获取lock时，可以通过openjpa.LockTimeout配置最长的等待时间（缺省值-1指定没有限制），超过这个时间后OpenJPA会抛出异常。	-
openjpa.Log	1. 配置日志属性，对应配置的API为org.apache.openjpa.conf.OpenJPAConfiguration.getLog 2. 格式：<property name="openjpa.Log" value="XXX"/>	openjpa.Log的值可以由以下4部分组成，配置的每个部分用逗号分隔。 File：可以指定日志保存的文件 DefaultLevel：日志默认的级别 DiagnosticContext：在所输出的日志前增加诊断信息 <channel>：日志类别 如：更详细的配置请见jpa预研文档
openjpa.QueryCache	QueryCache 主要用来缓存查询语句返回的结果： <pre><property name="openjpa.DataCache" value="true (CacheSize=5000, SoftReferenceSize=0)"/> <property name="openjpa.QueryCache" value="CacheSize=1000, SoftReferenceSize=100"/></pre>	true（默认）：启用查询结果缓存，同时openjpa.DataCache设置为true。 false：禁用启用查询结果缓存。 示例：禁用查询结果缓存 <pre><property name="openjpa.QueryCache" value=" false"/></pre> 关于缓存查询结果还有许多参数功能配置，例如： <pre><property name="openjpa.DataCache" value="true (ExcludedTypes=foo.bar .Person; foo.bar.Employee)"/></pre>
openjpa.QueryCompilat	缓存的数据存储类型：	true（默认）：

属性	功能描述	参数及描述
ionCache	<code><property name="openjpa.QueryCompilationCache" value="true" /></code>	将使用 org.apache.openjpa.util.CacheMap 缓存数据。CacheMap缓存固定容量和一些软引用的数据。 all: 将使用 org.apache.openjpa.lib.util.ConcurrentHashMap缓存数据。缓存效率高, 但数据永远不会自动清空, 如果频繁地动态查询, 将会占用大量的内存。但如果只是涉及参数查询, 则可以选中该方式。 false: 禁用缓存。
openjpa.ReadLockLevel	OpenJPA通过openjpa.ReadLockLevel和openjpa.WriteLockLevel来配置缺省的事务读写lock level。这些缺省配置只适用于非乐观事务: <code><property name="openjpa.ReadLockLevel" value="none" /></code> <code><property name="openjpa.WriteLockLevel" value="write" /></code> <code><property name="openjpa.LockTimeout" value="30000" /></code>	none, read, write, optimistic, optimistic-force-increment, pessimistic-read, pessimistic-write, pessimistic-force-increment
openjpa.TransactionMode	Openjpa用这个属性来配置事务管理采取哪种方式	local: 在本地执行事务操作。 managed: 集成到应用程序服务器的管理的全局事务 (JTA)。
openjpa.Specification	与openjpa.Compatibility类似, 都是用于兼容性的。如果应用需要用到的都是JPA1.0版本, 最简单的做法就是设置openjpa.Specification="JPA1.0", 指定版本。格式是name [major. [minor]] 其中name是规范名称如JPA或者JDO。 当指定了版本后在即可遵照指定规范执行各种操作。而如果想用JPA2.0 但是想要细粒度的指定哪些用1.0版本的功能则需要在openjpa.Compatibility属性中指定相应信息。	-
openjpa.WriteLockLevel	OpenJPA通过openjpa.ReadLockLevel和openjpa.WriteLockLevel来配置缺省的事务读写lock level。这些缺省配置只适用于非乐观事务。	none, read, write (默认), optimistic, optimistic-force-increment,

属性	功能描述	参数及描述
	<code><property name="openjpa.ReadLockLevel" value="none"/></code> <code><property name="openjpa.WriteLockLevel" value="write"/></code> <code><property name="openjpa.LockTimeout" value="30000"/></code>	pessimistic-read, pessimistic-write, pessimistic-force-increment

3.11.2 Openjpa JDBC属性

下表为openjpa JDBC属性说明。

表 3.11-2: openjpa JDBC属性说明

属性	功能描述	参数及描述
openjpa.jdbc.DBDictionary	设置数据库字典 (Dictionary)，主流情况下，OpenJpa根据jdbc url自动配置相应的数据库字典。但是当你选了一个非主流的驱动时，就需要指定这一配置，你可以自定义数据库字典，或者设置字典属性。如： <code><property name="openjpa.jdbc.DBDictionary" value="hsql (SimulateLocking=true)"/></code>	access: Microsoft Access字典 HSQL数据库字典
openjpa.jdbc.EagerFetchMode	抓取数据模式，优化openjpa负载持久化关系，可以运行时改变。默认: parallel	可选项: parallel (默认), join, none
openjpa.jdbc.FetchDirection	期望的访问查询结果时的顺序	可选值: forward (默认): 向前型 reverse: 倒转型 unknown: 未知型 示例如下: <code><property name="openjpa.jdbc.FetchDirection" value="reverse"/></code>
openjpa.jdbc.JDBCListeners	对jdbc的事件做监听，可以设置自定义的listener。配置如下: <code><property name="openjpa.jdbc.JDBCListeners" value="test.jpa.TestListener"/></code>	-

openjpa.jdbc.QuerySQLCache	openjpa每次执行一次数据库操作时先将jpql转储成sql statements，若开启了该属性，通过缓存的jpql语句作为key并缓存sql statements，每次执行jpql通过判断找到缓存中的转化过的语句。对于性能有一定的提升。因为每次运行同一条sql语句而不需要每次都去解析转换一次。 例如：<property name="openjpa.jdbc.QuerySQLCache" value="true(EnableStatistics=true)"/>	默认是false，设置为true时候即开启。同时还支持单独对某几个sql进行缓存： <pre><property name="openjpa.jdbc.QuerySQLCache" value="true(excludes='select c from Company c; select d from Department d')"/></pre>
openjpa.jdbc.ResultSetType	执行查询时返回的查询结果集类型，相当于设置java.sql.ResultSet中的返回类型。	可选值： forward-only（默认）：仅向前型 scroll-sensitive：滚动敏感型 scroll-insensitive：滚动不敏感型 例如： <pre><property name="openjpa.jdbc.ResultSetType" value="scroll-insensitive"/></pre>
openjpa.jdbc.Schema	在一个数据库中可以有多个应用的数据表，这些不同应用的表可以放在不同的schema之中。同时，每一个schema对应一个用户，不同的应用可以以不同的用户连接数据库。如test.user在openjpa中可以指定jdbc的schema。 <pre><property name="openjpa.jdbc.Schema" value="test" /></pre>	-
openjpa.jdbc.SubclassFetchMode	如何抓取其他表的子类数据，配置与openjpa.jdbc.EagerFetchMode类似。 通常是： <pre><property name="openjpa.jdbc.EagerFetchMode" value="parallel"/> <property name="openjpa.jdbc.SubclassFetchMode" value="join"/></pre>	可选项：parallel（默认），join，none
openjpa.jdbc.SynchronizeMappings	使用该属性，在运行时数据库会根据创建的实体关系自动建表。示例如下：<property name="openjpa.jdbc.SynchronizeMappings" value="buildSchema(ForeignKeys=true)"/>	-

openjpa.jdbc.TransactionIsolation	设置事务隔离级别。默认采用JDBC Driver的事务隔离级别。	可选项： none：无事务隔离 read-committed：这种隔离级别指的是，事务只能读取已经提交的数据。 read-uncommitted：这种隔离级别允许脏读（也就是可以读取到用户未提交的数据）。 repeatable-read：这种隔离级别不支持脏读，不支持可重复读，支持幻想读。主要是为了得到一致性的答案与防止丢失更新。 serializable：不允许脏读，重复读与幻想读，最高的隔离级别。这种隔离级别标明事务A在操作数据库的时候好像就只有事务A在操作，没有其他事务在操作数据库一样。
hibernate.transaction.jta.platform	TongWeb对hibernate的JPA事务支持	例如：<property name="hibernate.transaction.jta.platform" value="com.tongweb.tongejb.hibernate.OpenEJBtaPlatform" />

第4章 Dependency Inject

DI是指Dependency Inject依赖注入，它由JSR-330（Dependency Injection for Java）定义，所在包为javax.inject。常用标记包括@Inject、@Qualifier、Provider、@Named、@Scope。

4.1 @inject

@Inject标识了可注入的构造器、方法或属性。它取代了传统的new一个对象实例或者从工厂方法获取对象实例的方法，通过标记@Inject来获取对象实例。可用于静态或者实例成员。

注入顺序为构造器，字段，最后是方法。超类的字段、方法将优先于子类的字段、方法被注入。对于同一个类的字段和方法是不区分注入顺序的。例如：

```
public class Car {  
    //可注入的构造器。标记后将查找实参对构造器进行注入。  
    //构造器参数可以为空  
    @Inject public Car(Engine engine) { ... }  
    // 可注入的属性。标记后会查找Provider<Seat>并注入。  
    //非final的，属性名有效  
    @Inject private Provider<Seat> seatProvider;  
    // 可注入的方法。会查找实参并对方法注入。  
    //非abstract的，可无实参或形参，可有返回值，方法名有效  
    @Inject void install(Windshield windshield, Trunk trunk) { ... }  
}
```

当一个方法标注了@Inject并覆写了其他标注了@Inject的方法时，对于每一个实例的每一次注入请求，该方法只会被注入一次。当一个方法没有标注@Inject并覆写了其他标注了@Inject的方法时，该方法不会被注入。

要进行成员注入就必须标注@Inject。一个可注入的成员可以使用任何访问修饰符（包括 private）。不过受于平台或注入器限制（例如安全管理或缺乏反射支持），标注了@Inject的非公有成员可能将不被注入。

4.2 @Qualifier

@Qualifier用于标识限定器注解。任何人都可以定义新的限定器注解，限定器就像是用来区分不同内容的修饰语。

限定器是被@Qualifier、@Retention(RUNTIME)标注的，通常也被@Documented标注。可以拥有属性。如果标注了@Target可能会有一些用法限制。本规范只是指定了限定器注解可以被使用在字段和参数上，但一些注入器配置可能使用限定器注解在其他一些地方（例如方法或类）上。例如：

```
@java.lang.annotation.Documented  
@java.lang.annotation.Retention(RUNTIME)
```

```
@javax.inject.Qualifier
public @interface Leather {
    Color color() default Color.TAN;
    public enum Color { RED, BLACK, TAN }
}

@java.lang.annotation.Documented
@java.lang.annotation.Retention(RUNTIME)
@javax.inject.Qualifier
public @interface Plastic {
    Color color() default Color.White;
    public enum Color { White, BLACK, Yellow }
}
```

有了上述定义，就可以用@Leather或者@Plastic来分区某属性了。比如下面用@Leather标记表明汽车座椅Seat是皮毛的不是塑料的。

```
public class Car {
    @Inject private @Leather Provider<Seat> seatProvider;
}
```

4.3 Provider

Provider为接口类型，使用场景是当某些类型比如type T可以注入时，通常Provider<T>也就是可注入的。且Provider可以根据需要来提供不同实现。

例如：可以根据不用的需要来提供Seat的不同实现，是给司机的座位还是给乘客的座位。

```
class Car {
    @Inject Car(Provider<Seat> seatProvider) {
        Seat driver = seatProvider.get();
        Seat passenger = seatProvider.get();
        ...
    }
}
```

可以这么定义Provider，关键在于get的实现。要能提供多种type T的实现。

```
public class SeatProvider implements Provider<Seat> {
    private Seat seat;
    public SeatProvider () {
        this.seatProvider = new SeatProvider Impl();
    }
    public SeatProvider get() {
        return seatProvider;
    }
}
```

```
}  
}
```

4.4 @Named

@Name是@Qualifier的一种，因为@Qualifier只能使用一次，所以可以通过名字来对不同情况进行标识。比如当同为皮毛座位，需要对司机座位和乘客座位进行标识。

```
public class Car {  
    @Inject @Named("driver") Seat driverSeat;  
    @Inject @Named("passenger") Seat passengerSeat;  
    ...  
}
```

4.5 @Scope

@Scope用于标识作用域。缺省情况下，如果没有标识作用域，注入器将为每一次注入都创建（通过注入类型的构造器）新实例，所以@Scope控制某类型的实例如何被注入器重用。使用@Scope来标识一个作用域有助于注入器探测程序员使用了作用域注解但却忘了去配置作用域的情况。一个保守的注入器应该生成一个错误而不是去适用该作用域。这部分在CDI中会有各种Scope的介绍。

@Singleton是@Scope中的一种，作用域@Singleton确保始终只有一个标注的对象实例。当多于一个作用域注解或不被注入器支持的作用域注解被使用在同一个类上时，注入器将生成一个错误。

第5章 CDI

CDI是在JAVAE6被引入的新规范，指Contexts Dependency Inject环境依赖注入，可提供一个与j2ee环境相结合的DI的环境，实现了注入并集成生命周期管理。由JSR-299 (Contexts and Dependency Injection for the JavaTM EE platform) 定义的，所在包为javax.enterprise.context, javax.enterprise.inject, javax.enterprise.event, javax.enterprise.util, javax.decorator等。

CDI总的来说有以下几个方面：建立的J2EE领域的“DI”；负责针对于注入到CDI容器中Bean的生命周期的管理；负责bean与上下文之间的联系；基于j2ee的限定符区分；bean范围管理；针对于bean的AOP的拦截功能的interceptorBinding，是与interceptor规范进行配合的；装饰模式的decorator的运用，给对应的method增加新的功能；bean的事件定义，事件监听和发送等。

JAVAE7中，CDI规范升级到了Contexts and Dependency Injection (JSR-346)。

5.1 CDI新特性

相对于CDI1.0规范，CDI 1.1在一些功能进行了改进，改进点如下：

1. 加入全局拦截器、全局装饰器和使用@Priority 注解实现替代选择。
2. 给注解为@AroundConstruct 的构造器加入生命周期回调支持。
3. 允许绑定拦截构造器。
4. 拦截器绑定移入拦截器规范，以供其它规范使用。
5. Beans 内建支持装饰器加入 EventMetadata 允许事件元数据检查。
6. 加入@Vetoed 注解让 Classes 失去原有程序能力。
7. 强化beans 功能的改进,包括 @TransientReference 注解允许保留实力给调用它的方法或构造函数。
8. 加入激活和销毁周期的事件回调。
9. 加入 AlterableContext 允许 bean 实例被明确的销毁。
10. 给 bean 在周期注解、EJB、在 JavaEE 中自启动 CDI 功能。
11. 在 bean.xml 中加入 class 过滤排除，以防止扫描 classes 和 package。
12. 加入 Unmanaged 允许方便的访问无上下文的 bean 实例。
13. 加入 AfterTypeDiscovery 事件，允许在类型发现之后扩展附加注册。
14. 加入@WithAnnotation 注解作为一种改善负载性能的方式。

下面是比较主要的新特性示例。

1. 排除一些类不能为 bean

新规范有两种方式，一种是通过配置bean.xml实现。例如：

```
<beans xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://xmlns.j
cp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/beans_1_1.xsd"
bean-discovery-mode="all">
```

```

<scan>
<exclude name="org.javaee7.cdi.exclude.filter.beans.*">
</exclude>
</scan>
</beans>

```

粗体部分未过滤的包名，也可以指向一个具体的类名。

另外一种方式是使用@Vetoed注解在具体的类上注释，自动屏蔽一个bean被CDI管理。如：

```

@Vetoed
public class SimpleGreeting implements Greeting {
    @Override
    public String greet(String name) {
        return "Hello " + name;
    }
}

```

详细的用例请见：TongWeb/ samples/cdi/cdi11Sample中的Vetoed相关用例。

2. @Priority

设定优先级，下面以经常使用的拦截器做介绍。

```
@Priority(Interceptor.Priority.APPLICATION+10)
```

该注解后面加的数字越小代表优先级越高。下面的例子中HighPriorityInterceptor拦截器注解的优先级数字是100。

```

@Interceptor
@MyInterceptorBinding
@Priority(Interceptor.Priority.APPLICATION + 100)
public class HighPriorityInterceptor {
    @AroundInvoke
    public Object log(InvocationContext context) throws Exception {
        Object[] parameters = context.getParameters();
        if (parameters.length > 0 && parameters[0] instanceof String) {
            String param = (String) parameters[0];
            parameters[0] = "Hi " + param + " !";
            context.setParameters(parameters);
        }
        return context.proceed();
    }
}

```

而低优先级的优先级别数字为200，这样在拦截器工作时会先调用HighPriorityInterceptor，后调用

LowPriorityInterceptor里的实现。

```
@Interceptor
@MyInterceptorBinding
@Priority(Interceptor.Priority.APPLICATION + 200)
public class LowPriorityInterceptor {
    @AroundInvoke
    public Object log(InvocationContext context) throws Exception {
        Object[] parameters = context.getParameters();
        if (parameters.length > 0 && parameters[0] instanceof String) {
            String param = (String) parameters[0];
            parameters[0] = param + " Nice to meet you.";
            context.setParameters(parameters);
        }
        return context.proceed();
    }
}
```

最后应用输出的结果就是按照拦截器的顺序组织，例如输出结果为：Hi Arun ! Nice to meet you.
详细的用例请见：TongWeb/ samples/cdi/cdi11Sample中的PriorityInterceptor相关用例。

3. Unmanaged

加入 Unmanaged 允许方便的访问无上下文的 bean 实例，这样的话不需要 CDI 注入，也可以拿到 bean 实例。

```
Unmanaged<NonCDI> unmanaged = new Unmanaged(NonCDI.class);
UnmanagedInstance<NonCDI>inst = unmanaged.newInstance();
inst.produce().inject();
inst.postConstruct();
inst.preDestroy();
NonCDI nonCdi = inst.get();
```

详细的用例请见：TongWeb/ samples/cdi/cdi11Sample中的Unmanaged相关用例。

5.2 CDI的使用

本章节介绍CDI常用标记的使用。

5.2.1 @Qualifier的补充

CDI中对限定符@Qualifier做了几个补充，分别是@Default、@Any、@New。

如果一个Bean没有用@Named等来限定，那就是用默认的@Default来限定了。下面两个例子是等价的。

示例1:

```
@ConversationScoped
public class Order {
    private Product product;
    private User customer;
    @Inject
    public void init(@Selected Product product, User customer) {
        this.product = product;
        this.customer = customer;
    }
}
```

示例2:

```
@Default @ConversationScoped
public class Order {
    private Product product;
    private User customer;
    @Inject
    public void init(@Selected Product product, @Default User customer) {
        this.product = product;
        this.customer = customer;
    }
}
```

@Any和@New都是@Qualifier内置的标记。@New限定符允许应用程序获得一个新的bean的实例没有绑定到声明的Scope范围，但也执行依赖注入。而@Any就是任何bean都默认带着的标记，除了被@New标记的。

5.2.2 @Scope

@Scope承接DI中的Scope在CDI中预定义了一些，常用的包括@RequestScoped、@SessionScoped、@ApplicationScoped、@ConversationScope、@Dependent、@NormalScope。

@RequestScoped、@SessionScoped、@ApplicationScoped都是基于HttpSession的范围。除此之外，@RequestScoped和@ApplicationScoped在下面各种情况下也会处于活动状态：调用EJB远程方法时，调用EJB的异步方法时，EJB超时的时候，当有消息送达message-driven bean时当有消息送达MessageListener时。

另外，CDI还给web应用增加了@ConversationScope这一范围，任何JSF请求都能访问活动的conversation scope。要在其他的web框架中支持conversation scope，需要创建一个CDI extension.conversation context在任何JSF请求中都处于活动状态。conversation默认为transient状态的，在请求结束的时候终止。如果希望conversation能够继续，必须将其提升为long-running conversation。

要将与当前request关联的conversation提升为long-running conversation，在代码中调用begin()方法，要在当前的request结束时将当前的long-running conversation销毁，调用end()方法。例如：


```

@ConversationScoped @Stateful
public class OrderBuilder {
    private Order order;
    private @Inject Conversation conversation;
    private @PersistenceContext(type = EXTENDED) EntityManager em;
    @Produces public Order getOrder() {
        return order;
    }
    public Order createOrder() {
        order = new Order();
        conversation.begin();
        return order;
    }
    public void addLineItem(Product product, int quantity) {
        order.add(new LineItem(product, quantity));
    }
    public void saveOrder(Order order) {
        em.persist(order);
        conversation.end();
    }
    @Remove
    public void destroy() {}
}

```

@Depnedent范围的 bean的实例不会由不同的使用者或注入点共享。它依赖于其他对象，随其所属对象的创建而创建，随其所属对象的销毁而销毁，这个也是CDI最常见的一种范围。如果在EL表达式中引用，则每次计算这个表达式时都会创建这个bean的实例。这个实例不会在计算任何其他表达式时重用。

如果需要在JSF页面中通过EL名称直接访问bean，通常不能将其标记为@Dependent。否则任何通过JSF输入而赋给它的值会立即丢失。因此CDI提供了@Model，可以给bean一个名称，并且相当于同时标记为@RequestScoped。如果你确实需要在JSF页面中访问一个@Dependent的bean，把它注入到另外一个bean中，然后通过getter方法提供给EL。

标记为@Dependent的Beans不需要代理对象，其使用者可以直接引用它。

@NormalScope，这个注释定义目前的范围是否可以钝化。也就是说，这个原注释一般都是为了扩展而用的，以下是最典型的SessionScoped的定义：

```

@Target (value={TYPE, METHOD, FIELD})
@Retention (value=RUNTIME)
@Documented
@NormalScope (passivating=true)
@Inherited
public @interface SessionScoped

```

@New在Qualifier中已经描述过了，它会注入没有绑定到声明的Scope范围的bean。所以在引用超出了这个预定范围的场景下，要解决这个对象的引用为null的问题，可以给超出范围的引用，用@New来设置一个默认值。

5.2.3 @Produces与@Dispose

生产者@Produces在某些CDI容器无法inject的情况下可以自定义这个依赖对象的生产过程，或者是将生产过程批量化。@Produces标记的方法是非抽象的，@Produces可带多个参数且每个参数都是可注入的。@Dispose会在Scope范围结束时自动调用，如果要显式调用，需要和@Produces有一致的参数。例如：

```
@Produces @Preferred @SessionScoped
public PaymentStrategy getPaymentStrategy(@New CreditCardPaymentStrategy ccps,
@New CheckPaymentStrategy cps, @New PayPalPaymentStrategy ppps) {
    switch (paymentStrategy) {
        case CREDIT_CARD: return ccps;
        case CHEQUE: return cps;
        case PAYPAL: return ppps;
        default: return null;
    }
}

@Produces @RequestScoped Connection connect(User user) {
    return createConnection(user.getId(), user.getPassword());
}

void close(@Disposes Connection connection) {
    connection.close();
}
```

5.2.4 @Alternative

对于CDI而言，默认系统中可有两个bean有相同的部分（接口或者父类），通过限定符@Qualifier来区分究竟使用哪一个。但是在某些特殊情况下，也可能在系统中某bean需要完全替换另一个bean。

@Alternative替换，可用于标注bean、produce方法、属性以及模板stereotype。这个标注可以在不修改代码结构的基础上，将原有的实现类替换掉。例如有一个ATM实现类需要一个与银行交互接口的实现。默认使用标准接口实现standardTransport。

```
//ATM与银行交互接口
public interface ATMTransport {
    public void communicateWithBank(byte[] datapacket);
}

//ATM与银行交互接口实现
```

```
@Default
public class StandardAtmTransport implements ATMTransport {
    public void communicateWithBank(byte[] datapacket) {
        System.out.println("communicating with bank via Standard transport");
        ...
    }
}

//引用默认实现StandardAtmTransport
public class AutomatedTellerMachineImpl implements AutomatedTellerMachine {
    @Inject
    private ATMTransport transport;
    transport. communicateWithBank(null)
    ...
}
```

如果在后期或者第三方使用时想使用其他方式，比如jsonTransport，但是又不想修改既有的系统代码，这样就可以使用@Alternative。例如：

```
@Alternative
public class JsonRestAtmTransport implements ATMTransport {
    public void communicateWithBank(byte[] datapacket) {
        System.out.println("communicating via Standard transport");
        ...
    }
}
```

还需要在beans.xml中配置：

```
<beans xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/beans_1_0.xsd">
<alternatives>
<class>org.cdi.advocacy.JsonRestAtmTransport</class>
</alternatives>
</beans>
```

这样CDI容器就会加载并使用JsonRestAtmTransport实现。

5.2.5 @Specialize

@Specialize特殊化。相较于@Alternative，它更体现了完全替换。通过@Alternative替换掉的默认

类，如果还有其他标记的话，可以在系统中通过默认类的这些标记注入进来。而@Specialize化的类就不可以了，因为Specialize将继承父类所有的qualifiers甚至包括name。例如：

```
@Soap1 @Specializes
public class SoapAtmTransport1 extends StandardAtmTransport1 implements ATMTransport1
{
    public String communicateWithBank(byte[] datapacket) {
        return "communicateWithBank via SoapAtmTransport1";
    }
}
```

```
@Standard1 @Default
public class StandardAtmTransport1 implements ATMTransport1 {
    public String communicateWithBank(byte[] datapacket) {
        return "communicateWithBank via StandardAtmTransport1";
    }
}
```

SoapAtmTransport1除自身限定符外，还继承了StandardAtmTransport1的限定符@Default和@Standard1。假如StandardAtmTransport1有标记符@Named("standardAtmTransport1")，那这个标记符也会被继承。这样通过standardAtmTransport1来注入的时候，其实注入的是SoapAtmTransport1。但是@Alternative标记的场合通过standardAtmTransport1注入的就会是父类本身了。

5.2.6 @Decorator

@Decorator装饰器，在某些方面与interceptors相似，但是它们是互补型的。相较interceptors可以对日志、安全等进行拦截，decorator更注重业务方面，拦截对某些特定接口的调用，这些可拦截的接口通常被装饰器继承实现某些操作。例如：

```
//特定的java接口
public interface Account {
    public BigDecimal getBalance();
    public User getOwner();
    public void withdraw(BigDecimal amount);
    public void deposit(BigDecimal amount);
}

//装饰器，实现需拦截的特定接口Account
@Decorator
public abstract class LargeTransactionDecorator
    implements Account {
```

```
// Decorator的注入点Delegate，注入它们装饰类型的bean同时加上@Delegate
@Inject @Delegate @Any Account account;
@PersistenceContext EntityManager em;
public void withdraw(BigDecimal amount) {
    ...
}
public void deposit(BigDecimal amount);
...
}
}
```

使用@Decorator，同时需要在beans.xml中配置。

```
<beans
    xmlns="http://java.sun.com/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://java.sun.com/xml/ns/javaee
        http://java.sun.com/xml/ns/javaee/beans_1_0.xsd">
    <decorators>
    <class>org.mycompany.myapp.LargeTransactionDecorator</class>
    </decorators>
</beans>
```

例子中的注入点@Any使得Decorator与所有Account的实现类绑定，如果只想与特定的某实现绑定，可以将@Any替换成某实现自己的限定符。装饰器可以调用任何业务委托对象如Account中的方法。

5.2.7 event

event事件接口，通常为Event<T>，允许应用程序的特定类型T的bean通过fire发送事件给观察者。

T类型通常通过Event<T>的一个实例来fire事件。

```
@Inject @Any Event<LoggedInEvent> loggedInEvent;
```

fire方法接受Event<T>中的LoggedInEvent作为参数，也可以带一些可注入的参数bean。fire方法发送事件给所有观察者。

```
public void login() {
    ...
    loggedInEvent.fire( new LoggedInEvent(user) );
}
```

下面是一个观察者方法，用@Observes event作为事件参数。观察者在收到login事件后执行afterLogin方法。

```
public void afterLogin(@Observes LoggedInEvent event) { ... }
```

第6章 CommonAnnotation

commonAnnotation是由JSR-250 (Common Annotations for the JavaTM Platform) 定义的, 所在包为: javax.annotation.*。部分标记在JEE5中, 如@PostConstruct、@PreDestroy、@Resource(s)、@DeclareRoles、@DenyAll、@PermitAll、@RolesAllowed、@RunAs、@Generated。在JEE6中增加了一部分标记, 比较重要的包括@ManagedBean, 在JSR-316 (JavaTM Platform, Enterprise Edition 6 Specification) 中又专门做了定义。而在JEE7中, commonAnnotation仅增加了@Priority注解用于定义类顺序。

6.1 @ManagedBean

Managedbean最早是在JSF中提出的, 现在JEE6将它引入到规范中。

@ManagedBean可以标记一个简单Java对象为Managedbean, 在该对象满足如下条件的情况下:

1. 必须提供无参数的构造方法(在 CDI 可以宽松这个约束, 允许构造方法有一些参数);
2. 不可是 final、abstract 的类, 不能 static innerclass;
3. 不能序列化, 也就是不能 serializable;
4. 可以有选择的有名字@ManagedBean(“cart”), 也可不选择, 默认为类名且首字母小写;
5. Jndi 名称是唯一的, managed 的名字和 ejb 的名字不能一样;

对于上述的条件, 如果存在CDI容器, 那么这个bean就会被识别为ManagedBean。和简单Java对象通过第一节的DI注入相比, 使用@ManagedBean标记的bean可以通过jndi唯一的访问。一般可以通过三种方式 java:module、java:app、java:global就可以绝对的定位一个ManagedBean。例如定义一个managedBean:

```
@ManagedBean("managedBean")
@RequestScoped
public class MyManagedBean{
    public MyManagedBean(){
    }

    public String getMyManagedBean(){
        ...
    }
}
```

通过Jndi访问:

```
cx.lookup("java:module/managedBean")
```

6.2 @DataSourceDefinition(s)

@DataSourceDefinition(s) 数据源定义, 通过该标记可以定义一个可被jndi查找到的数据源。例如:

- 通过属性定义@DataSourceDefinition数据源:

```
@DataSourceDefinition(name="java:global/MyApp/MyDataSource",
```

```

        className="com.example.MyDataSource",
        portNumber=6689,
        serverName="example.com",
        user="lance",
        password="secret"
    )

```

- 通过url定义@DataSourceDefinition数据源：

```

@DataSourceDefinition(name="java:global/MyApp/MyDataSource",
    className="org.apache.derby.jdbc.ClientDataSource",
    url="jdbc:derby://localhost:1527/myDB",
    user="lance",
    password="secret"
)

```

- 使用数据源：

```

@Stateless
public class MyStatelessEJB {
    @Resource(lookup="java:global/MyApp/myDataSource")
    DataSource myDB;

    ...
}

```

- 使用多个数据源：

```

@DataSourceDefinitions ({
    @DataSourceDefinition(
        //定义参见DataSourceDefinition例子
        ...
    )
    @DataSourceDefinition(
        ...
    )
})

```

以上是对数据源创建、注入（通过@Resource的lookup属性）的简例，通过@Resource的其他方式比如name来注入数据源和通过数据源建立连接创建表等详细处理可以参考
 \${TongWeb_HOME}/samples/common-annotation/commonAnnotationSample.ear示例的使用。

第7章 Interceptor

@Interceptor拦截器是由JSR-318中的Interceptor1.2定义描述的，所在包为javax.interceptor。

Interceptor在JEE5中已经定义，JEE6中增加了一些内容即Interceptor1.1，主要包括@AroundTimeout、@interceptor、@InterceptorBinding。

拦截器执行顺序：

1. 默认的拦截器肯定是第一个进行执行的
2. 对于在方法体中定义的拦截器要比在类中定义的拦截器要先执行
3. 对于通过@Interceptors 定义的拦截器，要按照其顺序执行
4. 超类中的@Interceptors要先执行

对于默认的部署描述文件中定义的拦截器，会覆盖掉原注释定义的拦截器的定义。

7.1 @interceptor

拦截器的作用就是对于一个目标类的方法调用、生命周期事件中，通过定义一个拦截器类，将这个类与目标类进行关联，最终达到拦截的作用。在拦截的过程中，通过在拦截器类中拦截环绕方法中的InvocationContext，进行获取拦截的事件，拦截的调用对象，拦截传入的参数，进行拦截的具体的操作。拦截器需要一个公有无参构造器。例如：

```
@Interceptor
public class AuthorizationInterceptor {
    //通过注入的方式进行
    @Inject @LoggedIn User user;
    @Inject Logger log;
    //拦截器的环绕拦截，获取InvocationContext进行拦截具体操作
    @AroundInvoke
    public Object authorize(InvocationContext ic) throws Exception {
        try {
            if ( !user.isBanned() ) {
                log.fine("Authorized");
                return ic.proceed();
            }
            else {
                log.fine("Not authorized");
                throw new NotAuthorizedException();
            }
        }
        catch (NotAuthenticatedException nae) {
            log.fine("Not authenticated");
            throw nae;
        }
    }
}
```


7.2 @AroundTimeout

通常情况下对于超时的现象有很多方法比如环绕拦截就可能不再执行了，但拦截器的超时拦截可以保证执行，用来记录超时日志或者其他用途。

```
@Interceptor
public class AuthorizationInterceptor {
    //拦截器的超时拦截
    @AroundTimeout
    public void timeoutInterceptorMethod(InvocationContext ctx) {}
}
```

7.3 @InterceptorBinding

用来定义一种interceptor为interceptorBinding类型。通过interceptorBinding可以标注类或者方法。

- InterceptorBind Type定义:

```
@Inherited
@InterceptorBinding
@Target({TYPE, METHOD})
@Retention(RUNTIME)
public @interface Valid {}
```

- 标记interceptor:

```
@Valid @Interceptor
public class ValidationInterceptor { ... }
```

- 用interceptorBinding标记类:

```
@Valid
public class Order { ... }
```

- 如果@Valid中加上一些处理也可以得到数个标记为@Valid的interceptor:

```
@InterceptorBinding
@Target({METHOD, TYPE})
@Retention(RUNTIME)
public @interface Valid {
    boolean requiresNew() default false;
}
```

通过@Valid(requiresNew = true) @Interceptor和@Valid(requiresNew = false)@Interceptor可以

有数个@Valid类型的interceptor。

- 一个InterceptorBindingtype可以用来声明另一个InterceptorBinding:

```
@Inherited
@InterceptorBinding
@Target({TYPE, METHOD})
@Retention(RUNTIME)
@Valid
public @interface Secure {}
```

第8章 Validation

Bean Validation 1.1（JSR-349）是一个由RedHat领导的JSR。该规范的1.0版本（JSR-303）在Java EE 6中引入，提供了对Java EE和Java SE中的Java Bean进行验证的方式，主要使用注解的方式来实现对Java Bean的验证功能，并且这种方式会覆盖使用XML形式的验证描述符，从而使验证逻辑从业务代码中分离出来。

在1.1版本中，改进了与CDI（上下文和依赖注入）的集成，并增加了方法级别的验证，允许参数传递到方法或返回参数以被验证。以及一些小的改进和问题的修复。

8.1 Bean Validation1.0

8.1.1 内置约束

Validation校验是由JSR-303（Bean Validation）定义的，所在包为javax.validation。对bean进行了各种约束，包括规范提供的约束和自定义的约束，还提供了包括非空、长度等约束。具体标注在javax.validation.constraints包中。例如：

```
public class StudentBean {
    @NotNull
    @Min(value = 1)
    @Max(value = 9999)
    private Long id;
    @NotNull
    @Size(min = 1, max = 20, message = "名字的长度必须在 {min} 和 {max} 之间")
    private String name;
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}
```

校验方式：

```
StudentBean student = new StudentBean();
student.setName("student student student");
ValidatorFactory vf = Validation.buildDefaultValidatorFactory();
Validator validator = vf.getValidator();
Set<ConstraintViolation<StudentBean>> set = validator.validateProperty(student, "name");
for (ConstraintViolation<StudentBean> constraintViolation : set) {
    constraintViolation.getMessage();
}
```

```
}
```

如果学生名字超过20位校验不通过，可以获取到错误信息constraintViolation.getMessage()。

8.1.2 自定义约束

规范除了提供内置约束外，还支持自定义约束。创建自定义约束步骤：

创建约束原注释

```
@Target({ METHOD, FIELD, ANNOTATION_TYPE })
@Retention(RUNTIME)
@Constraint(validatedBy = CheckCaseOwnValidator.class)
public @interface CheckCaseOwn{...}
```

创建实现约束校验接口的校验器

```
public class CheckCaseOwnValidator implements
ConstraintValidator<CheckCaseOwn, String>{...}
```

使用原注释

```
@CheckCaseOwn(key = "x", value = yy, message = "{key} must be {value}") String t;
```

8.2 Bean Validation 1.1

1. 更加公开、开放

所有API、源码、测试工具及规范都可以在官网上下载到，也可以发邮件进行讨论。

2. 增加 DI、CDI 的集成

BeanValidation1.1提供一些组件如：MessageInterpolator、TraversableResolver、ParameterNameProvider、ConstraintValidatorFactory和ConstraintValidator。最新的版本也提供了对CDI集成方法的验证功能。

下面是一个比较常见的例子，使用CDI注射约束验证实现：

```
class ZipCodeValidator implements ConstraintValidator<ZipCode, String> {
    @Inject @France
    private ZipCodeChecker checker;
    public void initialize(ZipCode zipCode) {}
    public boolean isValid(String value, ConstraintValidationContext context) {
        if (value==null) return true;
        return checker.isZipCodeValid(value);
    }
}
```

3. 方法验证

Bean Valication1.1允许对方法和构造函数的参数和返回值进行约束。这样Bean Validation API就可以描述和验证应用于给定的方法及构造函数的契约。

- 在调用方法及构造方法之前必须满足先决条件。
- 在方法或构造函数调用返回后向调用方保证的后置条件。

例如，在没有方法验证之前的写法：

```
public User get(Integer uuid) {  
    //前置条件  
    Assert.notNull(uuid);  
    Assert.isTrue(uuid > 0, "uuid must lt 0");  
  
    //获取 User Model  
    Userl user = new User(); //此处应该从数据库获取  
  
    //后置条件  
    Assert.notNull(user);  
    return user;  
}
```

有方法验证后的写法：

```
public @NotNull User get(@NotNull @Size(min = 1) Integer uuid) {  
    //获取 User Model  
    User user = new User(); //此处应该从数据库获取  
    return user;  
}
```

4. 分组验证

如果想在新增的情况下验证id和name，在修改的情况下验证name和password，那么就需要分组了。

- 首先定义分组接口。分组接口就是两个普通的接口，用于标识，类似于Java.io.Serializable。

```
public interface First {  
}  
  
public interface Second {  
}
```

- 之后使用分组接口标识实体：

```
public class User implements Serializable {  
    @NotNull(message = "{user.id.null}", groups = {First.class})  
    private Long id;  
    @Length(min = 5, max = 20, message = "{user.name.length.illegal}", groups = {Se
```

```
cond.class}))
    @Pattern(regexp = "[a-zA-Z]{5,20}", message = "{user.name.illegal}", groups = {
Second.class}))
    private String name;
    @NotNull(message = "{user.password.null}", groups = {First.class, Second.class}))
    private String password;
}
```

- 验证时通过@Validate注解标识要验证的分组；如果要验证两个的话，可以@Validated({First.class, Second.class})。

```
@RequestMapping("/save")
public String save(@Validated({Second.class}) User user, BindingResult result) {
    if(result.hasErrors()) {
        return "error";
    }
    return "success";
}
```

5. 分组顺序

上面提到的user.name会显示两个错误消息，而且顺序不确定。如果先验证一个消息不通过，再验证另一个怎么办？可以通过@GroupSequence指定分组验证顺序：

```
@GroupSequence({First.class, Second.class, User.class})
public class User implements Serializable {
    private Long id;
    @Length(min = 5, max = 20, message = "{user.name.length.illegal}", groups = {First.class})
    @Pattern(regexp = "[a-zA-Z]{5,20}", message = "{user.name.illegal}", groups = {Second.class})
    private String name;
    private String password;
}
```

通过@GroupSequence指定验证顺序：先验证First分组，如果有错误立即返回而不会验证Second分组；如果First分组验证通过了，接着去验证Second分组，最后指定User.class表示那些没有分组的在最后。这样就可以实现按顺序验证分组了。

6. 级联验证

另一个比较常见的就是级联验证，例如：

```
public class User {
```

```
@Valid

@ConvertGroup(from=First.class, to=Second.class)

private Organization o;

}
```

级联验证只要在相应的字段上加@Valid即可，会进行级联验证；@ConvertGroup的作用是当验证o的分组是First时，那么验证o的分组是Second，即分组验证的转换。

7. 消息中使用 EL 表达式

假设需要显示如：用户名[NAME]长度必须在[MIN]到[MAX]之间。通常不希望把一些数据写死，如NAME、MIN、MAX，此时可以使用EL表达式如：

```
@Length(min = 5, max = 20, message = "{user.name.length.illegal}", groups = {First.class})
```

错误消息：

```
user.name.length.illegal=用户名长度必须在{min}到{max}之间
```

其中可以使用{验证注解的属性}得到这些值。如{min}得到@Length中的min值，其他的也类似。

如果想获得出错的那个输入值，如name=zhangsan。此时就需要EL表达式的支持，首先确定引入EL jar包使用，例如：

```
user.name.length.illegal=用户名[${validatedValue}]长度必须在5到20之间
```

使用如EL表达式：\${validatedValue}可得到输入的值，如zhangsan。还可以使用如\${min > 1 ? '大于1' : '小于等于1'}，及在EL表达式中也能拿到如@Length的min等数据。

另外还可以拿到一个java.util.Formatter类型的formatter变量进行格式化：

```
${formatter.format("%04d", min)}
```

第9章 资源应用

9.1 UserTransaction

使用TongWeb的事务管理器，可以通过 javax.transaction.UserTransaction 接口对事务进行管理，UserTransaction 实例不需要创建，可以通过JNDI查找和依赖注入的方式获得。

- JNDI 查找方式

```
InitialContext ic= new InitialContext();
UserTransaction ut = (UserTransaction)ic.lookup("java:comp/UserTransaction");
ut.begin();
datasource = (DataSource)ic.lookup("jdbc/oracle1");
conn = datasource.getConnection();
PreparedStatement prestate=conn.prepareStatement(sql);
prestate.executeUpdate();
conn.close();
ut.commit();
```

- 依赖注入方式

```
@Resource UserTransaction ut;
InitialContext ic= new InitialContext();
ut.begin();
datasource = (DataSource)ic.lookup("jdbc/oracle1");
conn = datasource.getConnection();
PreparedStatement prestate=conn.prepareStatement(sql);
prestate.executeUpdate();
conn.close();
ut.commit();
```

- 使用反射方式

```
Class userTransactionImplClass =
Class.forName("com.tongweb.tongejb.core.CoreUserTransaction");
Object obj = userTransactionImplClass.newInstance();
UserTransaction userTransaction = (UserTransaction)obj;
If(userTransaction != null)
    userTransaction.begin();
```

说明：

本章介绍的用法，可参见 \${TongWeb_HOME}/samples/jta 目录中的 transaction.war。

9.2 TransactionManager

通常，对于用户的应用，我们推荐使用UserTransaction来管理事务，但也有些特殊的应用场景必须使用TransactionManager。

TongWeb7.0为了方便使用TransactionManager，为TransactionManager绑定的JNDI名称为：“tongweb:TransactionManager”，用户可以在应用中lookup这个JNDI名称来获取TransactionManager。

```
InitialContext ic= new InitialContext();
TransactionManager txm=(TransactionManager) ic.lookup("tongweb:TransactionManager ");
if(txm != null){
    txm.begin();
    Transaction tx = txm.getTransaction();
    .....
    txm.commit();
}
```

用户也可以可在应用中lookup如下JNDI名称来获取TransactionManager。



JNDI JNDI树 ⓘ

此页显示了 TongWeb 服务器中绑定的 JNDI 树。

sysweb console heimdall transaction

JNDI 树	JNDI 信息
- app - comp - UserTransaction - TransactionManager - TransactionSynchronizationRegistry - ComponentName - Resources - env - module	JNDI 名: java:comp/TransactionManager JNDI 域: 应用域[sysweb] Java 接口: com.tongweb.tongejb.resource.GeronimoTransactionManagerFactory\$Destroya

示例如下所示。

```
InitialContext ic= new InitialContext();
TransactionManager txm=(TransactionManager) ic.lookup("java:comp/TransactionManager ");
if(txm != null){
    txm.begin();
    Transaction tx = txm.getTransaction();
    .....
    txm.commit();
}
```

说明：本章介绍的用法，可参见TongWeb7/samples/jta目录中的transaction.war。

9.3 TransactionSynchronizationRegistry

TransactionSynchronizationRegistry注入事务同步资源，该接口应用在系统级应用服务器组件上，如：持久化管理、资源适配器、EJB和Web应用组件。

TongWeb7.0为TransactionSynchronizationRegistry绑定的JNDI名称为：

“java:comp/TransactionSynchronizationRegistry”，用户可以在应用中lookup这个JNDI名称来获取TransactionSynchronizationRegistry。

```
InitialContext ic= new InitialContext();
TransactionSynchronizationRegistry txm=( TransactionSynchronizationRegistry)
ic.lookup("java:comp/TransactionSynchronizationRegistry ");
if(txm != null){
    txm.begin();
    Transaction tx = txm.getTransaction();
    .....
    txm.commit();
}
```

说明：本章介绍的用法，可参见TongWeb7/samples/jta目录中的transaction.war。

第10章 JMX

JMX是在TongWeb中用于获取监视量的工具，在TongWeb中使用JMX获取监视量的方法以及JMX对象ObjectName接口的对应表，可参见《008_TongWeb二次开发接口》第2章“JMX使用”。

第11章 REST调用

TongWeb提供了各个内部模块的REST接口，具体的调用方法及各模块接口参数列表，可参见《008_TongWeb二次开发接口》第3章“REST调用”。

第12章 CGI使用

CGI (Common Gateway Interface)为web服务器定义了一种与外部内容生成程序交互的方法，这些程序通常被称为CGI程序或CGI脚本。

CGI支持是使用servlet类`com.tongweb.catalina.servlets.CGIServlet`实现的，通常这个servlet被映射到`"/cgi-bin/*"`。在TongWeb中默认禁用CGI支持，当使用TongWeb作为HTTP服务器并需要CGI支持时，可以添加CGI支持。

12.1 启用CGI

在默认的`${TongWeb_HOME}/conf/default-web.xml`文件中有注释掉的示例CGI servlet。要在web应用程序中启用CGI支持，可将示例CGI servlet复制到web应用程序的WEB-INF/web.xml文件中。部署后的应用在`${TongWeb_HOME}/deployment/`目录下。

取消`${TongWeb_HOME}/conf/default-web.xml`文件中的CGI servlet示例的注释，可以同时为所有已部署的web应用程序启用CGI。

注意：

CGI脚本用于执行TongWebJVM外部的程序。如果正在使用Java SecurityManager，则将绕过在`tongweb.policy`中的安全策略配置。

```

<!-- Common Gateway Includes (CGI) processing servlet, which supports -->
<!-- execution of external applications that conform to the CGI spec -->
<!-- requirements. Typically, this servlet is mapped to the URL pattern -->
<!-- "/cgi-bin/*", which means that any CGI applications that are -->
<!-- executed must be present within the web application. This servlet -->
<!-- supports the following initialization parameters (default values -->
<!-- are in square brackets): -->
<!-- -->
<!--      cgiPathPrefix      The CGI search path will start at -->
<!--                        webAppRootDir + File.separator + this prefix. -->
<!--                        [WEB-INF/cgi] -->
<!-- -->
<!--      debug              Debugging detail level for messages logged -->
<!--                        by this servlet. [0] -->
<!-- -->
<!--      executable         Name of the executable used to run the -->
<!--                        script. [perl] -->
<!-- -->
<!--      parameterEncoding  Name of parameter encoding to be used with -->
<!--                        CGI servlet. -->
<!--                        [System.getProperty("file.encoding","UTF-8")] -->
<!-- -->
<!--      passShellEnvironment Should the shell environment variables (if -->
<!--                        any) be passed to the CGI script? [false] -->
<!-- -->
<!--      stderrTimeout      The time (in milliseconds) to wait for the -->
<!--                        reading of stderr to complete before -->
<!--                        terminating the CGI process. [2000] -->
<!-- -->

<!--
    <servlet>
      <servlet-name>cgi</servlet-name>
      <servlet-class>com.tongweb.catalina.servlets.CGIServlet</servlet-class>
      <init-param>
        <param-name>debug</param-name>
        <param-value>0</param-value>
      </init-param>
      <init-param>
        <param-name>cgiPathPrefix</param-name>
        <param-value>WEB-INF/cgi</param-value>
      </init-param>
      <load-on-startup>5</load-on-startup>
    </servlet>
-->

```

```

<!-- The mapping for the CGI Gateway servlet -->

<!--
    <servlet-mapping>
      <servlet-name>cgi</servlet-name>
      <url-pattern>/cgi-bin/*</url-pattern>
    </servlet-mapping>
-->

```

12.2 Init参数配置

CGI servlet的init参数及说明：

- **cgiMethods**: 用逗号分隔的HTTP方法列表。默认值是GET、POST。使用*作为脚本来处理所有的请求。
- **cgiPathPrefix**: CGI搜索路径将从web应用程序根目录+文件分隔符+这个前缀。默认情况下没有值，这

将导致web应用程序根目录被用作搜索路径。推荐的值是WEB-INF/cgi

- **cmdLineArgumentsDecoded**: 如果命令行参数emnts被启用(通过enableCmdLineArguments), 并且TongWeb在Windows上运行, 那么每个解码的命令行参数都必须匹配此模式, 否则请求将被拒绝。这是为了防止将命令行参数从Java传递到Windows时出现已知问题。这些问题可能导致远程代码执行。
- **cmdLineArgumentsEncoded**: 如果命令行参数emnts被启用(通过enableCmdLineArguments), 则单个已编码的命令行参数必须匹配此模式, 否则请求将被拒绝。默认值与RFC3875定义的允许值匹配, 为[a-zA-Z0-9\Q%;/?:@&,\$-_.!~*'()\E]+
- **enableCmdLineArguments**: 命令行参数是否根据RFC3875的4.4节从查询字符串生成, 默认值为false。
- **environment-variable**: 为CGI脚本的执行环境设置的环境。变量的名称取自参数名称。要配置名为F00的环境变量, 需要配置名为environment-variable-F00的参数。参数值用作环境变量值。默认情况下没有环境变量。
- **executable**: 用于运行脚本的可执行文件的名称。如果你的脚本本身是可执行的(例如, 一个exe文件), 你可以显式地设置这个参数为空字符串。默认是perl。
- **executable-arg-1, executable-arg-2等**: 可执行文件的附加参数。它们位于CGI脚本名称之前。默认情况下没有附加参数。
- **envHttpHeaders**: 一个正则表达式, 用于选择作为环境变量传递给CGI进程的HTTP报头。注意, 标题在匹配之前被转换为大写, 并且整个标题名称必须与模式匹配。默认是: ACCEPT[-0-9A-Z]*|CACHE-CONTROL|COOKIE|HOST|IF-[-0-9A-Z]*|REFERER|USER-AGENT
- **parameterEncoding**: CGI servlet中使用的参数编码的名称。默认是System.getProperty("file.encoding", "UTF-8")。如果系统属性不可用, 则为UTF-8。
- **passShellEnvironment**: 是否应该将TongWeb进程的shell环境变量(如果有的话)传递给CGI脚本?默认是false。
- **stderrTimeout**: 在结束CGI进程之前等待读取stderr完成的时间(以毫秒为单位)。默认是2000。

12.3 配置PHP示例

配置PHP步骤如下:

1. 下载安装 PHP, 此处略。
2. 复制 php.ini-development 到 php.ini, 并编辑 php.ini
3. 去掉 extension_dir = "ext"的注释
4. 修改 cgi.force_redirect 的值为 0, 并去掉注释
5. 配置\${TongWeb_HOME}/conf/default-web.xml 或者应用中 WEB-INF/web.xml

```
<servlet>
    <servlet-name>php</servlet-name>
    <servlet-class>com.tongweb.catalina.servlets.CGIServlet</servlet-class>
    <init-param>
        <param-name>debug</param-name>
        <param-value>0</param-value>
```

```

    </init-param>
    <init-param>
    <param-name>cgiPathPrefix</param-name>
    <param-value>WEB-INF/cgi</param-value>
    </init-param>
  <init-param>
  <param-name>executable</param-name>
  <param-value>F: \php\php-cgi.exe</param-value>
  </init-param>
  <load-on-startup>5</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>php</servlet-name>
<url-pattern>/phpbin/*</url-pattern>
</servlet-mapping>

```

6. 将 index.php 等 php 文件放入应用的 WEB-INF/cgi 目录下（部署后的应用在 \${TongWeb_HOME}/deployment/目录下）
7. 访问 http://ip:port/应用名/phpbin/index.php

第13章 WebDAV使用

WebDAV (Web-based Distributed Authoring and Versioning)，它是一组HTTP协议的扩展，允许用户在远程web服务器上协作编辑和管理文件。当使用TongWeb作为HTTP服务器并需要WebDAV支持时，可以添加WebDAV支持。

WebDAV支持是使用servlet类com.tongweb.catalina.servlets.WebdavServlet实现的。

13.1 启用WebDAV

注意：

WebdavServlet不能作为默认servlet使用(即映射到'/')，因为它不能在这个配置中工作。

1. 启用WebDAV需要修改应用程序的WEB-INF/web.xml文件。部署后的应用在
\${TongWeb_HOME}/deployment/目录下。

```
<servlet>
<servlet-name>webdav</servlet-name>
<servlet-class>com.tongweb.catalina.servlets.WebdavServlet</servlet-class>
<init-param>
<param-name>debug</param-name>
<param-value>0</param-value>
</init-param>
<init-param>
<param-name>listings</param-name>
<param-value>true</param-value>
</init-param>
</servlet>
<servlet-mapping>
<servlet-name>webdav</servlet-name>
<url-pattern>/webdav/*</url-pattern>
</servlet-mapping>
```

2. 以上配置将启用只读访问。若要启用读写访问，请添加：

```
<init-param>
<param-name>readonly</param-name>
<param-value>false</param-value>
</init-param>
```

3. 如果要使内容可通过不同的URL编辑，使用以下映射：

```
<servlet-mapping>
```

```
<servlet-name>webdav</servlet-name>  
<url-pattern>/webdavedit/*</url-pattern>  
</servlet-mapping>
```

4. 默认情况下，通过WebDAV无法访问/WEB-INF和META-INF。要访问这些url，请使用：

```
<init-param>  
<param-name>allowSpecialPaths</param-name>  
<param-value>true</param-value>  
</init-param>
```


第14章 NativeJdbcExtractor工具类

使用NativeJdbcExtractor工具类可通过TongWeb代理连接获取真实数据库连接对象，用法如下：

```
try {  
    DataSource dataSource = (DataSource) new InitialContext().lookup("jdbc/test");  
    Connection con = dataSource.getConnection();  
    System.out.println("逻辑连接: " + con);  
    Connection physicalconnection =  
NativeJdbcExtractor.extractNativeConnection(con);  
    System.out.println("物理连接: " + physicalconnection);  
    con.close();  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

附录A 术语及缩略语

名称	说明
TongWeb (简称TW)	东方通应用服务器
<code>\${TongWeb_HOME}</code>	TongWeb的安装路径
AJP	Apache JServ Protocol, 定向包协议
CDI	Contexts Dependency Inject, 上下文和依赖注入
CGI	Common Gateway Interface, 通用网关接口
EJB	Enterprise JavaBeans, 基于分布式事务处理的企业级应用
JDBC	Java Database Connectivity, Java数据库连接
JPA	Java Persistence API, Java持久化API
JSP	JavaServer Pages, Java服务器端页面
JSF	Java Server Faces, 构建Java Web应用程序的标准框架
JVM	Java Virtual Machine, Java虚拟机
SNMP	简单网络管理协议
URL	Uniform Resource Locator, 统一资源定位系统
WebDAV	Web-based Distributed Authoring and Versioning, Web分布式创作和版本管理

